

Componentes/Interface/STIImportSale.PRW

804 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "STIIMPORTSALE.CH"
6
7 #DEFINE CTXTSEL "[SELECIONADO] - "
8
9 Static oButton      := Nil //Objeto do button
10 Static oOrcList     := Nil //ListBox Seleção de Orçamento
11 Static aOptionSales := {} //Armazena opções de irçamento buscadas
12 Static aViewOptionsSales:= {} //Armazena opções de pagamento para apresentação e seleção na view
13 Static oSay3       := Nil //Label da importacao
14 Static aGetOrc     := {} //Grava o orcamento que esta sendo importado
15 Static oSayVend    := Nil //Objecto para exibir o vendedor do Orçamento
16 Static oSayCli     := Nil //Objecto para exibir o código do cliente do Orçamento
17 Static lCheck      := .F. //VARIável para controlar se um Orçamento foi marcado/desmarcado
18 Static lIsImpDes    := .F. //Indica se contem importação de orçamento.
19
20 //-----
21 /*{Protheus.doc} STIImportSale
22 Interface Importação de Orçamento
23
24 @param
25 @return Nil
26 @obs
27 @sample
28 */
29 //-----
30 Function STIImportSale()
31 Local cMsg      := ""
32 Local lVPPProd  := SuperGetMv("MV_LJVPPO",,.F.) //Permite vender Vale Presente e produtos na mesma venda
33 Local lContinua := .T.
34
35 //Tratativa para LGPD
36 If ExistFunc("LjPDACesso")
37     lContinua := LjPDACesso({"L1_NOMCLI","L1_CGCCLI"}) //Verifica se o usuário pode acessar uma determinada rotina devido a regra de proteção de dados.
38 EndIf
39
40 If lContinua
41
42     lContinua := !STBExistVP() .OR. lVPPProd
43
44     If lContinua
45
46         lContinua := STBISMultSales() //Valida se esta liberado para importar orcamento
47
48         //Valida se esta liberado para importar orcamento
49         If lContinua
50             STIExchangePanel( { || STIPanImportSale(@cMsg) } )
51             STIChangeCssBtn('oBtnOrc')
52

```

```

53         If !Empty(cMsg)
54             STFMessage("STIImportSale", "STOP", cMsg)
55             STFShowMessage("STIImportSale")
56             LjGrvLog("Importa_Orcamento:STIImportSale", cMsg)
57         EndIf
58     Else
59         STFMessage("STIImportSale", "STOP", STR0008 ) // "Só é possível importar um orçamento por venda"
60         STFShowMessage("STIImportSale")
61         LjGrvLog("Importa_Orcamento:STIImportSale", STR0008)    //"Só é possível importar um orçamento por venda"
62     EndIf
63 Else
64     STFMessage(ProcName(0),"STOP",STR0015) //"A venda do vale presente deve ser feita separada."
65     STFShowMessage(ProcName(0))
66     LjGrvLog("Importa_Orcamento:STIImportSale", STR0015)    //"A venda do vale presente deve ser feita separada."
67 EndIf
68
69 EndIf
70
71 Return Nil
72
73 //-----
74 /*/{Protheus.doc} STIPanImportSale
75 Contexto Importação de Orçamento
76
77 @param
78 @return  oPanGetImportSale  Painel Importação de Orçamento
79 @obs
80 @sample
81 */
82 //-----
83 Function STIPanImportSale(cMsg)
84
85 Local oPanGetImportSale := Nil
86 Local oGet1              := Nil           // Get Principal
87 Local oGet2              := Nil           // Get Adicional
88 Local cGet1              := ""           // Variável do get de busca principal
89 Local cGet2              := ""           // Variável do Get de busca adicional
90 Local cField1            := ""           // Campo de busca utilizado
91 Local cField2            := ""           // Campo de busca adicional
92 Local oPanelMVC          := STIGetPanel() // Painel principal
93 Local aField1            := {}           // Armazena informação do campo de busca de orçamento
94 Local aField2            := {}           // Armazena informação do campo adicional de busca de orçamento
95 Local nGetHeiPerc        := 90           // Percentual da Altura do Get de Busca
96
97 Default cMsg := ""
98
99 /* Panel Get do Produto */
100 oPanGetImportSale := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight/2))
101 oPanGetImportSale:SetCSS( POSCSS (GetClassName(oPanGetImportSale), CSS_PANEL_CONTEXT ))
102
103 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanGetImportSale,,,,,T,,,,) //"Vendas > Orçamento"
104 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))

```

```

105
106
107 /* Campo de busca principal*/
108 cField1 := AllTrim(SuperGetMv("MV_LJORCAM ","")) // Campo de busca principal
109
110 If Empty(cField1)
111     cField1 := IIF(STFGetCfg("lPafEcfc") .OR. STFGetCfg("lPAFNFCE"),"L1_NUMORC","A1_CGC")
112 EndIf
113
114 aField1 := STDISSearchField(cField1)
115
116 /* Campo de busca adicional*/
117 If !STFGetCfg("lPafEcfc") .OR. SuperGetMv("MV_LJORCAA ","") <> ""
118
119     cField2 := AllTrim(SuperGetMv("MV_LJORCAA ","L1_NUM")) // Campo de busca adicional
120     aField2 := STDISSearchField(cField2)
121
122 EndIf
123
124 If Len(aField1) > 0
125
126     /* Primeiro Get */
127     oSay3 := TSay():New(POSVERT_LABEL1,POSHOR_1,{||aField1[1]},oPanGetImportSale,,,,,T,,,LARG_SAY,)
128     oSay3:SetCSS( POSCSS (GetClassName(oSay3), CSS_LABEL_FOCAL ))
129
130     cGet1 := Space(aField1[5])
131
132     oGet1 := TGet():New(POSVERT_GET1,POSHOR_1,{||u| If(PCount())>0,cGet1:=u,cGet1}}, oPanGetImportSale,nGetHeiPerc,,,/*bValid*/,,,,,T,,,,,,,,,"cGet1")
133     oGet1:SetCSS( POSCSS (GetClassName(oGet1), CSS_GET_FOCAL ))
134
135     oGet1:bLostFocus := {|| Iif(ValType(oButton)== "0",oButton:SetFocus(),Nil) } //Tratamento para não perder o foco do Buscar Orçamento
136
137     oGet1:SetFocus()
138
139     If GetFocus() <> oGet1:HWND
140         oGet1:SetFocus()
141     Endif
142
143     If Len(aField2) > 0
144
145         /* Get Parâmetro */
146         oSay2 := TSay():New(POSVERT_LABEL2,POSHOR_1,{||aField2[1]},oPanGetImportSale,,,,,T,,,LARG_SAY,)
147         oSay2:SetCSS( POSCSS (GetClassName(oSay2), CSS_LABEL_FOCAL ))
148
149         cGet2 := Space(aField2[5])
150
151         oGet2 := TGet():New(POSVERT_GET2,POSHOR_1,{||u| If(PCount())>0,cGet2:=u,cGet2}}, oPanGetImportSale,nGetHeiPerc,,,/*bValid*/,,,,,T,,,,,,,,,"cGet2")
152         oGet2:SetCSS( POSCSS (GetClassName(oGet2), CSS_GET_FOCAL ))
153
154         /* Sem When*/
155         oGet1:bLostFocus := {|| IIf( !Empty(cGet1) , ;
156                                     ( ;

```

```

157             IIf( !Empty(cGet2) , ;
158                 Eval( {|| cGet2 := Space(Len(cGet2)) , ;
159                     STFMessage("STIImportSale", "STOP", STR0009) , ; //"Só é possível buscar por 1
        campo preenchido"
160                         STFShowMessage("STIImportSale") } ) ;
161                     , ) , ;
162                     oButton:SetFocus() ;
163                 ) ;
164                 ,IIf( Empty(cGet2) , ;
165                     oGet2:SetFocus(),;
166                     ) ) }
167             oGet2:LostFocus := {|| IIf( !Empty(cGet2) , ;
168                 ( ;
169                     IIf( !Empty(cGet1) , ;
170                         Eval( {|| cGet1 := Space(Len(cGet1)) , ;
171                             STFMessage("STIImportSale", "STOP", STR0009) , ; //"Só é possível buscar por 1
        campo preenchido"
172                                 STFShowMessage("STIImportSale") } ) ;
173                             , ) ;
174                             ,oButton:SetFocus() ;
175                         );
176                         , IIf( Empty(cGet1) , ;
177                             oGet1:SetFocus(),;
178                             ) ) }
179             EndIf
180
181
182             oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0002,oPanGetImportSale,{ || STIISearch( oPanGetImportSale, oPanelMVC , cField1, cGet1 , cField2 , cGet2 )
183             }, ; //"Buscar Orçamento"
184                 LARGBTN,ALTURABTN,,,.T.)
185             oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
186         Else
187             cMsg := STR0011 //"Erro na configuração de busca. Verificar parâmetro MV_LJORCAM."
188         EndIf
189     EndIf
190
191     Return oPanGetImportSale
192
193
194     //-----
195     /*{Protheus.doc} STIISearch
196     Busca de Orçamento
197
198     @param    cGet2
199     @param    oPanGetImportSale
200     @param    oPanelMVC
201     @return
202     @obs
203     @sample
204     */
205     //-----

```

```

206 Function STIISearch( oPanGetImportSale, oPanelMVC , cField1, cGet1 , cField2 , cGet2 )
207
208 Local aAux           := {}           // Armazena retorno função temporariamente
209 Local lUserSelect     := .F.         // Indica se orçamento deve ser selecionado
210 Local nLisHorPerc     := 0.039       // Percentual de Posição Horizontal do ListBox
211 Local nLisHeiPerc     := 0.27       // Percentual de Altura do ListBox
212 Local aListOrc        := {}         // Lista de orçamentos
213 Local cTextLabel      := "Codigo Orc.   - Cliente" + Space(43) + "   - Valor" // Label para exibicao do listBox //"Codigo Orc.   -   Cliente   -   Valor"
214 Local nListCheck      := 0          //Posição do Orçamento selecionado
215 Local nList           := 0          //Variável de controle
216
217 STFCleanInterfaceMessage()
218
219 If Empty(cGet1) .AND. Empty(cGet2)
220
221     STFMessage("STIImportSale", "STOP", STR0010 ) //"Campo de busca deve ser preenchido"
222     STFShowMessage("STIImportSale")
223     LjGrvLog("Importa_Orcamento:STIImportSale",STR0010) //"Campo de busca deve ser preenchido"
224
225 Else
226
227     /* Chamada Busca de opções de orçamento */
228     If !Empty(cGet1)
229         MsgRun(STR0016,'Aguarde....',{||aAux := STWISSearchOptions( cField1 , cGet1 )}) // "Buscando Orcamentos"
230     Else
231         MsgRun(STR0016,'Aguarde....',{||aAux := STWISSearchOptions( cField2 , cGet2 )}) // "Buscando Orcamentos"
232     EndIf
233
234     lUserSelect     := aAux[1]
235     aOptionSales     := aAux[2]
236
237     /*
238     Removido validação para importar orçamento direto quando era realizado
239     busca através do número do orçamento. Sistema esta apresentando erro ao
240     importar um orçamento filho (venda com retira posterior) gerado na retaguarda.
241     */
242
243     /* Formata o array para exibicao do listBox */
244
245     aListOrc := STIISFormOrc(aOptionSales)
246
247     /* Alimenta o array aViewOptionsSales */
248     STIISSetViewOptions( aOptionSales )
249
250     If STFGetCfg("lPafEcf") .OR. STFGetCfg("LPAFNFCF")
251
252         If SuperGetMv("MV_LJPRVEN",,.T.)
253             cTextLabel := STR0012 // "Pré-Venda   /   Cliente   /   Valor"
254         Else
255             cTextLabel := STR0013 // "DAV       /   Cliente   /   Valor"
256         EndIf
257

```

```

258 EndIf
259
260 oSay3:SetText( cTextLabel )
261
262 oSay3:CtrlRefresh()
263
264 /* Cria listBox com os orcamentos para selecionar */
265
266 oOrcList := TListBox():New( POSVERT_GET1,POSHOR_1,{|u|if(Pcount(>0,nList:=u,nList)},aListOrc,;
267 ( oPanelMVC:nWidth/2 - (2*nLisHorPerc*oPanelMVC:nWidth/2)),oPanelMVC:nHeight/2 * nLisHeiPerc,;
268 nil,oPanGetImportSale,/*[ bValid ]*/, Nil , Nil, .T., Nil, ;
269 {|| STIMarkImport(@nListCheck), lCheck:=.T.},,Nil, Nil,/* [ bWhen ]*/, Nil, Nil, Nil,Nil, /*[ bRClicked ]*/ )
270
271 If !STFGetCfg("lPafEcF") .AND. !STFGetCfg("lPafNFCE")
272   oSayVend := TSay():New(POSVERT_GET1+70 ,POSHOR_1,{|"Vendedor : " },oPanGetImportSale,,,,,.T.,,300,)
273   oSayVend:SetCSS( POSCSS (GetClassName(oSayVend), CSS_BREADCUMB ))
274   oSayCli := TSay():New(POSVERT_GET1+80 ,POSHOR_1,{|"Cód.Cliente: " },oPanGetImportSale,,,,.T.,,300,)
275   oSayCli:SetCSS( POSCSS (GetClassName(oSayCli), CSS_BREADCUMB ))
276
277   oOrcList:bChange:= {|| iIF(!lCheck, STIExibeInf(aOptionSales[nList]), (lCheck:=.F.,oOrcList:Select(nListCheck))) }
278 EndIf
279
280 oOrcList:SetCSS( POSCSS (GetClassName(oOrcList), CSS_LISTBOX ))
281 oOrcList:SetFocus()
282
283 oBtn := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0005,oPanGetImportSale,{ || MsgRun(STR0017,'Aguarde.....', {||STIFinishImp()}), Iif(ExistFunc("STIMataObj"),
284   STIMataObj(oPanGetImportSale), Nil) }, ; /*"Importar" /*"Importante Orcamento Selecionado"
285   LARGBTN,ALTURABTN,,,,.T.)
286 oBtn:SetCSS( POSCSS( GetClassName(oBtn), CSS_BTN_FOCAL ) )
287 EndIf
288
289 Return
290
291 //-----
292 /*{Protheus.doc} STIISetSales
293 Seta Orcamentos a importar
294
295 @param aAllSales
296 @return
297 @obs
298 @sample
299 */
300 //-----
301 Function STIISetSales( aAllSales )
302
303 Default aAllSales := {}
304
305 ParamType 0 Var aAllSales As Array Default {}
306
307 aSetSale := aAllSales
308

```

```

309 Return
310
311 //-----
312 /*{Protheus.doc} STIISFormOrc
313 Formata os orcamentos para apresentacao na tela
314
315 @param  aOrcs
316 @return
317 @obs
318 @sample
319 */
320 //-----
321 Function STIISFormOrc( aOrcs )
322
323 Local aListOrc      := {}           //Retorno dos orcamentos
324 Local nI            := 0           //Variavel de loop
325 Local nJ            := 0           //Variavel Loop
326 Local nZ            := 0           //Contador
327 Local cOrc         := ""          //String a ser apresentada no LisBox
328 Local aFieldsAdd    := STIISFieldsAdd() //Campos a serem apresentados na interface
329 Local aFieldsInt     := STIISParamFields(.F.) //Campos a apresentar na interface
330 Local aFieldValue   := {}         //Corresponcia campo valor entre interface e o resultado da busca
331 Local nPos          := 0
332 Local oCesta        := STDGPBModel()
333 Local cNumOrc       := ""
334 Local aPEOrcs       := {}
335
336 For nZ := 1 To Len(aFieldsInt)
337
338     Do Case
339
340         Case aFieldsInt[nZ] == "L1_NUM"
341             AADD( aFieldValue , { "L1_NUM" , 1 } )
342         Case aFieldsInt[nZ] == "L1_NUMORC"
343             AADD( aFieldValue , { "L1_NUMORC" , 2 } )
344         Case aFieldsInt[nZ] == "A1_NOME"
345             AADD( aFieldValue , { "A1_NOME" , 3 } )
346         Otherwise
347             AADD( aFieldValue , { aFieldsInt[nZ] , AScan( aFieldsAdd , { |x| x == aFieldsInt[nZ] } ) + 3 } ) // Adiciona 3 pq os 3 primeiros são padrão:= NUM,NUMOR,NOME
348
349     EndCase
350
351 Next nZ
352
353 // Busca se ja foi importado ja algum orçamento, caso sim elimina do array
354 If STDPBLength("SL2") > 0
355     oCesta := oCesta:GetModel("SL2DETAIL")
356     For nZ := 1 To STDPBLength("SL2")
357         oCesta:GoLine(nZ)
358         If !oCesta:IsDeleted()
359             cNumOrc := STDGPBasket( "SL2" , "L2_NUMORIG" , nZ )
360             nPos := AScan( aOrcs,{ |x| Alltrim(x[1]) == Alltrim(cNumOrc) } )

```

```

361         If nPos > 0
362             ADel(a0rcs,nPos)
363             ASize(a0rcs,len(a0rcs)-1)
364         EndIf
365     EndIf
366 Next nZ
367 EndIf
368
369 If ExistBlock("STORDIIMPORT")//Ordena os orçamentos a serem importados.
370     aPE0rcs := ExecBlock("STORDIIMPORT",.F.,.F.,{a0rcs})
371     If ValType(aPE0rcs) == "A"
372         a0rcs := aClone(aPE0rcs)
373     EndIf
374 Endif
375
376 For nI := 1 To Len(a0rcs)
377
378     c0rc := ""
379
380     For nJ := 1 To Len(aFieldsInt)
381
382         nPos := AScan( aFieldValue , { |x| x[1] == aFieldsInt[nJ] } )
383
384         If aFieldsInt[nJ] == "L1_VLRTOT"
385             c0rc := c0rc + "R$ " //Colocar simbolo moeda
386             c0rc := c0rc + PADL(AllTrim(Str(Val(STIISConvertToStr(a0rcs[nI][aFieldValue[nPos][2]])),16,2)), 10)
387         ElseIf aFieldsInt[nJ] == "A1_NOME"
388             c0rc := c0rc + padr(AllTrim(STIISConvertToStr(a0rcs[nI][aFieldValue[nPos][2]])) , 50)
389         Else
390             c0rc := c0rc + STIISConvertToStr(a0rcs[nI][aFieldValue[nPos][2]])
391         EndIf
392
393         If nJ == Len(aFieldsInt)
394
395             AADD( aOptionSales[nI] , c0rc )
396             AADD( aList0rc , c0rc )
397         Else
398
399             c0rc := c0rc + ' - '
400
401         EndIf
402     Next nJ
403 Next nI
404
405 Next nI
406
407 Return aList0rc
408
409
410 //-----
411 /*{Protheus.doc} STIISSetViewOptions
412 Seta opções de orçamento para carregar no modelo depois de ativo

```

```

413
414 @param  aOptionSales  Opções de orçamento buscados da retaguarda
415 @return
416 @obs
417 @sample
418 /*/
419 //-----
420 Function STIISetViewOptions( aOptionSales )
421
422 Default aOptionSales      := {}
423
424 ParamType 0 Var aOptionSales As Array Default {}
425
426 aViewOptionsSales := aOptionSales
427
428 Return .T.
429
430
431 //-----
432 /*/{Protheus.doc} STIISAfterSelected
433 Seta opções de orçamento para carregar no modelo depois de ativo
434
435 @param  aOptionSales  Opções de orçamento buscados da retaguarda
436 @return
437 @obs
438 @sample
439 /*/
440 //-----
441 Function STIISAfterSelected()
442
443 Local lRet           := .T.           // Retorno função
444 Local nI             := 0            // Variavel de loop
445 Local aSelectedSales := {}
446 Local lContinua     := .T.
447 Local lCFRetPos     := .F.           // Indica se faz somente a impressao do Cupom Fiscal devido ser uma finalizacao de venda com itens de "RETIRA
    POSTERIOR"
448 Local nPosImpOrc    := 0            // Linha do orçamento na qual esta posicionado
449 Local aImported     := {}
450 Local nDescVend     := 0            // Desconto no total da venda
451 Local aSL1          := {}
452
453 If Len(aViewOptionsSales) > 0
454
455     nPosImpOrc := oOrcList:GetPos() //Posição atual
456
457     if GetCliHTML()
458         oOrcList:GoTop()
459     Endif
460
461     For nI := 1 To Len(aViewOptionsSales)
462         oOrcList:Select(nI) //Posiciona na linha
463         If CTEXTSEL $ oOrcList:GetSelText()

```

```

464         aAdd(aSelectedSales, {aViewOptionsSales[oOrcList:GetPos(),1], aViewOptionsSales[oOrcList:GetPos(),2],;
465                               aViewOptionsSales[oOrcList:GetPos(),3], aViewOptionsSales[oOrcList:GetPos(),4],;
466                               aViewOptionsSales[oOrcList:GetPos(),5]}) // Armazena opções de orçamento selecionadas
467     EndIf
468 Next
469
470 If Len(aSelectedSales) == 0
471     oOrcList:Select(nPosImpOrc) //Posiciona na linha em que estava posicionado
472     aSelectedSales := {{ aViewOptionsSales[oOrcList:GetPos(),1], aViewOptionsSales[oOrcList:GetPos(),2],;
473                         aViewOptionsSales[oOrcList:GetPos(),3], aViewOptionsSales[oOrcList:GetPos(),4],;
474                         aViewOptionsSales[oOrcList:GetPos(),5] }} // Armazena opções de orçamento selecionadas
475 EndIf
476
477 EndIf
478
479 /* Validações */
480 If Len(aSelectedSales) == 0
481
482     lRet := .F.
483     STFMessage("STIIImportSale", "STOP", STR0006 ) //"Nenhum orçamento foi selecionado"
484     STFS showMessage("STIIImportSale")
485     LjGrvLog("Importa_Orcamento:STIISAfterSelected",STR0006)    //"Nenhum orçamento foi selecionado"
486
487 EndIf
488
489 /* Realizar importação */
490 If lRet
491
492     /* Get dos orçamentos a partir das opções selecionadas */
493     aImported := STWISImpAllSelected( aSelectedSales , "S" )
494     lContinua := aImported[1]
495
496     IF lContinua .AND. len (aImported[5])>0 //Existe orçamento com erro e sem erro.
497         STFMessage("STIIImportSale", "YESNO", aImported[3]) //"Confirma a impressão do cupom fiscal?"
498         lContinua := STFS showMessage("STIIImportSale")
499     ElseIf !lContinua
500         LjGrvLog("Importa_Orcamento:STIISAfterSelected","aImported  : ", aImported)
501     EndIf
502
503     LjGrvLog("Importa_Orcamento:STIISAfterSelected","lContinua  : ", lContinua)
504
505     If lContinua
506         /* Verifica se é finalizacao de venda de orcamento somente com itens de "RETIRA POSTERIOR", para imprimir o Cupom Fiscal e Finalizar. */
507         lCFRetPos := STBISRetPos(aImported[4][1][1])
508
509         If lCFRetPos
510             STFMessage("STIIImportSale", "YESNO", STR0014) //"Confirma a impressão do cupom fiscal?"
511             lContinua := STFS showMessage("STIIImportSale")
512         EndIf
513     EndIf
514
515     If lContinua          // Continua?

```

```

516
517     If lCFRetPos
518
519         STIAltTipo(aImported[4][1][1])
520
521
522
523         LjGrvLog("Importa_Orcamento:STIISAfterSelected", "Chama a importação de Orçamento filho com apenas itens de Retira Posterior")
524         //Chama a importação de Orçamento filho com apenas itens de Retira Posterior" (Imprime o Cupom direto e finaliza)
525         STBISFinPos(aImported[4][1])
526
527     Else
528
529         /* Guarda o orcamento que esta sendo importado */
530         aGetOrc := aImported[4]
531         aSL1     := aGetOrc[1][1]
532         //Verifico se tem permissão para alterar a forma de pagamento
533         If STFGetCfg("lChangePay", .F.)
534             If MsgYesNo(STR0019,STR0020) //Deseja alterar as formas de pagamento?/Atenção
535                 STIChangePay(@aGetOrc[1][3])
536             EndIf
537         EndIf
538
539         /* Cadastra cliente na base local caso não exista */
540         STBCadCli(aGetOrc)
541
542         /* Chamar Importação direto */
543         STWISRegSale( aImported[4] ) // aAllSales
544
545         /* Atualiza interface */
546         STIGridCupRefresh()
547
548         /* Pagamento */
549         //Nao utilizar nesse ponto o comando STICallPayment. Motivo: Utilizando o comando STWBtFimVnd o fluxo padrao para solicitar o CPF será respeitado
550         STWBtFimVnd() //Rotina com tratamento padrao para finalizar a venda
551
552         /*Localizamos se tem desconto na venda se tiver mantemos o que foi negociado*/
553         nDescVend := aSL1[AScan( aSL1 , { |x| x[1] == "L1_DESCONT" } )][2]
554         Iif(nDescVend > 0, lIsImpDes := .T., lIsImpDes)
555
556         STIISSetViewOptions() // Zera statica
557
558     EndIf
559 Else
560     If ExistFunc("STWGetMsg") .AND. !Empty(STWGetMsg())
561         STFMessage("STIImportSale", "POPUP", STWGetMsg() )
562         LjGrvLog("Importa_Orcamento:STIISAfterSelected", STWGetMsg())
563     Else
564         STFMessage("STIImportSale", "ALERT", STR0007 ) //"Não foi possível importar o orçamento"
565         LjGrvLog("Importa_Orcamento:STIISAfterSelected",STR0007) //"Não foi possível importar o orçamento"
566     EndIf
567

```

```

568      STFS showMessage( "STIImportSale")
569      lRet := .F.
570
571      EndIf
572
573 EndIf
574
575
576 Return(lRet)
577
578 //-----
579 /*/{Protheus.doc} STIISFieldsAdd
580 Retorna campos adicionais que foram configurados a aparecer na view
581
582 @param
583 @return aFieldsAdd Campos adicionais configurados a aparecer na view
584 @obs
585 @sample
586 */
587 //-----
588 Function STIISFieldsAdd()
589
590 Local aFieldsAdd      := {}           // Retorno função
591 Local aConfFields     := STIISParamFields(.T.) // Armazena todos os campos configurados para aparecer na interface
592 Local nI              := 0           // Contador
593
594 For nI := 1 To Len(aConfFields)
595
596     If !( aConfFields[nI] $ "L1_NUM|L1_NUMORC|A1_NOME" )
597
598         AADD( aFieldsAdd , aConfFields[nI] )
599
600     EndIf
601
602 Next nI
603
604 Return(aFieldsAdd)
605
606 //-----
607 /*/{Protheus.doc} STIFinishImp
608 Chama a rotina para pesquisar os orcamentos
609
610 @param
611 @return
612 @obs
613 @sample
614 */
615 //-----
616 Function STIFinishImp()
617
618 If !STFGetCfg("lPafEcf") .AND. !STFGetCfg("lPafNFCE")
619     oSayCli:lVisible:= .F.

```

```

620      oSayVend: lVisible:= .F.
621 Endif
622
623 STIISAfterSelected()
624
625 Return .T.
626
627 //-----
628 /*{Protheus.doc} STIGetOrc
629 Retorna o orcamento importado
630
631 @param
632 @return
633 @obs
634 @sample
635 */
636 //-----
637 Function STIGetOrc()
638 Return aGetOrc
639
640
641 //-----
642 /*{Protheus.doc} STIISConvertToStr
643 Converte valor passado para caractere
644
645 @param  xValeu          Valor a ser convertido
646 @return
647 @obs
648 @sample
649 */
650 //-----
651 Function STIISConvertToStr( xValue )
652
653 Do Case
654     Case Valtype(xValue) == "N"
655         cRet := AllTrim(Str(xValue))
656     Case Valtype(xValue) == "C"
657         cRet := xValue
658     Case Valtype(xValue) == "D"
659         cRet := DTOC(xValue)
660     Case Valtype(xValue) == "L"
661         If xValue
662             cRet := "T"
663         Else
664             cRet := "F"
665         EndIf
666     Otherwise
667         cRet := xValue
668 EndCase
669
670 Return cRet
671

```

```

672 //-----
673 /*{Protheus.doc}
674 Retorna campos que vao aparecer na interface
675
676 @param
677 @return aRetFields Campos aparecer interface
678 @obs
679 @sample
680 */
681 //-----
682 Function STIISParamFields(lBusca)
683 Local lNotPAF := !STFGetCfg("lPafEcf") .AND. !STFGetCfg("lPafNFCE")
684 Local cParam := Iif(lBusca,"L1_NUM/A1_NOME/L1_VLRTOT/L1_DESCONT/L1_JUROS" + IIF(SL1->(ColumnPos("L1_VLRJUR")) > 0 , "/L1_VLRJUR", "" )+ Iif(lNotPAF,
"/L1_VEND/L1_CLIENTE" , "" ), "L1_NUM/A1_NOME/L1_VLRTOT")//CAMPOS PARA A BUSCA E PARA A INTERFACE
685 Local aRetFields := STRTOKARR ( cParam , "/" ) // Retorno Função
686
687 If STFGetCfg("lPafEcf") .OR. STFGetCfg("lPafNFCE")
688   If AScan( aRetFields , { |x| x == "L1_NUM" } ) > 0
689     aRetFields[AScan( aRetFields , { |x| x == "L1_NUM" } )] := "L1_NUMORC"
690   EndIf
691 Else
692   If AScan( aRetFields , { |x| x == "L1_NUMORC" } ) > 0
693     aRetFields[AScan( aRetFields , { |x| x == "L1_NUMORC" } )] := "L1_NUM"
694   EndIf
695 EndIf
696
697 Return aRetFields
698
699 //-----
700 /*{Protheus.doc}
701 Função para limpar variavel estatica
702
703 @param
704 @return .T.
705 @obs
706 @sample
707 */
708 //-----
709 Function STIClrVar()
710
711 If Len(aGet0rc) > 0
712   aGet0rc := {}
713 EndIf
714
715 Return .T.
716
717 //-----
718 /*{Protheus.doc} STIMarkImport
719 Função marcar seleção de multi orçamentos
720
721 @param
722 @return .T.

```

```

723 @obs
724 @sample
725 */
726 //-----
727 Static Function STIMarkImport(nListCheck)
728 Local cTxtList := ""
729 Local nMax      := 0
730
731 If !SuperGetMv( "MV_LJMLTOR" , NIL , .T. ) //Valida se esta liberado para importar orcamento
732   STFMessage(ProcName(), "ALERT", STR0018 ) //"Seleção de multiplos orçamentos desabilitada(MV_LJMLTOR)"
733   STFShowMessage(ProcName())
734   LjGrvLog("Importa_Orcamento:STIMarkImport", STR0018)          //"Seleção de multiplos orçamentos desabilitada(MV_LJMLTOR)"
735 Else
736   If oOrcList <> Nil
737     cTxtList := oOrcList:GetSelText()
738     If CTXTSEL $ cTxtList //retira a seleção.
739       nMax := RAT(CTXTSEL,cTxtList)
740       oOrcList:Modify( SubStr(cTxtList, nMax+LEN(CTXTSEL)), oOrcList:GetPos() )
741     Else //coloca a seleção
742       oOrcList:Modify( CTXTSEL + AllTrim(cTxtList) , oOrcList:GetPos() )
743     EndIf
744     nListCheck:=oOrcList:GetPos()
745   EndIf
746 EndIf
747
748 Return .T.
749
750 //-----
751 /*/{Protheus.doc} STIExibeInf'
752 Função para exibir código e nome do vendedor e código do cliente do orçamento selecionado
753
754 @param  aList, array, informação do orçamento selecionado
755 @return NIL
756 */
757 //-----
758 Static Function STIExibeInf(aLista)
759 Local lExistFunc:= ExistFunc("STDNomeVen")
760
761 Iif(Len(aLista) >= 8, oSayVend:SetText( "Vendedor   : " + aLista[8] + iif(lExistFunc, STDNomeVen(aLista[8]),"") ) , nil )
762 Iif(Len(aLista) >= 9, oSayCli:SetText( "Cód.Cliente: " + aLista[9]), nil )
763
764 Return
765
766 /*/{Protheus.doc} STIGetImp
767 Função para retornar se tem desconto no total da venda na importação de orçamento .
768 @return lIsImpDes, logico, variavel que contém o conteúdo .T. se tem desconto .F. não tem desconto.
769 @obs
770 @sample
771 */
772 //-----
773 Function STIGetImp()
774 Return lIsImpDes

```

```

775
776 /*{Protheus.doc} STISetImp
777 Função para retornar o valor padrão da variavel lIsImpDes.
778 @param lResetImp, logico, seta um default para a variavel
779 @return Nil
780 @obs
781 @sample
782 */
783 //-----
784 Function STISetImp(lResetImp)
785 Default lResetImp := .F.
786 lIsImpDes := lResetImp
787 Return
788
789 /*{Protheus.doc} STIAltTipo
790 Quando se tratar da importação de um orçamento com retira posterior, precisamos
791 ajustar o L1_TIP0 para V, para que não gerar financeiro novamente na retaguarda.
792 @type      Function
793 @param      aSL1      , Array , Array com os dados da SL1
794 @return      Nil
795 */
796 Static Function STIAltTipo(aSL1)
797
798 Local nPos := Iif(Len(aSL1) > 0, AScan( aSL1 , { |x| x[1] == "L1_TIP0" } ), 0) // Posição do campo L1_TIP0
799
800 If nPos > 0
801     aSL1[nPos][2] := "V"
802 EndIf
803
804 Return Nil

```

Componentes/Interface/STIIncCustomer.prw

979 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIINCCUSTOMER.CH"
5
6 Static aIBGE      := STDUF()           //Array com informações de Estados e suas cidades
7 Static cSpaceMun  := space(10)        //Numero de espaços reservado para a descrição do municipio (Municipio em branco)
8 Static cSpaceCod  := space(5)         //Numero de espaços reservado para Codigo do IBGE
9 Static aCities    := {cSpaceMun}      //Array com informações sobre as cidades
10 Static aCitiesComp := {{cSpaceCod,cSpaceMun}} //Array com informações sobre cidades contendo Codigo do IBGE e nome da cidade
11 Static aSelectCit := {cSpaceCod,cSpaceMun} //Array com a informação sobre o texto selecionado no TEXTBOX
12 Static oMunicipio := Nil              //Objeto com TextBox
13
14 //-----
15 /*{Protheus.doc} STIIncCustomer
16 Chama a tela de cadastro de clientes
17
18 @param
19 @return lRet          Retorno se executou corretamente a funcao
20 @obs
21 @sample
22 */
23 //-----
24
25 Function STIIncCustomer()
26
27 STFCleanInterfaceMessage()
28 STIExchangePanel({|| STIPanIncCustomer() })
29
30 Return
31
32 //-----
33 /*{Protheus.doc} STIPanIncCustomer
34 Cria o painel de cadastro de clientes
35
36 @param
37 @return lRet          Retorno se executou corretamente a funcao
38 @obs
39 @sample
40 */
41 //-----
42
43 Static Function STIPanIncCustomer()
44 Local oPanelMVC      := STIGetDlg()      //Painel principal do dialog
45 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,(oPanelMVC:nWidth/2),(oPanelMVC:nHeight/2)) //Painel Cadastro de Cliente
46 Local oScr1Panel     := TScrollArea():New(oMainPanel,025,00,(oPanelMVC:nHeight/2)-70,(oPanelMVC:nWidth/2)-5 )
47 Local oFieldPanel    := TPanel():New(00,00,"",oScr1Panel,,,,,(oPanelMVC:nWidth/2)-5,350)
48
49 Local nLargGets      := 90
50 Local nLargGets2     := POSHOR_2+90-POSHOR_1
51
52 Local oCab           := Nil

```

```

53
54 Local oLblCodCliente:= Nil
55 Local oCodCliente := Nil
56 Local cCodCliente := CriarVar("A1_COD", .F.) //Alteracao do conteudo do get
57
58 Local oLblLojCliente:= Nil
59 Local oLojCliente := Nil
60 Local cLojCliente := CriarVar("A1_LOJA")
61
62 Local oLblNome := Nil
63 Local oNome := Nil
64 Local cNome := CriarVar("A1_NOME") //Alteracao do conteudo do get
65
66 Local oLblEndereco := Nil
67 Local oEndereco := Nil
68 Local cEndereco := CriarVar("A1_END") //Alteracao do conteudo do get
69
70 Local oLblTpCli := Nil
71 Local oTpCliente := Nil
72 Local cTpCliente := CriarVar("A1_TIPO") //Alteracao do conteudo do get
73 Local aItCliente := StrTokArr( AllTrim( GetSx3Cache("A1_TIPO","X3_CB0X") ),';') // Itens do Combo Tipo Cliente
74
75 Local oLblEstado := Nil
76 Local oEstado := Nil
77 Local cEstado := CriarVar("A1_EST") //Alteracao do conteudo do get
78
79 Local aEstados := {} // Itens do Combo Estado
80
81 Local oLblMunicipio := Nil
82 Local cMunicipio := CriarVar("A1_MUN") //Alteracao do conteudo do get
83
84 Local oLblTpPessoa := Nil
85 Local oTpPessoa := Nil
86 Local cTpPessoa := CriarVar("A1_PESSOA") //Alteracao do conteudo do get
87 Local aItPessoa := StrTokArr( AllTrim( GetSx3Cache("A1_PESSOA","X3_CB0X") ),';') // Itens do Combo Pessoa
88
89 Local oLblCGC := Nil
90 Local oCGC := Nil
91 Local cCGC := CriarVar("A1_CGC") //Alteracao do conteudo do get
92
93 Local oLblTel := Nil
94 Local oTelefone := Nil
95 Local cTelefone := CriarVar("A1_TEL") //Alteracao do conteudo do get
96
97 Local oLblEmail := Nil
98 Local oEmail := Nil
99 Local cEmail := CriarVar("A1_EMAIL") //Alteracao do conteudo do get
100
101 Local oBairro := Nil //Armazena Get com o bairro
102 Local cBairro := CriarVar("A1_BAIRRO") //Conteudo digitado no Get bairro
103 Local oLblBairro := Nil //objeto Label
104

```

```

105 Local oLblNascim      := Nil
106 Local dDtNascim       := dDatabase
107 Local oDtNascim       := Nil
108
109 Local oLblCEP          := Nil
110 Local cCep             := space(9)
111 Local oCep             := Nil
112
113 Local oInscr           := Nil
114 Local cInscr           := CriarVar("A1_INSCR")
115 Local oLblInscr        := Nil
116
117 Local oPais            := Nil
118 Local cPais            := CriarVar("A1_PAIS")
119 Local oLblPais         := Nil
120
121 Local oCompl           := Nil
122 Local cCompl           := CriarVar("A1_COMPLEM")
123 Local oLblCompl        := Nil
124
125 Local oGrTrib          := Nil
126 Local cGrTrib          := CriarVar("A1_GRPTRIB")
127 Local cF3GrTrib        := GetSx3Cache("A1_GRPTRIB", "X3_F3")
128 Local oLblGrTrib       := Nil
129
130 Local oLblFisTpPes     := Nil
131 Local oFisTpPes        := Nil
132 Local cFisTpPes        := CriarVar("A1_TPESSOA") //Alteracao do conteudo do get
133 Local aFisTpPes        := StrTokArr( " " ; "+AllTrim( GetSx3Cache("A1_TPESSOA","X3_CB0X") ),',' ) // Itens do Combo Pessoa
134 Local cGrpVen          := AllTrim(GetSx3Cache("A1_GRPVEN", "X3_RELACAO")) //Grupo de venda do cadastro de vendedores
135
136 Local oBtnConfirm      := Nil
137 Local oBtnCancel       := Nil
138
139 Local nVertCab          := 010 // Posicao vertical do cabecalho
140
141 Local nVertLbl1         := 005 // Posicao vertical dos label da linha 1
142 Local nVertGet1        := 015 // Posicao vertical dos get da linha 1
143
144 Local nVertLbl2         := nVertLbl1 + 30 // Posicao vertical dos label da linha 2
145 Local nVertGet2        := nVertGet1 + 30 // Posicao vertical dos get da linha 2
146
147 Local nVertLbl3         := nVertLbl2 + 30 // Posicao vertical dos label da linha 3
148 Local nVertGet3        := nVertGet2 + 30 // Posicao vertical dos get da linha 3
149
150 Local nVertLbl4         := nVertLbl3 + 30 // Posicao vertical dos label da linha 4
151 Local nVertGet4        := nVertGet3 + 30 // Posicao vertical dos get da linha 4
152
153 Local nVertLbl5         := nVertLbl4 + 30 // Posicao vertical dos label da linha 5
154 Local nVertGet5        := nVertGet4 + 30 // Posicao vertical dos get da linha 5+6
155
156 Local nVertLbl6         := nVertLbl5 + 30 // Posicao vertical dos label da linha 6

```

```

157 Local nVertGet6      := nVertGet5 + 30          // Posicao vertical dos get da linha 6
158
159 Local nVertLbl7       := nVertLbl6 + 30          // Posicao vertical dos label da linha 7
160 Local nVertGet7       := nVertGet6 + 30          // Posicao vertical dos get da linha 7
161
162 Local nVertLbl8       := nVertLbl7 + 30          // Posicao vertical dos label da linha 8
163 Local nVertGet8       := nVertGet7 + 30          // Posicao vertical dos get da linha 8
164
165 Local nVertLbl9       := nVertLbl8 + 30          // Posicao vertical dos label da linha 9
166 Local nVertGet9       := nVertGet8 + 30          // Posicao vertical dos get da linha 9
167
168 Local nVertLbl10      := nVertLbl9 + 30          // Posicao vertical dos label da linha 10
169 Local nVertGet10      := nVertGet9 + 30          // Posicao vertical dos get da linha 10
170
171 Local nVertLbl11      := nVertLbl10 + 30         // Posicao vertical dos label da linha 11
172 Local nVertGet11      := nVertGet10 + 30         // Posicao vertical dos get da linha 11
173
174 Local oFontCod        := TFont():New('Arial',,-16,.T.)
175
176 Local lAutoGenCod     := .T.                    // Geracao automatica de codigo // TO D0: Colocar parametro pra atribuir valor a esta variavel
177 Local nX              // Variavel para For
178
179 For nX := 1 To Len(aIBGE)
180     aAdd(aEstados,aIBGE[nX][1])
181 Next
182
183 If Empty(cLojCliente)
184     cLojCliente := Somal(cLojCliente)
185 EndIf
186
187 oScrlPanel:SetFrame( oFieldPanel )
188 oMainPanel:SetCSS(POSCSS(GetClassName(oMainPanel),CSS_PANEL_CONTEXT))
189 oScrlPanel:SetCSS("TScrollArea{ background-color: transparent; }")
190
191 oCab      := TSay():New(nVertCab,POSHOR_1,{||STR0001},oMainPanel,,,,,.T.,,,) //"Cadastro de Clientes"
192 oCab:SetCSS( POSCSS(GetClassName(oCab),CSS_BREADCUMB))
193
194
195 oLblCodCLiente:= TSay():New(nVertLbl1,POSHOR_1,{||STR0002},oFieldPanel,,,,,.T.,,,) //"Código do Cliente"
196 oLblCodCLiente:SetCSS( POSCSS(GetClassName(oLblCodCLiente),CSS_LABEL_FOCAL))
197
198 @ nVertGet1,POSHOR_1 MSGET oCodCliente VAR cCodCliente SIZE nLargGets,ALTURAGET VALID IIF(Empty(cLojCliente),.T.,STDExistChav("SA1",cCodCliente+cLojCliente,,STR0003)) PICTURE
   "@" When {||!lAutoGenCod} FONT oFontCod OF oFieldPanel PIXEL //"Já existe cliente cadastrado com o mesmo código e loja, favor alterar."
199
200 oLblLojCLiente:= TSay():New(nVertLbl1,POSHOR_2,{||STR0004},oFieldPanel,,,,,.T.,,,) //"Loja do Cliente"
201 oLblLojCLiente:SetCSS( POSCSS (GetClassName(oLblLojCLiente), CSS_LABEL_FOCAL ))
202
203
204 @ nVertGet1,POSHOR_2 MSGET oLojCliente VAR cLojCliente SIZE nLargGets,ALTURAGET VALID IIF(GetNewPar("MV_RMCLASS",.F.),.T.,STDExistChav("SA1",cCodCliente+cLojCliente,,STR0003)
   ) WHEN &(AllTrim(GetSx3Cache("A1_LOJA","X3_WHEN"))) PICTURE "@" OF oFieldPanel PIXEL //"Já existe cliente cadastrado com o mesmo código e loja, favor alterar."
205 oLojCliente:SetCSS(POSCSS(GetClassName(oLojCliente),CSS_GET_FOCAL))
206

```

```

207
208 oLb1Nome:= TSay():New(nVertLb12,POSHOR_1,{||STR0005},oFieldPanel,,,,,T,,,,) // "Nome"
209 oLb1Nome:SetCSS( POSCSS (GetClassName(oLb1Nome), CSS_LABEL_FOCAL ))
210
211 @ nVertGet2,POSHOR_1 MSGET oNome VAR cNome SIZE nLargGets2,ALTURAGET PICTURE "@!" OF oFieldPanel PIXEL
212 oNome:SetCSS(POSCSS(GetClassName(oNome),CSS_GET_NORMAL))
213
214
215 oLb1TpPessoa:= TSay():New(nVertLb13,POSHOR_1,{||STR0007},oFieldPanel,,,,,T,,,,) //"Tipo Pessoa"
216 oLb1TpPessoa:SetCSS( POSCSS (GetClassName(oLb1TpPessoa), CSS_LABEL_FOCAL ))
217
218 oTpPessoa := TComboBox():Create(oFieldPanel, {||u| if( Pcount( )>0, cTpPessoa := u, cTpPessoa ) }, nVertGet3, POSHOR_1, aItPessoa, nLargGets,
    ALTURAGET,,,,,T,,,,,cTpPessoa)
219 oTpPessoa:SetCSS( POSCSS (GetClassName(oTpPessoa), CSS_COMBOBOX ))
220
221 oLb1TpCli:= TSay():New(nVertLb13,POSHOR_2,{||STR0008},oFieldPanel,,,,,T,,,,) //"Tipo do Cliente"
222 oLb1TpCli:SetCSS( POSCSS (GetClassName(oLb1TpCli), CSS_LABEL_FOCAL))
223
224 oTpCliente := TComboBox():Create(oFieldPanel, {||u| if( Pcount( )>0, cTpCliente := u, cTpCliente ) }, nVertGet3, POSHOR_2, aItCliente, nLargGets,
    ALTURAGET,,,,,T,,,,,cTpCliente)
225 oTpCliente:SetCSS( POSCSS (GetClassName(oTpCliente), CSS_COMBOBOX ))
226
227
228 oLb1Nascim:= TSay():New(nVertLb14,POSHOR_1,{||STR0039},oFieldPanel,,,,,T,,,,) //"Data de Nascimento"
229 oLb1Nascim:SetCSS( POSCSS (GetClassName(oLb1Nascim), CSS_LABEL_FOCAL ))
230
231 @ nVertGet4,POSHOR_1 MSGET oDtNascim VAR dDtNascim SIZE nLargGets,ALTURAGET PICTURE "@D" OF oFieldPanel PIXEL HASBUTTON
232 oDtNascim:SetCSS(POSCSS(GetClassName(oDtNascim),CSS_GET_NORMAL))
233
234 oLb1CGC:= TSay():New(nVertLb14,POSHOR_2,{||STR0009},oFieldPanel,,,,,T,,,,) //"CPF/CNPJ"
235 oLb1CGC:SetCSS( POSCSS (GetClassName(oLb1CGC), CSS_LABEL_FOCAL ))
236
237 @ nVertGet4,POSHOR_2 MSGET oCGC VAR cCGC SIZE nLargGets,ALTURAGET VALID Empty(cCGC) .Or.;
238     IIF( SubStr(cTpCliente,1,1) == "X", .T., STICGCVld(cCGC,SubStr(cTpPessoa,1,1),@cCodCliente,oLojCliente,lAutoGenCod,@cLojCliente)[3]);
239     PICTURE "@R 99999999999999" OF oFieldPanel PIXEL
240 oCGC:SetCSS(POSCSS(GetClassName(oCGC),CSS_GET_NORMAL))
241
242
243 oLb1Endereco:= TSay():New(nVertLb15,POSHOR_1,{||STR0010},oFieldPanel,,,,,T,,,,) //"Endereço"
244 oLb1Endereco:SetCSS( POSCSS (GetClassName(oLb1Endereco), CSS_LABEL_FOCAL ))
245
246 @ nVertGet5,POSHOR_1 MSGET oEndereco VAR cEndereco SIZE nLargGets,ALTURAGET PICTURE "@!" OF oFieldPanel PIXEL
247 oEndereco:SetCSS(POSCSS(GetClassName(oEndereco),CSS_GET_NORMAL))
248
249 oLb1Bairro:= TSay():New(nVertLb15,POSHOR_2,{||STR0038},oFieldPanel,,,,,T,,,,) //"Bairro"
250 oLb1Bairro:SetCSS( POSCSS (GetClassName(oLb1Bairro), CSS_LABEL_FOCAL ))
251
252 @ nVertGet5,POSHOR_2 MSGET oBairro VAR cBairro SIZE nLargGets,ALTURAGET PICTURE "@!" OF oFieldPanel PIXEL
253 oBairro:SetCSS(POSCSS(GetClassName(oBairro),CSS_GET_NORMAL))
254
255 oLb1Estado:= TSay():New(nVertLb16,POSHOR_1,{||STR0011},oFieldPanel,,,,,T,,,,) //"Estado"
256 oLb1Estado:SetCSS( POSCSS (GetClassName(oLb1Estado), CSS_LABEL_FOCAL ))

```

```

257
258 oEstado := TComboBox():Create(oFieldPanel, {|u| if( Pcount( )>0, cEstado := u, cEstado )}, nVertGet6, POSHOR_1, aEstados, nLargGets, ALTURAGET,,,{|
    STIGetMun(cEstado)},,,,T,,,,,,cEstado)
259 oEstado:SetCSS( POSCSS (GetClassName(oEstado), CSS_COMBOBOX ))
260
261 oLblMunicipio:= TSay():New(nVertLbl6,POSHOR_2,{||STR0012},oFieldPanel,,,,,T,,,,) //"Município"
262 oLblMunicipio:SetCSS( POSCSS (GetClassName(oLblMunicipio), CSS_LABEL_FOCAL ))
263
264 oMunicipio := TComboBox():Create(oFieldPanel, {|u| if( Pcount( )>0, cMunicipio := u, cMunicipio )}, nVertGet6, POSHOR_2, aCities, nLargGets, ALTURAGET,,,{|
    STIFillCit()},,,,T,,,,,,cMunicipio)
265 oMunicipio:SetCSS( POSCSS (GetClassName(oMunicipio), CSS_COMBOBOX ))
266
267 oLblCEP:= TSay():New(nVertLbl7,POSHOR_1,{||STR0040},oFieldPanel,,,,,T,,,,) //"CEP"
268 oLblCEP:SetCSS( POSCSS (GetClassName(oLblCEP), CSS_LABEL_FOCAL ))
269
270 @ nVertGet7,POSHOR_1 MSGET oCep VAR cCep SIZE nLargGets,ALTURAGET PICTURE "99999-999" OF oFieldPanel PIXEL
271 oCep:SetCSS(POSCSS(GetClassName(oCep),CSS_GET_NORMAL))
272
273 oLblPais:= TSay():New(nVertLbl7,POSHOR_2,{||STR0042},oFieldPanel,,,,,T,,,,) //"País"
274 oLblPais:SetCSS( POSCSS (GetClassName(oLblPais), CSS_LABEL_FOCAL ))
275
276 @ nVertGet7,POSHOR_2 MSGET oPais VAR cPais SIZE nLargGets,ALTURAGET PICTURE "@" F3 'SYA' OF oFieldPanel PIXEL HASBUTTON
277 oPais:SetCSS(POSCSS(GetClassName(oPais),CSS_GET_NORMAL))
278
279 oLblCompl:= TSay():New(nVertLbl8,POSHOR_1,{||STR0043},oFieldPanel,,,,,T,,,,) //"Complemento"
280 oLblCompl:SetCSS( POSCSS (GetClassName(oLblCompl), CSS_LABEL_FOCAL ))
281
282 @ nVertGet8,POSHOR_1 MSGET oCompl VAR cCompl SIZE nLargGets2,ALTURAGET PICTURE "@" OF oFieldPanel PIXEL
283 oCompl:SetCSS(POSCSS(GetClassName(oCompl),CSS_GET_NORMAL))
284
285 oLblTel := TSay():New(nVertLbl9,POSHOR_1,{||STR0033},oFieldPanel,,,,,T,,,,) //"DDD) + Telefone"
286 oLblTel:SetCSS( POSCSS (GetClassName(oLblTel), CSS_LABEL_FOCAL ))
287
288 @ nVertGet9,POSHOR_1 MSGET oTelefone VAR cTelefone SIZE nLargGets,ALTURAGET PICTURE "(999) 999999999" OF oFieldPanel PIXEL
289 oTelefone:SetCSS(POSCSS(GetClassName(oTelefone),CSS_GET_NORMAL))
290
291 oLblEmail:= TSay():New(nVertLbl9,POSHOR_2,{||STR0034},oFieldPanel,,,,,T,,,,) //"E-mail"
292 oLblEmail:SetCSS( POSCSS (GetClassName(oLblEmail), CSS_LABEL_FOCAL ))
293
294 @ nVertGet9,POSHOR_2 MSGET oEmail VAR cEmail SIZE nLargGets,ALTURAGET OF oFieldPanel PIXEL
295 oEmail:SetCSS(POSCSS(GetClassName(oEmail),CSS_GET_NORMAL))
296
297 oLblInscr:= TSay():New(nVertLbl10,POSHOR_1,{||STR0044},oFieldPanel,,,,,T,,,,) //"Insc. Estad."
298 oLblInscr:SetCSS( POSCSS (GetClassName(oLblInscr), CSS_LABEL_FOCAL ))
299
300 @ nVertGet10,POSHOR_1 MSGET oInscr VAR cInscr SIZE nLargGets,ALTURAGET VALID Empty(cInscr) .Or. STIIEVld(cInscr,SubStr(cEstado,1,2)) PICTURE "@" OF oFieldPanel PIXEL
301 oInscr:SetCSS(POSCSS(GetClassName(oInscr),CSS_GET_NORMAL))
302
303 oLblGrTrib:= TSay():New(nVertLbl10,POSHOR_2,{||STR0045},oFieldPanel,,,,,T,,,,) //"Grp.Tribut."
304 oLblGrTrib:SetCSS( POSCSS (GetClassName(oLblGrTrib), CSS_LABEL_FOCAL ))
305
306 @ nVertGet10,POSHOR_2 MSGET oGrTrib VAR cGrTrib SIZE nLargGets,ALTURAGET PICTURE "@" OF oFieldPanel PIXEL HASBUTTON

```

```

307 if !empty(cF3GrTrib)
308     oGrTrib:cF3 := cF3GrTrib
309 endif
310 oGrTrib:SetCSS(POSCSS(GetClassName(oGrTrib),CSS_GET_NORMAL))
311
312 oLbLfisTpPes:= TSay():New(nVertLb111,POSHOR_1,{||STR0007},oFieldPanel,,,,,T.,,,) //"Tipo Pessoa"
313 oLbLfisTpPes:SetCSS( POSCSS (GetClassName(oLbLfisTpPes), CSS_LABEL_FOCAL ))
314
315 oFisTpPes := TComboBox():Create(oFieldPanel, {||u| if( Pcount( )>0, cFisTpPes := u, cFisTpPes) }, nVertGet11, POSHOR_1, aFisTpPes, nLargGets,
    ALTURAGET,,,,,T.,,,,,,cFisTpPes)
316 oFisTpPes:SetCSS( POSCSS (GetClassName(oFisTpPes), CSS_COMBOBOX ))
317
318
319 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0013,oMainPanel;; //"Cadastrar Cliente"
320     { || STIConfCustomer(cCodCliente, cLojCliente, cNome, Iif(!Empty(cNome),StrTokArr(cNome," ")[1]," "));
321         SubStr(cTpPessoa,1,1), cCGC, SubStr(cTpCliente,1,1), cEndereco ;;
322         SubStr(cEstado,1,2), aSelectCit[2], lAutoGenCod , /*lOpenRegItem*/;;
323         cBairro, SubStr(cTelefone,At("(",cTelefone)+1,3), SubStr(cTelefone,At(")",cTelefone)+1));
324         {"A1_FILIAL","A1_COD","A1_LOJA","A1_NOME","A1_NREDUZ","A1_END","A1_TIPO","A1_EST","A1_MUN",;
325         "A1_PESSOA","A1_CGC","A1_DDD","A1_TEL","A1_EMAIL","A1_BAIRRO","A1_COD_MUN",;
326         "A1_DTNASC","A1_CEP","A1_INSCR","A1_PAIS","A1_COMPLEM","A1_GRPTRIB","A1_TPESSOA","A1_GRPVEN"},;
327         {xFilial("SA1"),cCodCliente,cLojCliente,cNome,Iif(!Empty(cNome),StrTokArr(cNome," ")[1]," "),cEndereco,;
328         SubStr(cTpCliente,1,1),SubStr(cEstado,1,2),aSelectCit[2],;
329         SubStr(cTpPessoa,1,1),cCGC,SubStr(cTelefone,At("(",cTelefone)+1,3),SubStr(cTelefone,At(")",cTelefone)+1),;
330         cEmail,cBairro,aSelectCit[1],dDtNascim,STRTRAN(cCep,"-",""),;
331         cInscr , cPais , cCompl, cGrTrib ,SubStr(cFisTpPes,1,2),cGrpVen}} ,;
332         /*lValidFields*/, /*cSitua*/, aSelectCit[1], cEmail , STRTRAN(cCep,"-",""), dDtNascim,;
333         cInscr , cPais , cCompl, cGrTrib ,SubStr(cFisTpPes,1,2),oMainPanel } ,;
334         LARGBTN,ALTURABTN,,,,T.)
335
336 oBtnCancel := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0014,oMainPanel,{ || Iif(ExistFunc("StiMataObj"),StiMataObj(oMainPanel),Nil),STIRegItemInterface() }, ; //"Cancelar"
337     LARGBTN,ALTURABTN,,,,T.)
338
339 oBtnConfirm:SetCSS(POSCSS(GetClassName(oBtnConfirm),CSS_BTN_FOCAL))
340 oBtnCancel:SetCSS(POSCSS(GetClassName(oBtnCancel),CSS_BTN_NORMAL))
341
342 oNome:SetFocus()
343
344 Return oMainPanel
345
346 //-------------------------------------
347 /*{Protheus.doc} STICGCVld
348 Realiza a confirmacao da inclusao do cliente
349
350 @param
351 @return
352 */
353 //-------------------------------------
354 Function STICGCVld( cCGC,cTpPessoa,cCodCliente,oLojCliente,lAutoGenCod,cLojCliente )
355 Local lRet := .F.
356 Local aRet := Array(3)
357

```

```

358 IF STBCGCDigVerificador(cCGC)
359
360     lRet := .T.
361
362     If nModulo = 23
363         STFCleanInterfaceMessage()
364     EndIf
365
366     If lAutoGenCod
367         If ExistBlock("STCodCli")
368             LjGrvLog("STIIncCustomer","Antes do ponto de entrada STCodCli")
369             aRetPe := ExecBlock("STCodCli",.F.,.F.,{cCGC,cTpPessoa})
370             LjGrvLog("STIIncCustomer","Retorno do ponto de entrada STCodCli", aRetPe)
371             If (ValType(aRetPe) <> "A") .OR. (Len(aRetPe) < 2) .OR. (ValType(aRetPe[1]) <> TamSx3("A1_COD")[3]) .OR. (ValType(aRetPe[2]) <> TamSx3("A1_LOJA")[3])
372                 aRetPe := {}
373                 MsgStop(STR0049,STR0029) // "Retorno do ponto de entrada STCodCli inválido.", "Atenção!"
374             Else
375                 cCodCliente := aRetPe[1]
376                 cLojCliente := aRetPe[2]
377             EndIf
378             LjGrvLog("STIIncCustomer","Depois do ponto de entrada STCodCli")
379         Else
380             STICodeGenerator(cCGC,cTpPessoa,@cCodCliente,oLojCliente,@cLojCliente)
381         EndIf
382     EndIf
383
384 EndIf
385
386 aRet[1] := cCodCliente
387 aRet[2] := cLojCliente
388 aRet[3] := lRet
389
390 Return aRet
391
392 //-----
393 /*{Protheus.doc} STIIEVld
394 Realiza a confirmacao da inclusao do cliente
395
396 @param
397 @return
398 */
399 //-----
400 Static Function STIIEVld(cInscr,cEstado)
401 Local lRet := .F.
402
403 if IE(cInscr,cEstado, .F.)
404     lRet := .T.
405     STFCleanInterfaceMessage()
406 else
407     STFMessage("STIIEVld","STOP", STR0041) //"A Inscrição Estadual informada está inválida para esta unidade federativa."
408     STFShowMessage("STIIEVld")
409 endif

```

```

410
411 Return lRet
412
413 //-----
414 /*{Protheus.doc} STIConfCustomer
415 Realiza a confirmacao da inclusao do cliente
416
417 @param lOpenRegItem para exibir ou não a tela de registro de item após a ação
418 @param lValidFields valida campos
419 @param cSitua - Situa para gravacao do cliente
420 @return
421 */
422 //-----
423 Function STIConfCustomer( cCodCliente ,cLojCliente ,cNome ,cNFantasia ,;
424                          cTpPessoa ,cCGC ,cTpCliente ,cEndereco ,;
425                          cEstado ,cMunicípio ,lAutoGenCod ,lOpenRegItem ,;
426                          cBairro ,cDDD ,cTel ,aCampos ,;
427                          aDados ,lValidFields ,cSitua ,cCodMun ,;
428                          cEmail ,cCep ,dDtNascim ,cInscr ,;
429                          cPais ,cCompl ,cGrTrib ,cFisTpPes ,;
430                          oMainPanel)
431
432 Local lRet := .T.
433 Local aSA1 := {}
434 Local lSendCli := .F.
435 Local nSendOn := SuperGetMV("MV_LJSEND0",,0) //Retorno como sera a integracao do cliente - 0 - via job - 1 online - 2 startjob
436 Local nI := 1
437 Local IncErro := .F.
438 Local lTabAI0 := .F. //Busca campos da tabela AI0
439 Local aCamposAI0 := {}
440 Local nX := 0 //contador de campos
441 Local lA1_FILIAL := .F. //Existe campo Filial?
442 Local lEstadEstr := cEstado == "EX" // Estado estrangeiro
443 Local lTpCliEstr := cTpCliente == "X" // Tipo Cliente Estrangeiro
444 Local nAtuCliVda := SuperGetMv("MV_LJATCLI",,2) // Força a atualização do cliente
445 Local lRegNovo := .F. //Verifica se deve incluir ou alterar o cliente
446
447 Default lOpenRegItem := .T. //define se após executar a rotina irá mudar para o registro de item
448 Default cBairro := ""
449 Default cDDD := ""
450 Default cTel := ""
451 Default aCampos := {}
452 Default aDados := {}
453 Default lValidFields := .T. //Valida campos
454 Default cSitua := "00"
455 Default cCodMun := ""
456 Default cEmail := ""
457 Default cCep := ""
458 Default dDtNascim := dDataBase
459 Default cInscr := ""
460 Default cPais := ""
461 Default cCompl := ""

```

```

462 Default cGrTrib          := ""
463 Default cFisTpPes        := ""
464 Default oMainPanel       := Nil
465
466 lRet := STDExistChav("SA1",cCodCliente+cLojCliente,,STR0003,nAtuCliVda) //"Já existe cliente cadastrado com o mesmo código e loja, favor alterar."
467
468 //Se o lRet Retornar .F. indica que o cliente já ecxiste na base conforme a função STDExistChav
469 If lRet
470     lRegNovo := .T.
471 Endif
472
473 For nX := 1 to Len(aCampos)
474     If !lA1_FILIAL
475         lA1_FILIAL := aCampos[nX] == "A1_FILIAL"
476     EndIf
477     If nX <= Len(aCampos) .AND. "AI0_" $ aCampos[nX]
478         aAdd(aCamposAI0, { aCampos[nX], aDados[1][nX] } )
479         aDel(aCampos, nX)
480         aSize(aCampos, len(aCampos) - 1)
481         aDel(aDados[1], nX)
482         aSize(aDados[1], len(aDados[1]) - 1)
483     EndIf
484 Next
485
486 lTabAI0 := Len(aCamposAI0) > 0
487
488
489 If lRet .AND. lValidFields
490     lRet := STIchkFields(cCodCliente,cLojCliente,cNome          ,cNFantasia ,;
491                        cTpPessoa ,cCGC          ,cTpCliente ,cEndereco ,;
492                        cEstado   ,cMunicipio ,lAutoGenCod, cPais, cInscr)
493 EndIf
494
495 If lRet .AND. lValidFields .AND. ( !lEstadEstr .OR. !lTpCliEstr )
496     lRet := STBVldCGC(cCodCliente,cLojCliente,cTpPessoa, cCGC)
497     If !lRet
498         Conout(STR0031) //"CPF do cliente a ser cadastrado é inválido. "
499     EndIf
500 EndIf
501
502 If lRet .OR. nAtuCliVda == 1
503     If Len(aCampos) > 0 .AND. Len(aDados) > 0
504         If !lA1_FILIAL
505             aAdd(aSA1, {"A1_FILIAL" ,xFilial("SA1"), Nil })
506         EndIf
507         For nI := 1 To Len(aCampos)
508             aAdd(aSA1, {aCampos[nI] ,aDados[1][nI], Nil })
509         Next nI
510     Else
511         aAdd(aSA1,{"A1_FILIAL" , xFilial("SA1"), Nil })
512         aAdd(aSA1,{"A1_COD"      , cCodCliente , Nil })

```

```

514      Aadd(aSA1,{"A1_LOJA"      , cLojCliente  , Nil })
515      Aadd(aSA1,{"A1_NOME"      , cNome      , Nil })
516      Aadd(aSA1,{"A1_NREDUZ"    , cNFantasia , Nil })
517      Aadd(aSA1,{"A1_CGC"       , cCGC       , Nil })
518      Aadd(aSA1,{"A1_END"       , cEndereco  , Nil })
519      Aadd(aSA1,{"A1_EST"       , cEstado    , Nil })
520      Aadd(aSA1,{"A1_MUN"       , cMunicipio , Nil })
521      Aadd(aSA1,{"A1_COD_MUN"   , cCodMun    , Nil })
522      Aadd(aSA1,{"A1_TIPO"      , cTpCliente , Nil })
523      Aadd(aSA1,{"A1_PESSOA"    , cTpPessoa  , Nil })
524      Aadd(aSA1,{"A1_BAIRRO"    , cBairro    , Nil })
525      Aadd(aSA1,{"A1_DDD"       , cDDD       , Nil })
526      Aadd(aSA1,{"A1_TEL"      , cTel       , Nil })
527      Aadd(aSA1,{"A1_EMAIL"     , cEmail     , Nil })
528      Aadd(aSA1,{"A1_CEP"       , cCep       , Nil })
529      Aadd(aSA1,{"A1_DTNASC"    , dDtNascim  , Nil })
530      Aadd(aSA1,{"A1_INSCR"     , cInscr     , Nil })
531      Aadd(aSA1,{"A1_PAIS"      , cPais      , Nil })
532      Aadd(aSA1,{"A1_COMPLEM"   , cCompl     , Nil })
533      Aadd(aSA1,{"A1_GRPTRIB"   , cGrTrib    , Nil })
534      Aadd(aSA1,{"A1_TPESSOA"   , cFisTpPes  , Nil })
535  EndIf
536
537  Aadd(aSA1,{"A1_SITUA", cSitua , Nil }) // Grava o A1_SITUA local
538
539  //Foi removido tratamento de execAuto, pois não ha a necessidade de se chamar a rotina
540  //pois são gravados campos basicos para o cliente ser utilizado no PDV.
541  //Alem disso, para a versão 12 foram incluídas funções que utilizam comandos SQL, que nao são executados no PDV (em DBF).
542  Conout("Gravando cliente...")
543
544  If SA1->(RecLock("SA1",lRegNovo))
545      For nI := 1 To Len(aSA1)
546          SA1->&(aSA1[nI,1]) := aSA1[nI,2]
547      Next
548      SA1->(MSUnlock())
549  Else
550      IncErro := .T.
551  EndIf
552
553  If lTabAI0
554      AI0->(DbSetOrder(1)) //AI0_FILIAL + AI0_CODCLI + AI0_LOJA
555      If !AI0->(DbSeek(xFilial("AI0")+ SA1->A1_COD+SA1->A1_LOJA))
556          SA1->(RecLock("AI0",.T.))
557          AI0->AI0_FILIAL := xFilial("AI0")
558          AI0->AI0_CODCLI := cCodCliente
559          AI0->AI0_LOJA   := cLojCliente
560          For nI := 1 To Len(aCamposAI0)
561              AI0->&(aCamposAI0[nI,1]) := aCamposAI0[nI,2]
562          Next
563          AI0->(MSUnlock())
564      EndIf
565  EndIf

```

```

566 Conout("Fim da gravacao do cliente")
567
568 If IncErro
569     STFMessage(ProcName(),"STOP",STR0016) //"Houve erro na gravação do cliente."
570     STFSHOWMESSAGE(ProcName())
571     Conout(STR0016) //"Houve erro na gravação do cliente."
572     lRet := .F.
573 Else
574     If nAtuCliVda == 1
575         STFMessage(ProcName(),"STOP",STR0048) // "0 cliente foi atualizado com sucesso."
576     Else
577         STFMessage(ProcName(),"STOP",STR0017) //"0 cliente foi incluído com sucesso."
578     EndIf
579     STFSHOWMESSAGE(ProcName())
580     Conout(STR0017 + " Recno:" + AllTrim(Str(SA1->(RecNo())))) //"0 cliente foi incluído com sucesso."
581 EndIf
582 Else
583     Conout(STR0035 + " Recno:" + AllTrim(Str(SA1->(RecNo())))) //"0 Cliente não será cadastrado no PDV, regularize o cadastro na Retaguarda."
584     LjGrvLog(STR0001 + " STIConfCustomer",cCodCliente+cLojCliente,STR0035) // STR0001 -> "Cadastro de Clientes" || STR0035 -> "0 Cliente não será cadastrado no PDV, regularize o cadastro na Retaguarda."
585 EndIf
586
587 //Transmite o cliente para retaguarda - gorgulho
588 If lRet .AND. nSendOn == 1
589     MsgRun(STR0032,'Aguarde...',{||STDSendCli(aSA1,@lSendCli)}) //"Transmitindo cliente"
590 ElseIf lRet .AND. nSendOn == 2
591     StartJob("STDSEndCli", GetEnvServer(), .F., aSA1,@lSendCli,.T.,cEmpAnt,cFilAnt)
592 EndIf
593
594 If lRet .AND. !lSendCli .AND. cSitua == "00"
595
596     DbSelectArea("SA1")
597     SA1->(DbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
598     If DbSeek(xFilial("SA1")+cCodCliente+cLojCliente)
599
600         STFSLICreate( Nil , "UP" , "UP" , "NOVO" , ;
601                     Nil , Nil , Nil , Nil , ;
602                     "SA1" , SA1->(RecNo()), Nil )
603
604     Else
605
606         Conout(STR0030 + cCodCliente+cLojCliente ) //"Não achou"
607
608     EndIf
609 EndIf
610
611 If lRet .AND. lOpenRegItem
612     Iif(ExistFunc("StiMataObj"),StiMataObj(oMainPanel),Nil)
613     STIRegItemInterface()
614 EndIf
615
616 Return lRet

```

```

617
618 //-----
619 /*{Protheus.doc} STIChkFields
620 Verifica se os campos obrigatorios foram preenchidos e informa o usuario caso nao tenha sido.
621
622 @param
623 @return lRet - Retorna True se todos os campos obrigatorios foram preenchidos
624 */
625 //-----
626 Static Function STIChkFields( cCodCliente,cLojCliente,cNome,cNFantasia,;
627                               cTpPessoa,cCGC,cTpCliente,cEndereco,;
628                               cEstado,cMunicipio, lAutoGenCod, cPais,;
629                               cInscr)
630 Local lRet      := .T.
631 Local cEnter    := CHR(13)+CHR(10)
632 Local cAlert    := ''
633
634 Do Case
635   Case ( cTpCliente == "X" .AND. cEstado <> "EX")
636     lRet := .F.
637     STFMessage(ProcName(),"STOPPOPUP",STR0036 + cEnter + STR0035) //"Campo 'Tipo de cliente' informado como 'X' (Estrangeiro) e campo 'Estado' difente de 'EX'." || "0
638     Cliente não será cadastrado no PDV, regularize o cadastro na Retaguarda."
639     STFShowMessage(ProcName())
640     cAlert := STR0036
641   Case ( cTpCliente <> "X" .AND. cEstado == "EX")
642     lRet := .F.
643     STFMessage(ProcName(),"STOPPOPUP",STR0037 + cEnter + STR0035 ) // "Campo 'Estado' informado como 'EX' (Estrangeiro) e campo 'Tipo de Cliente' diferente de 'X'." ||
644     "0 Cliente não será cadastrado no PDV, regularize o cadastro na Retaguarda."
645     STFShowMessage(ProcName())
646     cAlert := STR0037
647   Case lAutoGenCod .AND. Empty(cCGC) .AND. ( cTpCliente <> "X" .AND. cEstado <> "EX")
648     lRet := .F.
649     STFMessage(ProcName(),"STOPPOPUP",STR0018 + cEnter + STR0035 ) //"Preencha o CPF para gerar o código do cliente!" || "0 Cliente não será cadastrado no PDV,
650     regularize o cadastro na Retaguarda."
651     STFShowMessage(ProcName())
652     cAlert := STR0018
653   Case Empty(cCodCliente)
654     lRet := .F.
655     STFMessage(ProcName(),"STOPPOPUP",STR0019 + cEnter + STR0035 ) //"0 campo Código do Cliente deve ser preenchido!" || "0 Cliente não será cadastrado no PDV,
656     regularize o cadastro na Retaguarda."
657     STFShowMessage(ProcName())
658     cAlert := STR0019
659   Case Empty(cLojCliente)
660     lRet := .F.
661     STFMessage(ProcName(),"STOPPOPUP",STR0020 + cEnter + STR0035 ) //"0 campo Loja do Cliente deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize
662     o cadastro na Retaguarda."
663     STFShowMessage(ProcName())
664     cAlert := STR0020
665   Case Empty(cNome)
666     lRet := .F.
667     STFMessage(ProcName(),"STOPPOPUP",STR0021 + cEnter + STR0035 ) //"0 campo Nome deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o cadastro
668     na Retaguarda."

```

```

663     STFSHOWMESSAGE(ProcName())
664     cAlert := STR0021
665     Case Empty(cNFantasia)
666         lRet := .F.
667         STFMessagem(ProcName(),"STOPPOPUP",STR0022 + cEnter + STR0035) //"0 campo Nome Fantasia deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
668         STFSHOWMESSAGE(ProcName())
669         cAlert := STR0022
670     Case Empty(cTpPessoa)
671         lRet := .F.
672         STFMessagem(ProcName(),"STOPPOPUP",STR0023 + cEnter + STR0035) //"0 campo Tipo Pessoa deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
673         STFSHOWMESSAGE(ProcName())
674         cAlert := STR0023
675     Case Empty(cTpCliente)
676         lRet := .F.
677         STFMessagem(ProcName(),"STOPPOPUP",STR0024 + cEnter + STR0035) //"0 campo Tipo do Cliente deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
678         STFSHOWMESSAGE(ProcName())
679         cAlert := STR0024
680     Case Empty(cEndereco)
681         lRet := .F.
682         STFMessagem(ProcName(),"STOPPOPUP",STR0025 + cEnter + STR0035) //"0 campo Endereço deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
683         STFSHOWMESSAGE(ProcName())
684         cAlert := STR0025
685     Case Empty(cEstado)
686         lRet := .F.
687         STFMessagem(ProcName(),"STOPPOPUP",STR0026 + cEnter + STR0035) //"0 campo Estado deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o cadastro
na Retaguarda."
688         STFSHOWMESSAGE(ProcName())
689         cAlert := STR0026
690     Case Empty(cMunicipio)
691         lRet := .F.
692         STFMessagem(ProcName(),"STOPPOPUP",STR0027 + cEnter + STR0035) //"0 campo Município deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
693         STFSHOWMESSAGE(ProcName())
694         cAlert := STR0027
695     Case ( empty(cInscr) .AND. cTpPessoa == "J" )
696         lRet := .F.
697         STFMessagem(ProcName(),"STOPPOPUP", STR0046 + cEnter + STR0035 ) // "Pessoa Jurídica. O campo Inscrição Estadual deve ser preenchido!" || "0 Cliente não será
cadastrado no PDV, regularize o cadastro na Retaguarda."
698         STFSHOWMESSAGE(ProcName())
699         cAlert := STR0046
700     Case ( empty(cPais) .AND. cEstado == "EX" )
701         lRet := .F.
702         STFMessagem(ProcName(),"STOPPOPUP",STR0047 + cEnter + STR0035 ) // "0 campo País deve ser preenchido!" || "0 Cliente não será cadastrado no PDV, regularize o
cadastro na Retaguarda."
703         STFSHOWMESSAGE(ProcName())
704         cAlert := STR0047
705     EndCase
706

```

```

707 If !lRet
708     LjGrvLog(STR0001 + " STIChkFields",cCodCliente+cLojCliente,cAlert) // STR0001 -> "Cadastro de Clientes"
709 EndIf
710
711 Return lRet
712
713 //-----
714 /*{Protheus.doc} STICodeGenerator
715 Gera o código do cliente a partir do CPF/CNPJ.
716
717 @param
718 @return lRet - Retorna True se todos os campos obrigatórios foram preenchidos
719 */
720 //-----
721 Function STICodeGenerator(cCGC,cTpPessoa,cCodCliente,oLojCliente,cLojCliente)
722
723 Local aArea          := GetArea()                //Area atual
724 Local aAreaSA1       := SA1->(GetArea())          //Area do SA1
725 Local nTamCod        := TamSx3("A1_COD")[1]      //Tamanho do campo
726 Local nDiv           := 0                        //Numero utilizado para geracao do A1_COD
727 Local nResto         := 0                        //Guarda o resto da divisao
728 Local cNumero        := Space(nTamCod)           //Numero gerado
729 Local lSTGenCli      := ExistBlock("STGenCli")
730 Local aRetPe         := {}                       //Retorno do ponto de entrada
731 Local cCodConut      := ""                       //codigo do contador
732 Local cLimit         := ""                       //Numero maximo de tentativas
733 Local aCodCliLoj     := {"",""}                 // Código gerado
734 Local cLjTpGCC       := SuperGetMV("MV_LJTPGCC",,"0") // Opções de geração de código de cliente: 0 ou nil-legado; 1- 9 dígitos numéricos; 2- algoritmo Base 36.
735 Local nTXGCodCli     := TamSXG("001")[1]         // Tamanho do grupo de Cliente
736 Local cQuerySA1      := ""
737 Local cAliasSA1      := ""
738 Local cCGCEmbrlh     := AllTrim(Embaralha(SubStr(cCGC,1,9),0)) // 9 primeiros dígitos embaralhados
739 Local nTamA1CGC      := TamSx3("A1_CGC")[1]       //Tamanho do campo
740
741 Default cLojCliente := "01"
742
743 LjGrvLog("STICodeGenerator", "MV_LJTPGCC - " + cLjTpGCC)
744
745 If !Empty(cCGC)
746     If cLjTpGCC == "1" .AND. nTXGCodCli >= 9 // Gera código com 9 dígitos
747         If SA1->(dbSeek( xFilial("SA1") + cCGCEmbrlh ) )
748             // Se o CPF ou CNPJ já está cadastrado mantém o código e soma a filial
749             cQuerySA1 += " SELECT MAX(A1_LOJA) AS MAXLOJA "
750             cQuerySA1 += " FROM " + RetSqlName("SA1") + " "
751             cQuerySA1 += " WHERE A1_COD = '" + cCGCEmbrlh + "' "
752
753             cAliasSA1 := MPSysOpenQuery(cQuerySA1)
754
755             cCodCliente := SA1->A1_COD
756             cLojCliente := Somal((cAliasSA1)->MAXLOJA)

```

```

759     Else
760         cCodCliente := cCGCEmbrlh
761         cLojCliente := "01"
762     EndIf
763
764 Elseif cLjTpGCC == "2" // Geração do Codcli usando algoritmo Base36.
765
766     If lSTGenCli
767
768         aRetPe := ExecBlock("STGenCli",.F.,.F.,{cCGC,cTpPessoa})
769         If ValType(aRetPe) <> "A"
770             aRetPe := {}
771             MsgStop(STR0028,STR0029)    //"Retorno do STGenCli tem tipo inválido.", "Atenção!"
772         Else
773             cCodCliente := aRetPe[1]
774             cLojCliente := aRetPe[2]
775         EndIf
776
777     Else
778
779         DbSelectArea("SA1")
780         SA1->(DbSetOrder(1))
781         cCodCliente:= STIGerCliB36(cCGC)
782
783         If SA1->(dbSeek( xFilial("SA1") + cCodCliente + cLojCliente) )
784
785             cQuerySA1 += " SELECT MAX(A1_LOJA) AS MAXLOJA "
786             cQuerySA1 += " FROM " + RetSqlName("SA1") + " "
787             cQuerySA1 += " WHERE A1_COD = '" + cCodCliente + "' "
788             cQuerySA1 += " AND A1_CGC = '" + PadR(Alltrim(cCGC),nTamA1CGC) + "' "
789
790             cAliasSA1 := MPSysOpenQuery(cQuerySA1)
791             cLojCliente := Soma1((cAliasSA1)->MAXLOJA)
792
793         Endif
794
795     Endif
796
797     LjGrvLog("STICodeGenerator", "CodCli - "+cCodCliente+" gerado através do algoritmo base 36")
798 Else
799
800     If lSTGenCli
801         aRetPe := ExecBlock("STGenCli",.F.,.F.,{cCGC,cTpPessoa})
802         If ValType(aRetPe) <> "A"
803             aRetPe := {}
804             MsgStop(STR0028,STR0029)    //"Retorno do STGenCli tem tipo inválido.", "Atenção!"
805         Else
806             cCodCliente := aRetPe[1]
807             cLojCliente := aRetPe[2]
808         EndIf
809     Else
810

```

```

811      If Len(AllTrim(cCGC)) > 11
812          //Pessoa Juridica
813          nDiv := Val(cCGC)
814      Else
815          //Pessoa Fisica
816          nDiv := Val(SubStr(cCGC,1,9))
817      EndIf
818
819      //Calcula codigo
820      While nDiv >= 35 .AND. Len(AllTrim(cNumero)) < 6
821          //Pega o inteiro do resto da divisao
822          nResto := int(nDiv % 35)
823          //Pega o valor inteiro da divisao
824          nDiv := int(nDiv / 35)
825          cNumero:= AllTrim(IIIf(nResto < 10, Str(nResto), Chr(nResto + 55))) + AllTrim(cNumero)
826      End
827
828      //Quando codigo gerado for diferente do tamanho do campo A1_COD, realizado ajuste
829      If Len(AllTrim(cNumero)) <> nTamCod
830          cNumero := AllTrim(IIIf(nResto < 10, Str(nResto), Chr(nResto + 55))) + AllTrim(cNumero)
831      EndIf
832
833      cCodCliente := Replicate("0", nTamCod - Len(AllTrim(cNumero))) + AllTrim(cNumero)
834  EndIf
835
836  If Empty(cLojCliente)
837      cLojCliente := Soma1(cLojCliente)
838  EndIf
839
840  DbSelectArea("SA1")
841  SA1->(DbSetOrder(1))
842  SA1->(dbSeek(xFilial("SA1") + cCodCliente + cLojCliente) )
843
844  If Len(AllTrim(cCGC)) <=11 .And. Alltrim(cCGC) == AllTrim(SA1->A1_CGC)
845      cLimit := Replicate("Z" , TamSx3("A1_LOJA")[1] )
846  Else
847      cLimit := Replicate("Z" , TamSx3("A1_COD")[1] )
848  EndIf
849  //No caso do codigo duplicado, soma posições ate que o codigo seja valido e contador dentro do limite
850  While ( SA1->(dbSeek(xFilial("SA1") + cCodCliente + cLojCliente) ) ) .And. cCodConut <> cLimit
851
852      // Se for o mesmo cpf incrementamos a loja
853      If Len(AllTrim(cCGC)) <=11 .And. Alltrim(cCGC) == AllTrim(SA1->A1_CGC)
854          cLojCliente := cCodConut := Soma1(cLojCliente)
855      Else
856          cCodCliente := cCodConut := Soma1(cCodCliente)
857      EndIf
858
859  EndDo
860  EndIf
861 EndIf
862

```

```

863 //Restaura a area atual e do arquivo SA1
864 RestArea(aAreaSA1)
865 RestArea(aArea)
866
867 aCodCliLoj[01] := cCodCliente
868 aCodCliLoj[02] := cLojCliente
869
870 Return aCodCliLoj
871
872 //-----
873 /*{Protheus.doc} STIGerCliB36
874 //Gera Código de cliente alfanumérica para CPF e CNPJ utilizando Base 36
875
876 @param cNumero, Character, Numero do CPF ou CNPJ
877 @return cCodB36, caracter, Código do cliente
878 */
879 //-----
880 Function STIGerCliB36(cNumero)
881 Local aNumero := {}
882 Local nNumero := 0
883 Local nNumPos := 0
884 Local cCodB36 := ""
885 DEFAULT cNumero := ""
886
887 If ValType(cNumero) == TamSx3("A1_COD")[3]
888
889     If cNumero <> ""
890         cNumero:= AllTrim(cNumero)
891
892         If Len(cNumero) == 14 //CNPJ
893             aNumero := { Val(Left(cNumero,2))+1196, Val(SubStr(cNumero,3,3)), Val(SubStr(cNumero,6,3)) }
894         ElseIf Len(cNumero) == 11 //CPF
895             aNumero := { Val(Left(cNumero,3)), Val(SubStr(cNumero,4,3)), Val(SubStr(cNumero,7,3)) }
896         EndIf
897
898         For nNumPos := 1 To 3
899             nNumero := Int(aNumero[nNumPos] / 36)
900             cCodB36 += IIf(nNumero < 10, Str(nNumero, 1), Chr(nNumero + 55))
901             nNumero := aNumero[nNumPos] - (nNumero * 36)
902             cCodB36 += IIf(nNumero < 10, Str(nNumero, 1), Chr(nNumero + 55))
903         Next
904     Endif
905 Endif
906
907 Return cCodB36
908
909
910 /*{Protheus.doc} STIGetMun
911
912 Alimenta o array aCitiesComp com o codigo do IBGE + Nome da cidade e o array aCities com a descrição das cidades
913
914 @type function

```

```

915 @param cState, caracter, Estado base para buscar as informações das cidades e código do IBGE
916 @return Nil
917
918 /*/
919
920 Static Function STIGetMun(cState)
921 Local nX // Variável para For
922 Local nPosSta := aScan(aIBGE,{|x| x[1] == cState}) // Localiza o estado no array matriz com as informações dos estados
923 Local lEndFis := SuperGetMv("MV_SPEDEND",, .F.) // Se estiver como F refere-se ao endereço de Cobrança se estiver T ao endereço de
Entrega.
924 Local cCitSM0 := Alltrim(UPPER(If(!lEndFis, SM0->M0_CIDCOB, SM0->M0_CIDENT))) // Indica qual cidade será considerada, se a de entrega ou cobrança
925
926 If nPosSta > 0
927
928 //Zero o conteúdo das variáveis estáticas prevendo a possível troca de estado, não acumulando com as cidades anteriormente carregadas.
929 //Obs: o Array deverá sempre nascer com o esqueleto moldado, fazendo com que a primeira posição do TextBox sempre esteja em branco forçando a alteração
930 //desta forma o valid é chamado dando sequência a algumas ações em cadeia.
931 aCities := {cSpaceMun}
932 aCitiesComp := {{cSpaceCod,cSpaceMun}}
933 aSelectCit := {cSpaceCod,cSpaceMun}
934
935 //Caso encontre a cidade da SM0 coloco ela em primeiro lugar para facilitar a navegação.
936 If nPosCitSM0 := aScan(aIBGE[nPosSta][2],{|x| Alltrim(x[2]) == cCitSM0})
937 Aadd(aCitiesComp,{aIBGE[nPosSta][2][nPosCitSM0][1],aIBGE[nPosSta][2][nPosCitSM0][2]})
938 Aadd(aCities,aIBGE[nPosSta][2][nPosCitSM0][2])
939 EndIf
940
941 For nX := 1 To Len(aIBGE[nPosSta][2])
942 If !(nPosCitSM0 <> 0 .And. cCitSM0 == Alltrim(aIBGE[nPosSta][2][nX][2]))
943 Aadd(aCitiesComp,{aIBGE[nPosSta][2][nX][1],aIBGE[nPosSta][2][nX][2]})
944 Aadd(aCities,aIBGE[nPosSta][2][nX][2])
945 EndIf
946 Next
947
948 EndIf
949
950 //Caso não tenha selecionado nenhum estado limpo o TextBox com os municípios
951 If Empty(cState)
952 oMunicipio:SetItems({cSpaceMun})
953 Else
954 oMunicipio:SetItems(aCities)
955 EndIf
956
957 Return
958
959 /*/{Protheus.doc} STIFillCit
960
961 Função responsável por preencher o array aSelectCit com as informações de Código do IBGE da cidade e cidade selecionada.
962
963 @type function
964 @return Nil
965

```

```
966 /*/  
967  
968 Static Function STIFillCit()  
969  
970 Local nPosSelect      := Iif(oMunicipio:nAt == 0, 1, oMunicipio:nAt)      //Retorna a posição selecionada no objeto  
971 Local nPosCitComp     := aScan(aCitiesComp,{|x| x[2] == oMunicipio:aItems[nPosSelect]}) //Retorna a posição da cidade seleciona no array de "cidades completo"  
972  
973 //Reinício a variavel estatica  
974 aSelectCit := {}  
975  
976 aAdd(aSelectCit,aCitiesComp[nPosCitComp][1]) // Código do IBGE  
977 aAdd(aSelectCit,aCitiesComp[nPosCitComp][2]) // Descrição da Cidade  
978  
979 Return
```

Componentes/Interface/STIInfoCNPJ.prw

415 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWInfoCNPJ.CH"
4 #INCLUDE "POSCSS.CH"
5 #INCLUDE "STPOS.CH"
6
7 //-----
8 /*{Protheus.doc} STICustomerSelection
9 Funcao responsavel por chamar a troca do painel atual pelo painel de selecao de clientes.
10
11
12 */
13 //-----
14
15 Function STIInfoCNPJ( cCgcCli, cNome, cEnd, lInfoEnd,;
16                     lTelaFimVnd)
17
18 STIExchangePanel( { || STIPanInfoCNPJ( cCgcCli, cNome, cEnd, lInfoEnd,;
19                                     lTelaFimVnd) } )
20
21 STIChangeCssBtn()
22
23 Return
24
25 //-----
26 /*{Protheus.doc} STIPanInfoCNPJ
27 Funcao responsavel pela criacao do Painel de selecao de clientes.
28
29 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
30 */
31 //-----
32
33 Static Function STIPanInfoCNPJ(cCgcCli, cNome, cEnd, lInfoEnd,;
34                               lTelaFimVnd)
35
36 Local oPanelMVC      := STIGetPanel()           // Objeto do Panel onde a interface de selecao de clientes sera criada
37 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
38
39 Local oLblCab         := Nil                    // Objeto do label "Seleção de Cliente"
40 Local nVertCab        := oMainPanel:nHeight/100
41
42 Local oLblCGC         := Nil
43 Local nPosAlt1        := oMainPanel:nHeight/16.8
44
45 Local oCgc            := Nil                    // Objeto para digitar o CGC
46 Local nPosAlt2        := oMainPanel:nHeight/12
47
48 Local oLblNome        := Nil
49 Local nPosAlt3        := oMainPanel:nHeight/8.4
50
51 Local oNome           := Nil                    // Objeto para digitar o Nome do Cliente
52 Local nPosAlt4        := oMainPanel:nHeight/7.08

```

```

53
54 Local oLblEnd          := Nil
55 Local nPosAlt5         := oMainPanel:nHeight/4.4
56
57 Local oEnd             := Nil                      // Objeto para digitar o Endereço do Cli
58 Local nPosAlt6         := oMainPanel:nHeight/4
59
60 Local oBtnOk           := Nil
61 Local oBtnCPF          := Nil                      // Boão para acionamento do PinPad para digitação
62
63 Local nPosHor2         := oMainPanel:nWidth/2.8
64
65 Local oTEF20           := Nil
66
67 Local nOpcDigCPF       := SuperGetMV("MV_LJDGCPF",1) // Forma de digitação do CPF - 1 - Pela tela, 2 - Opção Pelo PinPad
68
69 Local bActionOk        :={||} }                  //Bloco a ser executado ao clicar no botão 'OK'
70
71 Default cCgcCli        := SPACE(TamSX3("A1_CGC")[1])
72 Default cNome          := SPACE(TamSX3("A1_NOME")[1])
73 Default cEnd           := SPACE(TamSX3("A1_ENDENT")[1])
74 Default lInfoEnd       := .F.
75 Default lTelaFimVnd    := .F.
76
77 ParamType 0 var cCgcCli As Character Default ""
78 ParamType 1 var cNome   As Character Default ""
79 ParamType 2 var cEnd    As Character Default ""
80 ParamType 3 var lInfoEnd As Logical   Default .F.
81
82 // Ativa teclas de atalho
83 STIBtnActivate()
84
85 // Como é a primeira tela , sempre seta que não é recebimento
86 STISetRecTit(.F.)
87
88 //Abre tela para digitar
89 DEFINE FONT oBold BOLD
90 DEFINE FONT oFontText NAME "Courier New" SIZE 09,20
91
92 STIChangeCssBtn()
93
94 If !STFGetCfg("lUseSat")
95     oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0027},oMainPanel,,,,,.T.,,,) // "Deseja informar: CPF/CNPJ/PASSAPORTE/RNE no Comprovante de Venda ?"
96 Else
97     oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0008},oMainPanel,,,,,.T.,,,) // "Deseja informar o CPF/CNPJ para impressão do Comprovante de Venda?"
98 EndIf
99
100 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
101
102 If !STFGetCfg("lUseSat")
103     oLblCGC := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0028},oMainPanel,,,,,.T.,,,) // "CPF/CNPJ/PASSAPORTE/RNE"
104 Else

```

```

105     oLblCGC := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0002},oMainPanel,,,,,T.,,,) // CPF/CNPJ
106 EndIf
107 oCgc := TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount()>0,cCgcCli:=u,cCgcCli)},oMainPanel,120,ALTURAGET,"@!",,,,,,T.,,,,,,"cCgcCli")
108
109 oLblCGC:SetCSS( POSCSS (GetClassName(oLblCGC), CSS_LABEL_FOCAL ))
110 oCgc:SetCSS( POSCSS (GetClassName(oCgc), CSS_GET_FOCAL ))
111
112 // Verifica se apresenta o botão do PinPad
113 If nOpcDigCPF == 2
114     oTEF20 := STBGetTEF()
115     If oTEF20 <> Nil
116         If (ValType(oTef20:oConfig:oComSitef) == "0" .AND. oTef20:oConfig:oComSitef:verPinPad() == 1) .OR. (oTEF20:oConfig:ISCCCD()) .OR. (ExistFunc("LjUsePayHub") .And.
            LjUsePayHub() .And. oTEF20:oConfig:ISPayHub())
117
118             bActionOk := {|| oBtnCPF:Disable() ,cCgcCli := STWDitaCPF(), oMainPanel:Refresh(), IIf(!Empty(cCgcCli) .AND. !lInfoEnd,oBtnOk:SetFocus(),IIF(lInfoEnd .AND.
                !Empty(cCgcCli),oNome:SetFocus(),Nil)), oBtnCPF:Enable() }
119             oBtnCPF := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0022,oMainPanel, bActionOk, LARGBTN,ALTURABTN,,,,T.) //"CPF via PinPad"
120             oBtnCPF:SetCSS( POSCSS (GetClassName(oBtnCPF), CSS_BTN_FOCAL ))
121
122         EndIf
123     EndIf
124 Endif
125
126 If lInfoEnd
127     @ POSVERT_LABEL2,POSHOR_1 SAY oLblNome VAR STR0009 OF oMainPanel PIXEL //"Nome do Cliente"
128     @ POSVERT_GET2,POSHOR_1 MSGET oNome VAR cNome SIZE 120,ALTURAGET FONT oFontText OF oMainPanel PIXEL
129
130     @ nPosAlt5,POSHOR_1 SAY oLblEnd VAR STR0004 OF oMainPanel PIXEL //"Endereco"
131     @ nPosAlt6,POSHOR_1 MSGET oEnd VAR cEnd SIZE 120,ALTURAGET FONT oFontText OF oMainPanel PIXEL
132
133     oLblNome:SetCSS( POSCSS (GetClassName(oLblNome), CSS_LABEL_FOCAL ))
134     oNome:SetCSS( POSCSS (GetClassName(oNome), CSS_GET_FOCAL ))
135
136     oLblEnd:SetCSS( POSCSS (GetClassName(oLblEnd), CSS_LABEL_FOCAL ))
137     oEnd:SetCSS( POSCSS (GetClassName(oEnd), CSS_GET_FOCAL ))
138     //to do: Quando template de Posto deve imprimir Placa/Km no Cupom Fiscal
139
140 EndIf
141
142 /* parametrizacao para que mostre a Tela de CPF no final do lançamento dos itens*/
143 oBtnOk := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0014,oMainPanel,;
144     {|| Iif(ExistFunc("StiMataObj"),StiMataObj(),Nil), IIF(STICGCConfirm(CCGCCLI, CNome, CEnd, lInfoEnd,Iif(lTelaFimVnd,.F.),));
145     IIF(lTelaFimVnd, STICallPayment(), .T.), Nil) }, ;
146     LARGBTN,ALTURABTN,,,,T.) //"Informar CPF"
147
148 oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk), CSS_BTN_FOCAL ))
149
150 oCgc:SetFocus()
151 If GetFocus() <> oCgc:HWND
152     oCgc:SetFocus()
153 Endif
154

```

```

155 Return oMainPanel
156
157 //-----
158 /*{Protheus.doc} STIValCGC
159 Valida Documento do cliente
160 @param      ExpC1 - CGC do cliente
161 @param      lShowRgIt - exibe tela de registro do item
162
163 @return      ExpL1 - Retorno da funcao T ou F
164 @obs
165 @sample
166 */
167 //-----
168 Function STICGCCConfirm(      CCGCCLI , CNome, cEnd, lInfoEnd ,;
169                          lShowRgIt, oCgc)
170 Local lRet      := .T.
171 Local nCallCPF  := SuperGetMV("MV_LJDCCLI",,0) //0 momento onde será mostrado o CPF na tela
172
173 Default lShowRgIt := .T.
174 Default oCgc = Nil
175
176 IF STIValCGC(@CCGCCLI, oCgc) .AND. STIVaCamp(CCGCCLI, CNome, cEnd, lInfoEnd)
177     STI7InfCPF(.T.) // Seta para o componente de registro de item que a tela de CPF ja foi chamada
178     If Len(Alltrim(cCGCCli)) > 9
179         STDSPBasket("SL1","L1_CGCCLI",cCGCCli)
180     Else
181         STDSPBasket("SL1","L1_PFISICA",cCGCCli) //Cliente estrangeiro
182     EndIf
183     If (nCallCPF = 0 .OR. nCallCPF = 1) .OR. (!lShowRgIt .AND. (nCallCPF = 2 .OR. nCallCPF = 3))
184         STD7CPFOverReceipt(Alltrim(CCGCCLI),Alltrim(cNome),Alltrim(cEnd)) // Guarda as informacoes digitadas, que serao utilizadas posteriormente, na abertura do cupom
185     EndIf
186
187     If lShowRgIt
188         STIRegItemInterface(.T.) // Chama a tela de registro de item
189     EndIf
190 Else
191     lRet := .F.
192 EndIf
193
194 Return lRet
195
196 //-----
197 /*{Protheus.doc} STIValCGC
198 Valida Documento do cliente
199 @param1      ExpC1 - CGC do cliente
200 @return      ExpL1 - Retorno da funcao T ou F
201 @obs
202 @sample
203 */
204 //-----
205 Static Function STIValCGC(cCgc, oCgc)
206

```

```

207 Local lRet      := .F.                // Retorno da funcao
208 Local cVerCgc   := AllTrim(cCgc)      // CGC do cliente
209 Local cNumCNPJ  := Space(TamSX3("A1_CGC"))[1] // Varivel para armazenar o retorno da funcao CGC
210 Local nObrigaCPF:= 0                  // Qual item obriga informar o CPF
211
212 Default cCgc    := ""
213
214 ParamType 0 var cCgc As Character Default ""
215
216 If !Empty(cVerCgc)
217
218     //Retira caracteres que nao sao usados
219     cVerCgc := StrTran(cVerCgc,".", "")
220     cVerCgc := StrTran(cVerCgc,"-", "")
221     cVerCgc := StrTran(cVerCgc,"/", "")
222     cVerCgc := StrTran(cVerCgc,"*", "")
223     cVerCgc := Alltrim(cVerCgc)
224
225     //Valida se o CGC digitado eh valido
226     If CGC(cVerCgc, @cNumCNPJ,.F.) .And. !STWVldNRep(cVerCgc)
227         lRet := .T.
228         cCgc := cVerCgc
229         STFCleanInterfaceMessage()
230     ElseIf (!IsNumeric(cVerCgc) .And. Len(cVerCgc) >=5 .And. Len(cVerCgc)<= 9) .And. !STFGetCfg("lUseSat")
231         lRet := .T.
232         cCgc := cVerCgc
233         STFCleanInterfaceMessage()
234     Else
235         STFMessage("STIValCGC","STOP", STR0029 ) //"CPF/CNPJ/Documento inválido!"
236         STFShowMessage("STIValCGC")
237         lRet := .F.
238         cCgc := Space(18)
239         If ValType(oCgc) == '0'
240             oCgc:SetFocus()
241         EndIf
242     EndIf
243
244 Else
245     If ExistFunc("Lj950ImpCpf") .And. Lj950ImpCpf(STDGPBasket("SL1","L1_VLRTOT"),@nObrigaCPF)
246
247         If nObrigaCPF == 1
248             STFMessage("STIValCGC_1","POPUP", STR0024 ) // "Segundo legislação é obrigatório informar documento do cliente (CPF/CNPJ)."
```

```

258     Else
259         //Se estiver em branco, valida a insercao
260         lRet := .T.
261         STFCleanInterfaceMessage()
262     EndIf
263 EndIf
264
265 Return lRet
266
267
268 //-----
269 /*{Protheus.doc} STIVaCamp
270 Valida campos
271 @param1      ExpC1 - CGC do Cliente
272 @param2      ExpC2 - Nome do Cliente
273 @param3      ExpC3 - Endereco
274 @param4      ExpL1 - Informa Endereco
275 @return      ExpL1 - Retorno da funcao T ou F
276 @obs
277 @sample
278 */
279 //-----
280 Static Function STIVaCamp(cCgc, cNome, cEnd, lInfoEnd)
281
282 Local lRet := .T. // Retorno da Função
283
284 Default cCgc := ""
285 Default cNome := ""
286 Default cEnd := ""
287 Default lInfoEnd := .F.
288
289 ParamType 0 var cCgc      As Character Default ""
290 ParamType 1 var cNome     As Character Default ""
291 ParamType 2 var cEnd      As Character Default ""
292 ParamType 3 var lInfoEnd  As Logical   Default .F.
293
294 STFMessage("STIVaCamp","STOP", STR0007 ) //"Nome/Endereço não informado"
295
296 //Quando PAF, verifica Nome e/ou Endereço infomado, obriga informar o CPF/CNPJ
297 If lInfoEnd
298     lRet := !( Empty(cCgc) .AND. (!Empty(cNome) .OR. !Empty(cEnd)) )
299
300     If !lRet
301         STFShowMessage("STIVaCamp")      // "CPF/CNPJ inválido!"
302     EndIf
303 EndIf
304
305 Return lRet
306
307
308 //-----
309 /*{Protheus.doc} STWGetCGC

```

```

310 Captura da tela dados do cliente
311 @param1      ExpC1 - CGC do Cliente
312 @param2      ExpC2 -Nome do Cliente
313 @param3      ExpC3 - Endereco
314 @param4      ExpL1 - Informa Endereco
315 @return      ExpC1 - CGC do Cliente
316 @obs
317 @sample
318 */
319 //-----
320 Function STWGetCGC( cCgcCli, cNome, cEnd, lInfoEnd )
321
322 Local oDlg      := Nil                // Objeto da Tela
323 Local oCgc      := Nil                // Objeto para digitar o CGC
324 Local oNome     := Nil                // Objeto para digitar o Nome do Cliente
325 Local oEnd      := Nil                // Objeto para digitar o Endereço do Cliente
326 Local oTPanel   := Nil                // Cria o Panel para digitacao dos dados
327 Local oFontText := Nil                // Fonte do texto
328 Local nTamTela  := 420                // Tamanho padrão da tela
329 Local nAltBotao := 35                 // Altura padrão do botão
330 Local lCont     := .F.                // Continua apos digitar CGC
331
332 Default cCgcCli := SPACE(TamSX3("A1_CGC")[1])
333 Default cNome   := SPACE(TamSX3("A1_NOME")[1])
334 Default cEnd    := SPACE(TamSX3("A1_ENDENT")[1])
335 Default lInfoEnd := .F.
336
337 ParamType 0 var cCgcCli      As Character Default ""
338 ParamType 1 var cNome        As Character Default ""
339 ParamType 2 var cEnd         As Character Default ""
340 ParamType 3 var lInfoEnd     As Logical   Default .F.
341
342
343 //Abre tela para digitar
344 DEFINE FONT oBold BOLD
345 DEFINE FONT oFontText NAME "Courier New" SIZE 09,20
346
347
348 //Ajusta Tamanho da Tela e posicao do botao
349 If lInfoEnd
350     nTamTela := 540
351     nAltBotao := 97
352 EndIf
353
354 // "INFORME O CPF / CNPJ PARA IMPRESSÃO"
355 DEFINE MSDIALOG oDlg TITLE STR0001 FROM 323,412 TO nTamTela,738 PIXEL STYLE DS_MODALFRAME STATUS // "INFORME O CPF / CNPJ PARA IMPRESSÃO"
356
357 oTPanel := TPanel():New(0,0,"",oDlg,NIL,.T.,.F.,NIL,NIL,170,060,.T.,.F.)
358 oTPanel:Align := CONTROL_ALIGN_ALLCLIENT
359
360 // "Digite abaixo:"
361 @ 003,007 SAY STR0002 SIZE 050, 007 OF oTPanel PIXEL // "CPF/CNPJ"

```

```

362 @ 010,007 MSGET oCgc VAR cCgcCli SIZE 150,10 FONT oFontText OF oTPanel PIXEL
363
364 If lInfoEnd
365 @ 023,007 SAY STR0003 SIZE 050, 007 OF oTPanel PIXEL //"Cliente"
366 @ 030,007 MSGET oNome VAR cNome SIZE 150,10 FONT oFontText OF oTPanel PIXEL
367
368 @ 043,007 SAY STR0004 SIZE 050, 007 OF oTPanel PIXEL //"Endereco"
369 @ 051,007 MSGET oEnd VAR cEnd SIZE 150,10 FONT oFontText OF oTPanel PIXEL
370
371 //to do: Quando template de Posto deve imprimir Placa/Km no Cupom Fiscal
372
373 @ 085,007 SAY STR0005 SIZE 150, 007 COLOR CLR_RED FONT oBold OF oTPanel PIXEL //"Imprime dados no cupom fiscal?"
374
375 EndIf
376
377 DEFINE SBUTTON FROM nAltBotao, 104 TYPE 1 ENABLE OF OTPANEL ACTION (LCONT := .T. , IIF( STICGCConfirm(@CCGCCLI, CNOME, CEND, lInfoEnd) ,ODLG:END(),NIL))
378 DEFINE SBUTTON FROM nAltBotao, 134 TYPE 2 ENABLE OF OTPANEL ACTION (LCONT := .F. , ODLG:END() , STI7InfCPF(.T.), STIRegItemInterface(.T.) )
379
380 ACTIVATE MSDIALOG oDlg CENTERED
381
382 If !lCont
383 cCgcCli := ""
384 cNome := ""
385 cEnd := ""
386 //Caso seja Venda Assistida, alimenta variavel de memoria
387 EndIf
388
389 Return cCgcCli
390
391 //-----
392 /*{Protheus.doc} STWldNRep
393 Validar se o usuario esta digitando os mesmos numeros na informacao CPF ou CNPJ
394 @param1 ExpC1 - CGC do Cliente
395 @param2
396 @param3
397 @param4
398 @return Valor Logico
399 @obs
400 @sample
401 */
402 //-----
403 Function STWldNRep(cCNPJCPF)
404 Local lRet := .F.
405
406 If !(Len(cCNPJCPF)=11 .Or. Len(cCNPJCPF)=14)
407 lRet := .T.
408 EndIf
409
410 // Se repetir todos os numeros informados o retorno sera invalido
411 If Repl(Substr(cCNPJCPF,1,1), If(Len(cCNPJCPF) == 11, 11, 14)) == cCNPJCPF
412 lRet := .T.
413 Endif

```

```
414  
415 Return(lRet)
```

Componentes/Interface/STIItemCheck.prw

176 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "FWCSS.CH"
5 #INCLUDE "STPOS.CH"
6 #INCLUDE "STIITEMCHECK.CH"
7
8 Static lChkAtiv := .F.
9
10 //-----
11 /*{Protheus.doc} STIItemCheck
12 Função para montar tela para conferencia de item
13 @param      SL2
14 @return      lRet
15 */
16 //-----
17 Function STIItemChk( aSL2, lZeraPay )
18 Local lRet      := .T.
19 Local aMVLJITOP := StrToKarr(SuperGetMv("MV_LJITOP", , "*", "-", "%", "?"), ",") // Parametro com os codigos dos caracteres de atalho de operacoes do item no registro do item
20 Local cAtalhosItem := ""
21 Local cGetCodProd  := Space(TamSX3("L1_PRODUTO")[1]) // utiliza tamanho do campo do cabeçalho, pode utilizar código de barras e esse campo ser maior que o L2_PRODUTO e
    impossibilitar a pesquisa
22 Local aListItem    := {}
23 Local oGetCodProd   := Nil
24 Local oLbLcodProd   := Nil
25 Local oMsgErro     := Nil
26 Local oDlgCheck     := Nil
27 Local oListCheck    := Nil
28 Local oBtnOk        := Nil
29 Local oBtnPesq      := Nil
30 Local oBtnDelIt     := Nil
31 Local oBtnCancel    := Nil
32 Local aFontes       := STFDefFont()
33 Local nX            := 0
34
35 Default lZeraPay    := .F.
36
37
38 aListItem := STBIItLstCk(aSL2)
39
40 //Somente abre a tela se tiver item pendente para bipar
41 For nX := 1 to Len(aListItem)
42     If aListItem[nX][6]
43         lRet := .F.
44         Exit
45     EndIf
46 Next nX
47
48 If !lRet
49
50     //desabilita controles da tela principal. Motivo: Se o usuário utilizar uma tecla de atalho, pode dar erro na tela de conferência de item
51     STIBtnDeActivate()

```

```

52
53   lChkAtiv := .T.
54
55   DEFINE MSDIALOG oDlgCheck TITLE STR0001 STYLE DS_MODALFRAME FROM 0,0 TO 650,670 PIXEL OF oMainWnd  //"Conferência de Item"
56   @ 08,08 LISTBOX oListCheck FIELDS HEADER "",STR0002, STR0003, STR0004, STR0005, STR0006 FIELDSIZES 14,15,140,43,43,25 SIZE 320,260 PIXEL FONT aFontes[3] OF oDlgCheck
57   //"Item" //"Descrição" //"Qtde Orcamento" //"Qtde Conferida" //"Entrega"
58
59   oDlgCheck:lEscClose      := .F. //Nao permite sair ao pressionar a tecla ESC.
60
61   oListCheck:SetArray(aListItem)
62   oListCheck:bLDbClick := {|| BrwLegenda(STR0007,STR0008,STIIItCkLeg()) } //"Status Conferência" //"Legenda"
63   oListCheck:bLine := {|| {;
64       STIIItCkCor(aListItem[oListCheck:nAt][6],aListItem[oListCheck:nAt][3]-aListItem[oListCheck:nAt][4],aListItem[oListCheck:nAt][8]);
65       aListItem[oListCheck:nAt][1];
66       aListItem[oListCheck:nAt][2];
67       aListItem[oListCheck:nAt][3];
68       aListItem[oListCheck:nAt][4];
69       aListItem[oListCheck:nAt][5];
70   }}
71   oListCheck:Refresh()
72
73   If Len(aMVLJITOP) > 0 .AND. !Empty(aMVLJITOP[1])
74       cAtalhosItem := STR0009 + " ( "+AllTrim(aMVLJITOP[1])+") / " + STR0010  //"Quantidade" //"Código do Produto"
75       oLbHelpGet := TSay():New(270,009,{|| cAtalhosItem},oDlgCheck,,,,,T,,,,)
76       oLbHelpGet:SetCSS( POSCSS (GetClassName(oLbHelpGet), CSS_LABEL_NORMAL ))
77   EndIf
78
79   oLbCodProd := TSay():New(283,009,{|| STR0010 },oDlgCheck,,,,,T,,,,) //"Código do Produto"
80   oLbCodProd:SetCSS( POSCSS (GetClassName(oLbCodProd), CSS_LABEL_FOCAL ))
81
82   oMsgErro := TSay():New(295,120,{|| Space(50) },oDlgCheck,,,,,T,,,,)
83   oMsgErro:SetCSS( POSCSS (GetClassName(oMsgErro), CSS_LABEL_FOCAL ))
84   oMsgErro:lVisible := .F.
85
86   oGetCodProd := TGet():New( 292, 009, { | u | If( PCount() == 0, cGetCodProd, cGetCodProd := u ) },oDlgCheck, 110, 010, "!",@,, 0, 16777215,,,F,,,T,,,F,,,F,,,F,,,F,,,F.
87   ,,"cGetCodProd",,,,/*lHasButton*/ .T. )
88   oGetCodProd:cF3 := "SB1"
89   oGetCodProd:bLostFocus := { || IIF(!Empty(cGetCodProd),STBIItPesq(@cGetCodProd, @oGetCodProd, @oListCheck, @oMsgErro, @oBtnOk),.T.) }
90   oGetCodProd:SetCSS( POSCSS (GetClassName(oGetCodProd), CSS_GET_NORMAL ))
91   oGetCodProd:SetFocus()
92
93   oBtnPesq := TButton():New( 310,118, STR0012, oDlgCheck,{|| PosSB1(aListItem[oListCheck:nAt][7]), cGetCodProd := IIF(ConPad1(,,"SB1"),SB1->B1_COD,"")
94   },50,15,,,F,,,T,,,F,,,F,,,F. ) //"Pesquisar"
95   oBtnPesq:SetCSS( POSCSS (GetClassName(oBtnPesq), CSS_BTN_FOCAL ))
96
97   oBtnDelIt := TButton():New( 310,173, STR0013, oDlgCheck,{|| STBIItCkDel(@oListCheck, @oMsgErro) },50,15,,,F,,,T,,,F,,,F,,,F. ) //"Cancelar Item"
98   oBtnDelIt:SetCSS( POSCSS (GetClassName(oBtnDelIt), CSS_BTN_FOCAL ))
99
100  oBtnOk := TButton():New( 310,225, STR0011, oDlgCheck,{|| lRet := STBIItCkOk( oListCheck, oMsgErro, @aSL2, @lZeraPay ),
101  IIF(lRet,Iif(Empty(aSL2),(oDlgCheck:End(),lChkAtiv := .T.,lRet := .F.),(oDlgCheck:End(),lChkAtiv := .F.)),lChkAtiv := .F.) },50,15,,,F,,,T,,,F,,,F,,,F. )
102  //"Confirmar"
103  oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk), CSS_BTN_FOCAL ))

```

```

99      oBtnCancel := TButton().New( 310,280, STR0014, oDlgCheck, {|| IIf( ApMsgYesNo( STR0015 + Chr(13) + Chr(10)+ STR0016),oDlgCheck:End(),Nil), lChkAtiv := .T.
100      },50,15,,,F.,.T.,.F.,,.F.,,.F.,,.F. ) //"Cancelar" //"Deseja cancelar a conferência de Itens?" //" (Se confirmado, a importação do orçamento será cancelada)"
101      oBtnCancel.SetCSS( POSCSS (GetClassName(oBtnOk) , CSS_BTN_FOCAL ))
102
103      ACTIVATE MSDIALOG oDlgCheck CENTERED
104
105      //Habilita controles da tela principal
106      STIBtnActivate()
107
108  EndIf
109
110  Return lRet
111
112
113  //-----
114  /*{Protheus.doc} STIIItCkLeg
115  Função para montar legenda de status dos itens
116  @param
117  @return      aRet
118  */
119  //-----
120  Static Function STIIItCkLeg( )
121  Local aRet := {}
122
123  aAdd( aRet,{"BR_VERMELHO" ,STR0017}) //"Pendente conferência"
124  aAdd( aRet,{"BR_VERDE" ,STR0018}) //"Conferência realizada"
125  aAdd( aRet,{"BR_AMARELO" ,STR0019}) //"Sem controle de conferência"
126  aAdd( aRet,{"BR_PRETO" ,STR0020}) //"Cancelado"
127
128  Return aRet
129
130  //-----
131  /*{Protheus.doc} STIIItCkCor
132  Função para retornar cores da legenda de status dos itens
133  @param      lCheck, nQtdPend, lAtivo
134  @return      aRet
135  */
136  //-----
137  Static Function STIIItCkCor( lCheck, nQtdPend, lAtivo )
138  Local oRet
139
140  If !lAtivo
141      oRet := LoadBitmap(GetResources(), "BR_PRETO") //Cancelado
142  ElseIf !lCheck
143      oRet := LoadBitmap(GetResources(), "BR_AMARELO") //Sem controle de conferência
144  ElseIf nQtdPend > 0
145      oRet := LoadBitmap(GetResources(), "BR_VERMELHO") //Pendente conferência
146  Else
147      oRet := LoadBitmap(GetResources(), "BR_VERDE") //Conferência realizada
148  EndIf
149

```

```
150 Return oRet
151
152 //-----
153 /*{Protheus.doc} STIIItCkAct
154 Retorna a variável lChkAtiv
155 @param
156 @return      lChkAtiv
157 */
158 //-----
159 Function STIIItCkAct()
160
161 Return lChkAtiv
162
163 //-----
164 /*{Protheus.doc} PosSB1
165 De acordo com o item posicionado na grid, posiciona na SB1
166 @param
167 @return      .T.
168 */
169 //-----
170 Function PosSB1(cCodProd)
171 Default cCodProd := ""
172
173 SB1->(DbSetOrder(1))
174 SB1->(DbSeek(xFilial("SB1")+cCodProd))
175
176 Return .T.
```

Componentes/Interface/STIItemRegister.PRW

1111 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIITEMREGISTER.CH"
5 #INCLUDE "STPOS.CH"
6
7 Static oLbProdDados:= Nil
8 Static oPanGetProd := Nil
9 Static cPictValUnit := PesqPict("SL2","L2_VRUNIT")
10 Static nTotItem := 0
11 Static cQtdDesc := ''
12 Static oImg := Nil
13 Static lFinServ := SuperGetMv("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
14 Static lStCodDig := ExistBlock("STCDDIG") // PE para poder alterar o código digitado no PDV
15 Static lStbExistVP := ExistFunc("STBExistVP")
16 Static lVend := Nil
17 Static oCodProd := Nil // Código Produto
18 Static oButton := Nil
19 Static lLJ8099 := ExistBlock("LJ8099") //Ponto de Entrada que permite retornar o percentual de desconto que sera aplicado sobre o item
20 Static aMVLJITOP := STIOperIT()
21 Static lIsSat := STFGetCfg("lUseSAT",.F.)
22 Static lAtivMotor := ExistFunc("PshAtivMTP") .AND. PshAtivMTP() // Verifica se o motor de promoções esta ativo
23 //-----
24 /*{Protheus.doc} STIRegItemInterface
25 Pannel Registro de Item.
26
27 @param
28 @return
29 @obs
30 @sample
31 */
32 //-----
33 Function STIRegItemInterface(lLjPdVen, lDefaultQD)
34 Local lProssegue := .T.
35 Local nTpPedeCnpj := 0
36 Local lLjInfDocCli := ExistFunc("LjInfDocCli")
37 Local lEmitNFCe := ExistFunc("LjEmitNFCe") .AND. LjEmitNFCe() // Indica a utilizacao da NFC-e
38 Local lCRdesItTt := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
39 Local lDesligaRD := .F. // Desabilita a Regra de Desconto do Varejo
40
41 Default lLjPdVen := .F.
42 Default lDefaultQD := .T. // Seta Quantidade e Desconto para Default?
43
44 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
45     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
        STDGPBasket('SL1','L1_LOJA'))
46 EndIf
47
48 lVend := IIF(lVend == Nil,SuperGetMV("MV_LJPDVEN",,.F.),lVend) //Variavel para ativar o vendedor
49

```

```

50 oLblProdDados := Nil
51
52 If lDefaultQD
53     STBSetDefQuant() // Seta quantidade padrao
54     STBDefItDiscount() // Seta desconto padrao
55 EndIf
56
57 If !lLjInfDocCli
58     lProssegue := .T.
59 Else
60     If lLjInfDocCli
61         nTpPedeCnpj := LjInfDocCli()
62
63         If nTpPedeCnpj <> 2
64
65             If lEmitNFCe .AND. nTpPedeCnpj == 1 .AND. !STIGInfCPF() .AND. Isincallstack("STIConfirmProd")
66                 STI7InfCPF(.T.)
67             Endif
68             //Verifico se sera necessario apresentar a tela para informar o CPF/CNPJ, caso contrario chamo o registro de item
69             lProssegue := !STWInfoCNPJ()
70         EndIf
71     EndIf
72 EndIf
73
74 If lProssegue
75     If lVend .AND. lLjPdVen .And. (!ExistFunc("STIGSelVend") .Or. !STIGSelVend()) // Determina se apresenta tela para Alterar o vendedor na venda no POS
76         STISalesmanSelection() // Apresenta tela para Alterar o vendedor na venda no POS
77     Else
78         If !lVend .Or. !lLjPdVen
79             STIBtnActivate() //Ativa os botoes da tela
80         EndIf
81
82         //Para o motor de promoções sempre limpar os desconto quando voltar ao carrinho
83         STIExchangePanel( { ||IIF(lAtivMotor, Eval({||STIClearDisc(.T.), PshCodMTP(), PshClearPromo()}), Iif(lCRdesItTt .AND. !lDesligaRD, Eval({||PshClearPromo()}), .T.)),
            STIPanItemRegister() } )
84     EndIf
85 EndIf
86
87 Return
88
89 //-----
90 /*{Protheus.doc} STIPanItemRegister
91 Painel Registro de Item
92
93 @param
94 @return
95 @obs
96 @sample
97 */
98 //-----
99 Function STIPanItemRegister()
100

```

```

101 Local oPanelMVC      := Nil                                // Objeto Painei MVC
102 Local oLblCab        := Nil                                // Objeto Cabeçalho Label
103 Local oLblCodProd    := Nil                                // Objeto Código Produto
104 Local cGetProd       := Space(40)                          // Descrição Produto
105 Local oLblHelpGet    := Nil                                // Objeto Label Help Get
106 Local oLblInfProd    := Nil                                // Objeto Label Info Produto
107 Local cDados         := STR0010 + CRLF + CRLF + CRLF + STR0009 // Descrição do Produto ### "Valor"
108 Local aCustomer      := IIF(ExistFunc('STDFindCust'),STDFindCust(),{}) //Carrega informacoes do cliente Nome/Cpf
109 Local aSalesMan      := IIF(ExistFunc('STDFindSMan'),STDFindSMan(),{}) //Carrega o nome do vendedor
110 Local lMobile        := STFGGetCfg("lMobile" , .F. )       //PDV em Versao Mobile
111 Local oGetQtde       := Nil                                // TGet quantidade do item
112 Local oGetVlrDesc    := Nil                                // TGet Valor desconto
113 Local oGetPercDesc   := Nil                                // TGet percentual de desconto
114 Local nGetQtde       := 1                                  // Quantidade
115 Local nGetVlrDesc    := 0                                  // Valor de desconto
116 Local nGetPercDesc   := 0                                  // Percentual de desconto
117 Local oButRegItem    := Nil                                //Botao registrar item
118 Local oButRegCamItem:= Nil                                //Botao registrar item com camera
119 Local bRegItem       := {||}                               //Bloco para registro do item
120 Local bCamRegItem    := {||}                               //Bloco para registro do item com camera
121 Local cPathImg       := AllTrim(SuperGetMV("MV_LJIMAGE", .F., "")) // Caminho da imagem do produto Opcional
122 Local cAtalhosItem   := ""
123 Local aCartCRD       := IIF(ExistFunc('STDFindCart'),STDFindCart(),{}) //Carrega informacoes do cartão utilizado no SIGACRD
124 Local oLblCartCrd    := nil                                //Objeto Label Cartão (SIGACRD)
125 Local aRes           := GetScreenRes()                      // Retorna a Resolução atual da tela.
126 Local oRaas          := Nil
127 Local oFidelityC     := Nil
128
129 // -- Limpeza dos descontos do FidelityCore
130 If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
131   oRaas := STBGetRaas()
132
133   If ValType(oRaas) == "0" .AND. oRaas:ServiceIsActive("TFC")
134
135     oFidelityC := oRaas:GetFidelityCore()
136
137     If oFidelityC:ChoseToUse()
138       oFidelityC:Clean()
139     EndIf
140
141     If STDGPBasket( "SL1", "L1_DESCFID" ) > 0
142       STIClearDisc(.T.,STDGPBasket( "SL1", "L1_DESCFID"))
143     Endif
144
145   EndIf
146 EndIf
147
148 If !Empty(aMVLJITOP[1])
149   cAtalhosItem += "Quantidade ( "+AllTrim(aMVLJITOP[1])+ " ) / "
150 EndIf
151
152 If !Empty(aMVLJITOP[2])

```

```

153     cAtalhosItem += "Valor Desconto ( "+AllTrim(aMVLJITOP[2])+ " ) OU "
154 EndIf
155
156 If !Empty(aMVLJITOP[3])
157     cAtalhosItem += "Perc. Desconto ( "+AllTrim(aMVLJITOP[3])+ " ) / Código do Produto "
158 EndIf
159
160 nTotItem := 0           // Inicia o totalizador para a nova venda
161
162 lMobile := ValType(lMobile) == "L" .AND. lMobile
163
164 oPanelMVC      := STIGetPanel()
165 oPanGetProd    := TPanel():New(00,00,"",oPanelMVC,;
166                               ,,,,;
167                               ,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
168 STIChangeCssBtn('oBtnRegItem')
169
170 oLblProdDados := Nil
171 //Verifica se e versao mobile
172 If lMobile
173
174     /*/
175     Objetos
176     /*/
177     oLblCab := TSay():New(POSVERT_CAB,POSHOR_1,{|| STR0021 + " > " + STR0001},oPanGetProd,,,,,.T.,,,) //"Vendas" "Registro de Item"
178     oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
179
180
181     //Quantidade
182     oLblQuantidade := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0022},oPanGetProd,,,,,.T.,,,100) //"Quantidade"
183     oLblQuantidade:SetCSS( POSCSS (GetClassName(oLblQuantidade), CSS_LABEL_NORMAL ))
184
185
186     oGetQtde:= TGet():New( POSVERT_GET1,POSHOR_1,{|u| If(PCount())>0,nGetQtde:=u,nGetQtde}}, ;
187                         oPanGetProd,40 ,ALTURAGET,"@E 999",{|| nGetQtde > 0 .AND. nGetQtde <= 999 },,,,,.T.,,,,,,"nGetQtde",,,,,.T.)
188     oGetQtde:SetCSS( POSCSS (GetClassName(oGetQtde), CSS_GET_NORMAL ))
189
190
191     //Get de valor de desconto
192     oLblVlrDesc := TSay():New(POSVERT_LABEL1,oPanelMVC:nWidth/7,{||STR0023},oPanGetProd,,,,,.T.,,,100) //"Valor de Desconto"
193     oLblVlrDesc:SetCSS( POSCSS (GetClassName(oLblVlrDesc), CSS_LABEL_NORMAL ))
194
195     oGetVlrDesc:= TGet():New( POSVERT_GET1,oPanelMVC:nWidth/7,{|u| If(PCount())>0,nGetVlrDesc:=u,nGetVlrDesc}}, ;
196                         oPanGetProd,60 ,ALTURAGET,"@E 999,999.99",,,,,.T.,,,,,,"nGetVlrDesc",,,,,.T.)
197     oGetVlrDesc:SetCSS( POSCSS (GetClassName(oGetVlrDesc), CSS_GET_NORMAL ))
198
199     //Percentual de Desconto
200     oLblPercDesc := TSay():New(POSVERT_LABEL1,oPanelMVC:nWidth/3.5,{||STR0024},oPanGetProd,,,,,.T.,,,100) //"Percentual de Desconto"
201     oLblPercDesc:SetCSS( POSCSS (GetClassName(oLblPercDesc), CSS_LABEL_NORMAL ))
202
203     oGetPercDesc:= TGet():New( POSVERT_GET1,oPanelMVC:nWidth/3.5,{|u| If(PCount())>0,nGetPercDesc:=u,nGetPercDesc}}, ;
204                         oPanGetProd,60 ,ALTURAGET,"@E 99.99",,,,,.T.,,,,,,"nGetPercDesc",,,,,.T.)

```

```

205 oGetPercDesc:SetCSS( POSCSS (GetClassName(oGetPercDesc), CSS_GET_NORMAL ))
206
207 oLblSimbPerc := TSay():New(POSVERT_GET1 + 5, oPanelMVC:nWidth/3.5 + ((oGetPercDesc:nWidth/2) + 3 ) ,{||"%"},oPanGetProd,,,,,T,,,,100)
208 oLblSimbPerc:SetCSS( POSCSS (GetClassName(oLblSimbPerc), CSS_LABEL_NORMAL ))
209
210
211 //Codigo do Produto
212 oLblCodProd := TSay():New(POSVERT_LABEL2,POSHOR_1,{| |STR0002},oPanGetProd,,,,,T,,,,) //"Código do Produto"
213 oLblCodProd:SetCSS( POSCSS (GetClassName(oLblCodProd), CSS_LABEL_FOCAL ))
214
215 oCodProd := TGet():New(POSVERT_GET2,POSHOR_1,{|u| If(PCount())>0,cGetProd:=u,cGetProd}},oPanGetProd,165,ALTURAGET,"@!" ,,,,,,T,,,,,"cGetProd")
216 oCodProd:SetCSS( POSCSS (GetClassName(oCodProd), CSS_GET_FOCAL ))
217
218 bRegItem := { | | IIF( !Empty(AllTrim(cGetProd)) , ;
219 Eval( { | | cGetProd := IIF(nGetQtde > 0 .AND. nGetQtde <= 999,AllTrim(STR(nGetQtde))+AllTrim(aMVLJITOP[1]),"")+ ; //Quantidade
220 IIF(nGetVlrDesc > 0 , AllTrim(STR(nGetVlrDesc)) + AllTrim(aMVLJITOP[2]) , IIF(nGetPercDesc > 0 , AllTrim(STR(nGetPercDesc)) +
AllTrim(aMVLJITOP[3]) , "" ) ) + ; //Desconto
221 cGetProd , ;
222 cQtdDesc := cGetProd , ;
223 STIItemregister(cGetProd) , ;
224 cGetProd := Space(40) , ;
225 nGetQtde := 1 , ;
226 nGetVlrDesc := 0 , ;
227 nGetPercDesc := 0 , ;
228 oCodProd:SetFocus() } ) , .T. ) }
229
230 oCodProd:bValid := bRegItem
231
232 //"Botao registrar Item"
233 oButRegItem := TButton():New( POSVERT_GET2,oPanelMVC:nWidth/3.5, STR0025 + CRLF + STR0026 ,oPanGetProd,bRegItem,40,20,,,,T,,,,) //"Registrar" ### "Item"
234 oButRegItem:SetCSS( POSCSS (GetClassName(oButRegItem), CSS_BTN_NORMAL ))
235
236
237 bCamRegItem := { | | Eval( { | | cGetProd := IIF(nGetQtde > 0 .AND. nGetQtde <= 999,AllTrim(STR(nGetQtde))+AllTrim(aMVLJITOP[1]),"")+ ; //Quantidade
238 IIF(nGetVlrDesc > 0 , AllTrim(STR(nGetVlrDesc)) + AllTrim(aMVLJITOP[2]) , IIF(nGetPercDesc > 0 , AllTrim(STR(nGetPercDesc)) +
AllTrim(aMVLJITOP[3]) , "" ) ) + ; //Desconto
239 cGetProd , ;
240 cQtdDesc := cGetProd , ;
241 STICamRegItem(cGetProd) , ;
242 cGetProd := Space(40) , ;
243 nGetQtde := 1 , ;
244 nGetVlrDesc := 0 , ;
245 nGetPercDesc := 0 , ;
246 oCodProd:SetFocus() } ) }
247
248 //"Botao registrar Item com camera"
249 oButRegCamItem := TButton():New( POSVERT_GET2,oPanelMVC:nWidth/2.8,STR0025 + CRLF + STR0027,oPanGetProd,bCamRegItem,70,20,,,,T,,,,) // "Registrar" ### "com câmera"
250 oButRegCamItem:SetCSS( POSCSS (GetClassName(oButRegCamItem), CSS_BTN_BARCODE ))
251
252
253 oLblInfProd := TSay():New(POSVERT_GET3,POSHOR_1,{| |cDados},oPanGetProd,,,,,T,,,,100)
254 oLblInfProd:SetCSS( POSCSS (GetClassName(oLblInfProd), CSS_LABEL_NORMAL ))

```

```

255
256 //Botao finalizar vendas
257 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0003 ,oPanGetProd,{ || IIF(STIPePay(),STICallPayment(),.F.) }, ; //"Finalizar Venda"
258     LARGBTN,ALTURABTN,,,,.T.)
259
260 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
261
262 /* Identificação do Cartão relacionado na integração do SIGACRD */
263 If Len(aCartCRD) > 0
264     oLblCartCRD := TSay():New(POSVERT_LABEL4,POSHOR_1,{||IIf(!Empty(AllTrim(aCartCRD[1])),STR0033 + AllTrim(aCartCRD[1]),""},oPanGetProd,,,,,.T,,,,100) //"Cartão "
265     oLblCartCRD:SetCSS( POSCSS (GetClassName(oLblCartCRD), CSS_BREADCUMB ))
266 EndIf
267
268 //Identificacao do vendedor
269 If Len(aSalesMan) > 0
270     oLblSalesMan := TSay():New(POSVERT_LABEL7,POSHOR_1,{||STR0012 + " " + AllTrim(aSalesMan[1])},oPanGetProd,,,,,.T,,,,100)
271     oLblSalesMan:SetCSS( POSCSS (GetClassName(oLblSalesMan), CSS_LABEL_FOCAL ))
272 EndIf
273
274 //Identificacao do cliente
275 If Len(aCustomer) > 0
276     oLblCustomer := TSay():New(POSVERT_LABEL5,POSHOR_1,{||AllTrim(aCustomer[1])},oPanGetProd,,,,,.T,,,,100)
277     oLblCustomer:SetCSS( POSCSS (GetClassName(oLblCustomer), CSS_LABEL_FOCAL ))
278
279     oLblCpf := TSay():New(POSVERT_LABEL6,POSHOR_1,{||AllTrim(aCustomer[2])},oPanGetProd,,,,,.T,,,,100)
280     oLblCpf:SetCSS( POSCSS (GetClassName(oLblCpf), CSS_LABEL_FOCAL ))
281 EndIf
282
283 oCodProd:SetFocus()
284
285
286 Else
287
288     /*
289     Objetos
290     */
291     oLblCab := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanGetProd,,,,,.T,,,,) //"Registro de Item"
292     oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
293
294     oLblCodProd := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0002},oPanGetProd,,,,,.T,,,,) //"Código do Produto"
295     oLblCodProd:SetCSS( POSCSS (GetClassName(oLblCodProd), CSS_LABEL_FOCAL ))
296
297     oCodProd := TGet():New(POSVERT_GET1,POSHOR_1,{||u| If(PCount()>0,cGetProd:=u,cGetProd)},oPanGetProd,170,ALTURAGET,"@!",,,,,,.T,,,,,,,,,"cGetProd")
298     oCodProd:SetCSS( POSCSS (GetClassName(oCodProd), CSS_GET_FOCAL ))
299
300     oLblHelpGet := TSay():New(POSVERT_GET1+ALTURAGET+5,POSHOR_1,{||cAtalhosItem},oPanGetProd,,,,,.T,,,,) //"Quantidade (*) / Valor Desconto (-) OU Perc. Desconto (%) /
    Código do Produto"
301     oLblHelpGet:SetCSS( POSCSS (GetClassName(oLblHelpGet), CSS_LABEL_NORMAL ))
302
303     oLblInfProd := TSay():New(POSVERT_GET2,POSHOR_1,{||cDados},oPanGetProd,,,,,.T,,,,100)
304     oLblInfProd:SetCSS( POSCSS (GetClassName(oLblInfProd), CSS_LABEL_NORMAL ))
305

```

```

306
307 /*Caso exija CPF ou tenha paramterização para chamar tela de CPF no final da venda*/
308 If ExistFunc("STWBtFimVnd")
309     oButton := TButton():New(POSVERT_BTNOFICAL,POSHOR_BTNOFICAL,STR0003+CRLF+"(CTRL+F)",; //"Finalizar Venda"
310         oPanGetProd,{ || IIF(!AtivMotor,PshMtPdvP(),.T.) ,IIF(STIPePay(),STWBtFimVnd(),.F.) },;
311         LARGBTN,ALTURABTN,,,,.T.)
312     SetKey(6, { || IIF(!AtivMotor,PshMtPdvP(),.T.),IIF(STIPePay(),STWBtFimVnd(),.F.) }) // Tecla de atalho para 'Finalizar Venda'
313 Else
314     oButton := TButton():New(POSVERT_BTNOFICAL,POSHOR_BTNOFICAL,STR0003+CRLF+"(CTRL+F)",; //"Finalizar Venda"
315         oPanGetProd,{ || IIF(STIPePay(),IIF(lFinServ,STWFindServ(),STICallPayment()),.F.) },;
316         LARGBTN,ALTURABTN,,,,.T.)
317     SetKey(6, { || IIF(STIPePay(),IIF(lFinServ,STWFindServ(),STICallPayment()),.F.) }) // Tecla de atalho para 'Finalizar Venda'
318 EndIf
319 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
320
321 //Verifica se utiliza imagens por folder ou RPO
322 If Empty(cPathImg)
323     @ POSVERT_GET2,oPanelMVC:nWidth/3.01886 REPOSITORY oImg OF oPanGetProd NOBORDER SIZE 100,100 PIXEL
324 Else
325     oImg := TBitmap():New( POSVERT_GET2,oPanelMVC:nWidth/3.01886,100,100,,"",.T.,oPanGetProd,{||},,.F.,.F.,,.F.,.T.,.F.)
326     oImg:lStretch:= .T.
327 EndIf
328 oImg:lVisibleControl := .F.
329
330 /* Identificação do Cartão relacionado na integração do SIGACRD */
331 If Len(aCartCRD) > 0
332     oLblCartCRD := TSay():New(POSVERT_LABEL4,POSHOR_1,{||Iif(!Empty(AllTrim(aCartCRD[1])),STR0033 + AllTrim(aCartCRD[1]),""),oPanGetProd,,,,.T.,,100) //"Cartão"
333     oLblCartCRD:SetCSS( POSCSS (GetClassName(oLblCartCRD), CSS_BREADCUMB ))
334 EndIf
335
336 /* Identificacao do vendedor */
337 If Len(aSalesMan) > 0
338     oLblSalesMan := TSay():New(POSVERT_LABEL7,POSHOR_1,{||STR0012 + " " + AllTrim(aSalesMan[1])},oPanGetProd,,,,.T.,,100)
339     oLblSalesMan:SetCSS( POSCSS (GetClassName(oLblSalesMan), CSS_BREADCUMB ))
340
341     LjGrvLog( /*cNumControl*/, "Nome do Vendedor", aSalesMan[1] )
342 EndIf
343
344 /* Identificacao do cliente */
345 If Len(aCustomer) > 0
346
347     oLblCustomer := TSay():New(POSVERT_LABEL5,POSHOR_1,{||AllTrim(SubStr(aCustomer[1],1,Int(aRes[1]*0.03)))},oPanGetProd,,,,.T.,,100)
348     oLblCustomer:SetCSS( POSCSS (GetClassName(oLblCustomer), CSS_BREADCUMB ))
349
350     oLblCpf := TSay():New(POSVERT_LABEL6,POSHOR_1,{||AllTrim(aCustomer[2])},oPanGetProd,,,,.T.,,100)
351     oLblCpf:SetCSS( POSCSS (GetClassName(oLblCpf), CSS_BREADCUMB ))
352
353     LjGrvLog( /*cNumControl*/, "Nome do Cliente", aCustomer[1] )
354     LjGrvLog( /*cNumControl*/, "CPF/CNPJ do Cliente", aCustomer[2] )
355 EndIf
356
357 oCodProd:bGotFocus := { || STWReaderFocus(.T.) }

```

```

358
359     oCodProd:bLostFocus := { || STWReaderFocus(.F.),;
360                           IIF( !Empty(AllTrim(cGetProd)),;
361                               Eval( { || cQtdDesc := cGetProd      ,;
362                                     STIItemregister(cGetProd) ,;
363                                     cGetProd := Space(40)      ,;
364                                     oCodProd:SetFocus() } ) ,;
365                               (oButton := STIGetBtIR()          ,;
366                                 ) ) }
367
368     oCodProd:SetFocus()
369     If GetFocus() <> oCodProd:HWND
370         oCodProd:SetFocus()
371     Endif
372
373 EndIf
374
375 Return oPanGetProd
376
377
378 //-----
379 /*{Protheus.doc} STICamRegItem
380 Registro de Item com camera
381
382 @param  cGetProd - Texto do Get de produtos
383 @return Nil
384 @obs
385 @sample
386 */
387 //-----
388 Static Function STICamRegItem( cGetProd )
389
390 Local oMbl           := Nil // Objeto Tmobile para codigo de barras
391 Local cBarType       := ""  // Tipo de codigo de barras utilizado
392 Local aBarResult     := {}   // Resultado da consulta ao objeto de CodBar
393 Local lContinua     := .T.  // Controle de Loop
394
395 Default cGetProd := ""
396
397 oMbl:= TMobile():New()
398
399 While lContinua
400
401     aBarResult:= oMbl:BarCode(cBarType)
402
403     If aBarResult[2] = ""
404         lContinua := .F.
405     Else
406         STIItemregister(AllTrim(cGetProd) + AllTrim(aBarResult[1] ) )
407     EndIf
408
409 EndDo

```

```

410
411 Return Nil
412
413
414 //-----
415 /*{Protheus.doc} STIItemRegister
416 Registro de Item Interface
417
418 @param  lItemFiscal Indica se é item fiscal ou não
419 @return
420 @obs
421 @sample
422 */
423 //-----
424 Function STIItemRegister( cGetProd, lItemFiscal )
425
426 Local cCodItem      := ""
427 Local nQuant        := 0
428 Local cCliCode      := STDGPBasket("SL1","L1_CLIENTE")
429 Local cCliLoja      := STDGPBasket("SL1","L1_LOJA")
430 Local nPosQuant     := 0
431 Local nPosValDesc    := 0
432 Local nPosPercDesc   := 0
433 Local nDiscount     := 0
434 Local cTypeDesc     := ""
435 Local nItemLine     := STDPBLength("SL2") + 1
436 Local nItemLnAte    := 0
437 Local lConsultaProd := .F.
438 Local cTesPad       := ""
439 Local aRet          := {}
440 Local cLI1Num       := ""
441 Local lVPPProd      := SuperGetMv("MV_LJVPPRO",,.F.) //Permite vender Vale Presente e produtos na mesma venda
442 Local nLJ8099      := 0 //retorno do ponto de entrada LJ8099
443 Local lRet         := .T.
444 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
445 Local lDesligaRD    := .F. // Desabilita a Regra de Desconto do Varejo
446
447 Default cGetProd    := ""
448 Default lItemFiscal := .T.
449
450 LjGrvLog( "STIItemRegister", "Inicio do registro do item ", cGetProd)
451
452 If lRet .AND. FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
453     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
        STDGPBasket('SL1','L1_LOJA'))
454 EndIf
455
456 If lSTCodDig // PE para alterar a informacao digitada no GET da tela
457     aRet := ExecBlock("STCODDIG",.F., .F., {cGetProd} )
458     If ValType(aRet) == "A" .AND. Len(aRet) > 0

```

```

459         cGetProd := aRet[1]
460     EndIf
461 EndIf
462
463 cGetProd         := AllTrim(cGetProd)
464 nPosQuant        := AT(AllTrim(aMVLJITOP[1]),cGetProd)
465 nPosValDesc      := AT(AllTrim(aMVLJITOP[2]),cGetProd)
466 nPosPercDesc     := AT(AllTrim(aMVLJITOP[3]),cGetProd)
467
468 If lStbExistVP .And. !lVPPProd .And. STBExistVP() .And. !STDisVP(cGetProd)
469     STFMessage(ProcName(0),"STOP",STR0020) //"A venda do vale presente deve ser feita separada."
470     STFShowMessage(ProcName(0))
471     Return(.F.)
472 EndIf
473
474 //Se o sinal de desconto definido na segunda posicao do parametro MV_LJITOP estiver na primeira posicao significa que tentou digitar um valor zerado
475 //entao desconsidera este valor pois deve ser da quantidade
476 If nPosValDesc == 1
477     nPosValDesc := AT(AllTrim(aMVLJITOP[2]), SUBSTR(cGetProd, 2, Len(cGetProd) ))
478     If nPosValDesc > 0
479         ++nPosValDesc
480     EndIf
481 EndIf
482
483 lConsultaProd := SubStr(cGetProd,1, 1) == AllTrim(aMVLJITOP[4])
484
485 /*
486     0 bloco abaixo avalia a string digitada. Se na string cGetProd houver '*' sera setada quantidade
487     Caso haja '-' sera setado o valor do desconto e se houver '%' sera setado o percentual de desconto
488 */
489 If ( nPosQuant + nPosValDesc + nPosPercDesc ) > 0
490     /*/
491     Desconto primeiro, Quantidade depois
492     /*/
493     If nPosQuant > nPosValDesc .AND. nPosValDesc > 0
494         nQuant := STIComma(SubStr(cGetProd,nPosValDesc+1,nPosQuant-1))
495
496         nDiscount := STIComma(SubStr(cGetProd,1,nPosValDesc-1))
497         cCodItem  := SubStr(cGetProd,nPosQuant+1)
498         cTypeDesc := "V"
499     ElseIf nPosQuant > nPosPercDesc .AND. nPosPercDesc > 0
500         nQuant := STIComma(SubStr(cGetProd,nPosPercDesc+1,nPosQuant-1))
501
502         nDiscount := STIComma(SubStr(cGetProd,1,nPosPercDesc-1))
503         cCodItem  := SubStr(cGetProd,nPosQuant+1)
504         cTypeDesc := "P"
505
506     /*/
507     Quantidade primeiro, desconto depois
508     /*/
509     ElseIf nPosQuant > 0
510         nQuant := STIComma(SubStr(cGetProd,1,nPosQuant-1))

```

```

511
512     If nPosValDesc > 0
513         nDiscount := STIComma(SubStr(cGetProd,nPosQuant+1,nPosValDesc-1))
514         cCodItem  := SubStr(cGetProd,nPosValDesc+1)
515         cTypeDesc := "V"
516     ElseIf nPosPercDesc > 0
517         nDiscount := STIComma(SubStr(cGetProd,nPosQuant+1,nPosPercDesc-1))
518         cCodItem  := SubStr(cGetProd,nPosPercDesc+1)
519         cTypeDesc := "P"
520     Else
521         cCodItem := SubStr(cGetProd,nPosQuant+1)
522     EndIf
523
524     /*/
525     Apenas Descontos
526     /*/
527     Else
528         If nPosValDesc > 0
529             nDiscount := STIComma(SubStr(cGetProd,1,nPosValDesc-1))
530             cCodItem  := SubStr(cGetProd,nPosValDesc+1)
531             cTypeDesc := "V"
532         ElseIf nPosPercDesc > 0
533             nDiscount := STIComma(SubStr(cGetProd,1,nPosPercDesc-1))
534             cCodItem  := SubStr(cGetProd,nPosPercDesc+1)
535             cTypeDesc := "P"
536         Else
537             cCodItem := cGetProd // Apenas Produto
538         EndIf
539     EndIf
540
541     /*/
542     Passado apenas o código do produto
543     /*/
544     Else
545         cCodItem := cGetProd // Apenas Produto
546     EndIf
547
548     /*
549     Ponto de Entrada LJ8099 - Retorna o percentual de desconto (%)
550     Se o usuario informou qualquer tipo de desconto, o ponto de entrada nao sera executado
551     */
552     If !LJ8099 .AND. nDiscount = 0
553         nLJ8099 := Execblock("LJ8099", .F., .F.)
554         If ValType(nLJ8099) == "N" .AND. nLJ8099 > 0 .AND. nLJ8099 < 100
555             nDiscount := nLJ8099
556             cTypeDesc := "P"
557         Else
558             LjGrvLog(cL1Num, "Retorno invalido do PE LJ8099", nLJ8099)
559         EndIf
560     EndIf
561
562     If !Empty(cCodItem)

```

```

563     cL1Num := STDGPBasket('SL1','L1_NUM')
564
565     If !lConsultaProd // 0 item apenas sera registrado quando nao for uma consulta
566
567         /*/
568         Set Quantidade
569         /*/
570         If nQuant > 0 // Pode ter setado a quantidade antes por tecla de atalho
571
572             STBSetQuant( nQuant , 2)
573
574             LjGrvLog("L1_NUM: " + cL1Num, "Quantidade", nPosQuant) //Grava Log =====
575
576         ElseIf nPosQuant > 0
577             STFMessage(ProcName(),"STOP",STR0006 + " " + STR(nQuant) ) //"Quantidade Inválida."
578             STFSHOWMessage(ProcName())
579             Return(.F.)
580         EndIf
581
582         /*/
583         Set Desconto
584         /*/
585         If nDiscount > 0 .AND. (cTypeDesc $ "V|P")
586
587             STWIDBefore( nDiscount , cTypeDesc )
588
589         ElseIf nPosValDesc > 0 .OR. nPosPercDesc > 0
590             STFMessage(ProcName(),"STOP",STR0007 + " " + STR(nDiscount) ) //"Desconto Inválido."
591             STFSHOWMessage(ProcName())
592             Return(.F.)
593         EndIf
594
595         /*/
596         Registra Item
597         /*/
598
599         /* Busca TES Servico Financeiro */
600         If lFinServ .AND. STBIsFinService( cCodItem )
601             MG8->(dbSetOrder(2))
602
603             If MG8->(dbSeek(xFilial("MG8") + cCodItem))
604                 cTesPad := MG8->MG8_TESSB1
605             EndIf
606         EndIf
607
608         LjGrvLog( "L1_NUM: "+cL1Num, "Registra Item - Item" , nItemLine) //Gera LOG
609         LjGrvLog( "L1_NUM: "+cL1Num, "Registra Item - Produto" , cCodItem ) //Gera LOG
610         LjGrvLog( "L1_NUM: "+cL1Num, "Registra Item - Cliente" , cCliCode ) //Gera LOG
611         LjGrvLog( "L1_NUM: "+cL1Num, "Registra Item - Loja" , cCliLoja ) //Gera LOG
612
613         If nDiscount > 0 .AND. cTypeDesc $ "V|P"
614             aInfoItem := STWFindItem(cCodItem)

```

```

615         If aInfoItem[ITEM_TIPO] == "KT"
616             STFMessage(ProcName(),"STOP", STR0036)//"Não é permitido atribuir desconto sobre o produto do tipo KIT"
617             STFShowMessage(ProcName())
618             lRet:= .F.
619         Endif
620     Endif
621     //Com desconto aplicado verifica se tem que apresentar o tratamento do motivo de desconto
622     If lRet
623         If nDiscount > 0 .AND. cTypeDesc $ "V|P" .AND. SuperGetMV("MV_LJMTDIT",,.F.) ;
624             .AND. FindFunction("STBRFDAtiv") .AND. STDRsnDesc()
625             If !STBRFDAtiv(nItemLine,cCodItem,cTesPad,nTotItem)
626                 STFMessage(ProcName(),"STOP","Motivo de Desconto Inválido")
627                 Return(.F.)
628             EndIf
629         Else
630             STWSetRegIt(nItemLine,cCodItem,cTesPad,nTotItem,;
631                 lItemFiscal)
632         EndIf
633     Endif
634     Else
635         cCodItem := SubStr(AllTrim(cCodItem),2)
636     EndIf
637
638     /*
639     Verifica se informou apenas desconto
640     */
641     ElseIf nPosValDesc > 0 .OR. nPosPercDesc > 0
642
643         If nDiscount > 0 .OR. (cTypeDesc $ "V|P") // Pode ser que o usuário queira zerar o desconto, por isso o .OR.
644
645             /*
646             Workflow de desconto após registro de item via interface
647             */
648             STWIDAfter( nDiscount , cTypeDesc )
649
650         EndIf
651
652     EndIf
653
654     nItemLnAte := STDPBLength("SL2") // Busca quantidade de itens gravados da SL2
655     nItemLnAte := Iif(nItemLine > nItemLnAte,nItemLine,nItemLnAte)
656
657     STIGridCupRefresh(nItemLine,nItemLnAte) // Sincroniza a Cesta com a interface
658
659     LjGrvLog( "STIItemRegister", "Final do registro do item ", cGetProd)
660
661     Return (.T.)
662
663     //-----
664     /*{Protheus.doc} STIShowProdData
665     Mostra na tela as informacoes do item que sera cancelado
666

```

```

667 @param  nItem      - Numero do item
668 @return  Nil
669 @obs
670 @sample
671 /*/
672 //-----
673 Function STIShowProdData(nItem)
674 Local oPanelMVC      := STIGetPanel()           // Painel Principal
675 Local aLines         := FwSaveRows() // Array de linhas
676 Local oModelCesta    := STDGPBModel()          // Model da cesta
677 Local cProdData      := ""
678 Local cMoedaSimb      := SuperGetMV( "MV_SIMB" + Str(STBGetCurrency(),1) ) // Simbolo da moeda corrente
679 Local nPosVertical    := POSVERT_GET2          //Posicao vertical
680 Local lMobile         := STFGetCfg("lMobile", .F.) ///Verifica se e versao Mobile
681 Local cPathImg        := AllTrim(SuperGetMV("MV_LJIMAGE", .F., "")) // Caminho da pasta de imagens
682 Local cExtImg         := AllTrim(SuperGetMV("MV_LJIMGEX", .F., ".jpg")) // Extensão das imagens Opcional
683 Local aProdData       := {}
684 Local cTexto1         := ""
685 Local cTexto2         := ""
686 Local cMVLJB1COD      := AllTrim(SuperGetMV("MV_LJB1COD", "B1_COD")) //Campo que será utilizado para consulta da imagem do produto
687 Local cNomeImg        := ""
688
689 Default nItem         := 0
690
691 //Versao Mobile
692 If ValType(lMobile) == "L" .AND. lMobile
693     nPosVertical := POSVERT_GET3
694 EndIf
695
696 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
697
698 If nItem > 0
699     oModelCesta:GoLine(nItem)
700
701     cProdData := AllTrim(oModelCesta:GetValue("L2_DESCRI"))+CRLF+CRLF+CRLF;
702               +cMoedaSimb+AllTrim(Transform(oModelCesta:GetValue("L2_PRCTAB"),cPictValUnit))
703
704     //Só atualiza os valores do label se o panel existir
705     If ValType(oPanGetProd) <> "U"
706
707         aProdData := StrTokArr(cProdData,CHR(10))
708
709         If Len(aProdData) > 1
710             If Len(aProdData[1]) > 50
711                 cTexto1 := SubStr(aProdData[1],1,50)
712                 cTexto2 := SubStr(aProdData[1],51,50) + IIF ( Len(SubStr(aProdData[1],51,50)) < 50 ,"", "... " )
713
714                 cProdData := cTexto1 + CHR(10) + cTexto2 + CHR(10) + CHR(10) + aProdData[2]
715             EndIf
716         EndIf
717
718     If ValType(oLblProdDados) <> "U"

```

```

719         oLblProdDados:SetText(cProdData)
720     Else
721         oLblProdDados:= TSay():New(nPosVertical +10,POSHOR_1,{||cProdData},oPanGetProd,,,,,.T,,,,100)
722         oLblProdDados:SetCSS( POSCSS (GetClassName(oLblProdDados), CSS_LABEL_FOCAL ))
723     EndIf
724
725 EndIf
726
727 If !Empty(oImg)
728     If Empty(cPathImg)
729         cNomeImg := AllTrim(STDLoadImg(oModelCesta:GetValue("L2_PRODUTO")))
730         If !Empty(cNomeImg) .And. oImg:ExistBmp(cNomeImg)
731             ShowBitmap(oImg, cNomeImg, "SEMFOTO")
732             oImg:lVisibleControl := .T.
733         Else
734             oImg:lVisibleControl := .F.
735         EndIf
736     Else
737         cNomeImg := AllTrim(SB1->&(cMVLJB1COD))
738         If oImg:Load( Nil ,cPathImg + cNomeImg + cExtImg )
739             oImg:lVisibleControl := .T.
740         Else
741             oImg:SetEmpty()
742             oImg:lVisibleControl := .F.
743         EndIf
744     EndIf
745 EndIf
746 EndIf
747
748 FwRestRows(aLines)
749
750 Return
751
752 //-----
753 /*/{Protheus.doc} STICallPayment
754 Funcao que verifica se ha alguma NCC para o cliente selecionado.
755 Se houver, sera chamada a tela de selecao de NCCs ao invés da tela de pagamento.
756
757 @param
758 @return
759 @obs
760 @sample
761 */
762 //-----
763 Function STICallPayment()
764
765 Local aNCCs          := STDGetNCCs("1")
766 Local oTotal         := STFGetTot()
767 Local nTotalVend      := oTotal:GetValue("L1_VLRTOT")
768 Local cEstado         := SuperGetMV("MV_ESTADO")
769 Local lRet            := .T.
770 Local lVersao         := Val(GetVersao(.F.)) >= 12
771
772 // Recebe o Objeto totalizador
773 // Valor total da venda
774
775 // Retorno
776 // Codigo de barras para NCC soente a partir da versao 12

```

```

771 Local nVLBonif      := IIF(ExistFunc("STIGetBonif"), STIGetBonif(), 0)
772 Local lSTDslBrin    := AliasIndic("MGC") .AND. ExistFunc("STDSalvaBrindes")
773 Local lReadOnly     := ExistFunc("STIGetPayR0") .AND. STIGetPayR0() // Indica se os campos de pagamento estão como Somente Leitura (permissão Alterar Parcelas do
    caixa)
774 Local lEmitNFCe     := ExistFunc("LjEmitNFCe") .AND. LjEmitNFCe() // Indica a utilizacao da NFC-e
775 Local aValLim       := Iif( ExistFunc("Lj950UFVal"), Lj950UFVal(), {} )
776 Local nPos          := 0
777 Local cCliCode       := STDGPBasket("SL1","L1_CLIENTE")
778 Local cCliLoja       := STDGPBasket("SL1","L1_LOJA")
779 Local cCliPad        := SuperGetMV("MV_CLIPAD",,"") // Parametro que indica o CLIENTE PADRAO
780 Local cLojPad        := SuperGetMV("MV_LOJAPAD",,"") // Parametro que indica o LOJA PADRAO
781 Local lRegDesc       := SuperGetMV('MV_LJRGDES',,.F.) .AND. ExistFunc("STBFORMBF") .AND. ExistFunc("STIDSCBONF") .AND. GetAPOInfo("STFTOTALUPDATE.PRW")[4] >=
    Ctod("19/12/2019") //Regra de desconto
782 Local lContinua     := .T.
783 Local oRaas         := Nil
784 Local oLjCusFCore   := Nil
785 Local oFidelityC     := Nil
786 Local oModelCli     := Nil
787 Local oPhone        := Nil
788 Local nBonus        := 0
789 Local lLimiteSat     := lIsSat .AND. STDPBLength("SL2",.T.)>500 // Limite de itens permitido em uma venda com SAT
790 Local aItensFid      := {}
791 Local nX             := 0
792
793 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Chamada da Tela de Pagamento" ) //Gera LOG
794
795 nPos := Ascan(aValLim,{|x| x[1] == cEstado})
796
797 If lEmitNFCe // Só permite a venda acima de 10.000 se o cliente for identificado
798     If cCliCode + "/" + cCliLoja == cCliPad + "/" + cLojPad .AND. nTotalVend > 10000
799         STFMessage(ProcName(),"STOP",STR0032) // "Não é permitido vendas acima de 10.000 para cliente não identificado!"
800         STFShowMessage(ProcName())
801         lRet := .F.
802     ElseIf nPos > 0 .AND. Len(aValLim) > 0 .AND. nTotalVend > aValLim[nPos][2]
803         STFMessage(ProcName(), "STOP", STR0030 + TRANSFORM(aValLim[nPos][2], "@E 999,999") + STR0031) //"Não é permitido vendas acima de " ", cancele algum item para
            continuar!"
804         STFShowMessage(ProcName())
805         lRet := .F.
806     EndIf
807 ElseIf cEstado == "SP" .AND. nTotalVend > 10000
808     STFMessage(ProcName(),"STOP",STR0004) //"Não é permitido vendas acima de 10.000, cancele algum item para continuar!"
809     STFShowMessage(ProcName())
810     lRet := .F.
811 EndIf
812
813 if lRet .AND. lLimiteSat
814     STFMessage(ProcName(),"STOP",STR0034) //É permitido no máximo 500 registros no Cupom Fiscal do SAT!"
815     STFShowMessage(ProcName())
816     MsgAlert(STR0034 + STR0035 ) //É permitido no máximo 500 registros no Cupom Fiscal do SAT! Esse limite foi atingido! Esse limite foi atingido, favor rever e cancelar
        registros excedentes!
817     lRet := .F.
818 EndIf

```

```

819
820 If STBMR0brigat() .AND. !STBMRIRegistered() .AND. STDMRValid()
821     /*
822     Se o registro de midia for obrigatorio e nao foi informado,
823     chama tela para informar, caso tenha midias validas cadastradas
824     */
825     STFMessage(ProcName(),"STOP",STR0011) //"E obrigatorio informar a midia"
826     STFShowMessage(ProcName())
827     STIMultiMedia()
828     lRet := .F.
829 EndIf
830
831 If lRet
832
833     If nTotalVend > 0
834
835         /*
836         se os pagamentos estão Somente Leitura (caixa sem permissão para Alterar Parcelas E orçamento sem alteração),
837         entao redefinimos a variavel lPayImport, para que os pagamentos sejam obtidos do orçamento importado
838         */
839         If lReadOnly
840             STISetPayImp(.F.)
841         EndIf
842
843         //Salva os brindes na MGC
844         If lSTDslBrin
845             STDSalvaBrindes()
846         EndIf
847
848         If lVersao //A partir da versão 12
849             lContinua := .T.
850             If lRegDesc
851                 STBFormBf() //Calcula Bonificação
852                 IF oTotal:GetValue("L1_BONIF") > 0 .AND. oTotal:GetValue("L1_DESCONT") > 0
853                     STIExchangePanel( { || STIDscBonf() } )
854                     lContinua := .F.
855                 EndIf
856             EndIf
857
858             // Inicia FidelityCore apos todos os descontos e antes da NCC
859             // Proteger tambem
860
861             If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
862                 oRaas := STBGetRaas()
863
864                 If ValType(oRaas) == "0" .AND. oRaas:ServiceIsActive("TFC")
865
866                     oFidelityC := oRaas:GetFidelityCore()
867
868                     If ValType(oModelCli := STDGcliModel()) == "0"
869                         oPhone := LjPhone():New(oModelCli:GetValue("SA1MASTER","A1_DDD"),oModelCli:GetValue("SA1MASTER","A1_TEL"))
870                         oLjCusFCore := LjCustomerFidelityCore():New(oModelCli:GetValue("SA1MASTER","A1_NOME"),oModelCli:GetValue("SA1MASTER","A1_CGC"),;

```

```

871                                     oModelCli:GetValue("SA1MASTER","A1_EMAIL"),oPhone,oModelCli:GetValue("SA1MASTER","A1_DTNASC"),oModelCli:Ge
tValue("SA1MASTER","A1_DTNASC"))
872         EndIf
873
874         // -- Desativa Botões na tela
875         STIBtnDeActivate()
876
877         //Itens da Venda
878         For nX := 1 To STDPBLength("SL2")
879             aAdd(aItensFid, { nX ,;
880                 STDGPBasket("SL2", "L2_DESCRI" ,nX),;
881                 STDGPBasket("SL2", "L2_PRODUTO" ,nX),;
882                 STDGPBasket("SL2", "L2_QUANT" ,nX),;
883                 STDGPBasket("SL2", "L2_VLRITEM" ,nX) + STDGPBasket("SL2", "L2_VALDESC", nX),;
884                 STDGPBasket("SL2", "L2_VLRITEM" ,nX) })
885         Next nX
886
887         oFidelityC:Initiation(xfilial("SL1") + STDGPBasket("SL1","L1_NUM"),oTotal:GetValue("L1_VLRLIQ"),oLjCusFCore,aItensFid)
888         // -- Ativa Botões na tela
889         STIBtnActivate()
890
891         If oFidelityC:ChoseToUse() .And. (nBonus := oFidelityC:GetBonus()) > 0
892             STBTotDiscApply(nBonus,"V","FIDELITYCORE",.T.)
893             STDSPBasket("SL1","L1_DESCFID", nBonus)
894         EndIf
895
896     EndIf
897
898 EndIf
899
900
901 If lContinua
902     If Len(aNCCs) > 0
903         STIExchangePanel( { || STINCCSelection() } )
904     Else
905         STIExchangePanel( { || STIPayment() } )
906     EndIf
907 EndIf
908
909 Else
910     If Len(aNCCs) > 0
911         STIExchangePanel( { || STINCCSelection() } )
912     Else
913         If (nVLBonif > 0) .And. (nTotalVend > nVLBonif )
914             STIExchangePanel( { || STIPayment() } )
915             STIAddPay("BF", Nil, 1, Nil, Nil, nVLBonif) //Bonificacao
916         Else
917             STIExchangePanel( { || STIPayment() } )
918         Endif
919     EndIf
920 Endif
921

```

```

922     Else
923         STFMessage(ProcName(),"STOP",STR0005) //"Nenhum item foi registrado."
924         STFShowMessage(ProcName())
925     EndIf
926
927 EndIf
928
929 Return
930
931
932 //-----
933 /*{Protheus.doc} STIComma
934 Converte String com decimais em vírgula para numérico com decimais em ponto
935
936 @param      cValue      String a ser convertida
937 @return      nValue      Numerico com ponto
938 @obs
939 @sample
940 */
941 //-----
942 Function STIComma( cValue )
943
944 Local nValue      := 0      // Retorna valor
945
946 If !Empty(cValue)
947     If AT(", ",cValue) > 0
948         cValue := StrTran(cValue, ", ", ".")
949     EndIf
950     nValue := Val( cValue )
951 EndIf
952
953 Else
954     nValue := 0
955 EndIf
956
957 Return( nValue )
958
959
960
961
962
963
964
965 //-----
966 /*{Protheus.doc} STIGetProd
967 Retorna o que o usuario digitou no get de produto
968
969 @param
970 @return      cGetProd
971 @obs
972 @sample
973 */

```

```

974 //-----
975 Function STIGetProd()
976
977 Return cQtdDesc
978
979 //-----
980 /*{Protheus.doc} STISetPrd
981 Ajusta o campo de produto conforme repassado por parametro
982
983 @param
984 @return      cGetProd
985 @obs
986 @sample
987 */
988 //-----
989
990 Function STISetPrd(cQtdDAux)
991
992 Default cQtdDAux := ""
993
994 cQtdDesc := cQtdDAux
995
996 Return cQtdDesc
997
998
999 //-----
1000 /*{Protheus.doc} STIGetPanP
1001 Retorna o Get de digitacao do produto para poder ser manipulado , setfocus por exemplo
1002
1003 @param
1004 @return      oPanGetProd
1005 @obs
1006 @sample
1007 */
1008 //-----
1009 Function STIGetCodP()
1010
1011 Return oCodProd
1012
1013
1014 //-----
1015 /*{Protheus.doc} STIGetBtIR
1016 Retorna o botão responsavel por finalizar a venda.
1017
1018 @return      oButton
1019 @obs          Essa função foi criada para resolver o problema do foco, quando se apertava o enter quando o get do produto estava vazio
1020 @sample
1021 */
1022 //-----
1023 Function STIGetBtIR()
1024 Return oButton
1025

```

```

1026
1027 /*{Protheus.doc} STISetBtIR
1028 Seta um valor ao botão responsavel por finalizar a venda
1029
1030 @param      oBtn
1031 @return
1032 @obs
1033 @sample
1034 */
1035 //-----
1036 Function STISetBtIR(oBtn)
1037 Default oBtn := Nil
1038
1039 oButton := oBtn
1040
1041 Return Nil
1042
1043 //-----
1044 /*{Protheus.doc} STI0perIT
1045 Retorna o conteudo do parâmetro MV_LJITOP tratado.
1046
1047 @param
1048 @return      aMVLJITOP Array com o conteudo do parâmetro MV_LJITOP tratado.
1049 @obs
1050 @sample
1051 */
1052 //-----
1053 Static Function STI0perIT()
1054 Local cMVLJITOP := ""
1055 Local nI       := 0
1056
1057 /*
1058 Configuracao do Parametro MV_LJITOP:
1059   Posicao 1 = * Quantidade
1060   Posicao 2 = - Valor Desconto
1061   Posicao 3 = % Perc. Desconto
1062   Posicao 4 = ? Pesquisa produto
1063 */
1064 cMVLJITOP := SuperGetMv("MV_LJITOP", "*,-,%,?") // Parametro com os codigos dos caracteres de atalho de operacoes do item no registro do item
1065
1066 aMVLJITOP := StrToKarr(cMVLJITOP, ",")
1067
1068 For nI := 1 To Len(aMVLJITOP)
1069   If AllTrim(aMVLJITOP[nI]) == "0"
1070     aMVLJITOP[nI] := ""
1071   EndIf
1072 Next nI
1073
1074 If Len(aMVLJITOP) < 4
1075   MsgAlert("Conteúdo do parametro MV_LJITOP inválido. Por favor verifique!") //STR0029 "Conteúdo do parametro MV_LJITOP inválido. Por favor verifique!"
1076   aMVLJITOP := StrToKarr("*,-,%,?" , ",")
1077 EndIf

```

```

1078
1079 Return aMVLJITOP
1080
1081
1082 //-----
1083 /*{Protheus.doc} STIPePay
1084 Executa o PE StCallPay antes de passar para a proxima tela
1085
1086 @param
1087 @return      lRet - Retorna um valor logico
1088 @obs
1089 @sample
1090 */
1091 //-----
1092 Function STIPePay()
1093
1094 Local lStCallPay      := ExistBlock("StCallPay") //Verifica se existe o ponto de entrada StCallPay
1095 Local oModelCesta    := STDGPBModel() // Model da cesta
1096 Local lRet           := .T. //Variavel de retorno
1097
1098 If lStCallPay
1099     oModelCesta := oModelCesta:GetModel("SL2DETAIL")
1100     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Antes de executar o ponto de entrada StCallPay")
1101     lRet := ExecBlock("StCallPay",.F.,.F.,{STDGPBasket("SL1","L1_CLIENTE"),STDGPBasket("SL1","L1_LOJA"),oModelCesta})
1102     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Depois de executar o ponto de entrada StCallPay")
1103     If !(ValType(lRet) == "L")
1104         LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 ponto de entrada StCallPay nao retornou um valor logico e por conta disso nao abriu a tela de pagamento")
1105         lRet := .F.
1106     ElseIf !lRet
1107         LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 ponto de entrada StCallPay retornou um valor logico igual a .F. e por isto nao ira abrir a tela de pagamento")
1108     EndIf
1109 EndIf
1110
1111 Return lRet

```

Componentes/Interface/STIMemryFisc.prw

78 linhas após remoção de metadados de autoria/data

```

1 #Include "PROTHEUS.CH
2 #INCLUDE "STMEMRYFISC.CH"
3
4 //-----
5 /*{Protheus.doc} STIMemryFisc
6 Function interface da memória fiscal
7 @param      nTipoRel -Tipo do relatório
8 @param      cReducIni - Redução Inicial
9 @param      cReducFim - Redução Final
10 @param      nCheck1 - CheckBox
11 @param      nOpca - Opção selecionada
12 @param      dDataIni - Data Inicial
13 @param      dDataFinal - Data Final
14 @param      lCont - Continua execução
15 @return      lRet - Abertura com sucesso
16 @obs
17 @sample
18 */
19 //-----
20 Function STIMemryFisc(nTipoRel, cReducIni, cReducFim, nCheck, ;
21                      nOpca, dDataIni, dDataFim, lCont,;
22                      oFont)
23
24
25 Local oDlgMF // Instaciacao do objeto de montagem da tela
26 Local oReducIni // Objeto da reducao inicial
27 Local oReducFim // objeto da reducao final
28 Local oCheck1 // Objeto do check (impressora / disco )
29 Local oDataIni // Objeto da data inicial
30 Local oDataFim // Objeto da data final
31 Local oCheck1 // Objeto do check (impressora / disco )
32 Local oTipoRel // Objeto do tipo de relatorio memoria fiscal ou MFD
33 Local oKeyb // Objeto do teclado
34
35 DEFINE MSDIALOG oDlgMF FROM 39,85 TO 450,340 TITLE STR0003 PIXEL OF oMainWnd //"Leitura de Memória Fiscal"
36
37 DEFINE FONT oFont NAME "Ms Sans Serif" BOLD
38 // Definindo o Objeto Teclado
39
40 @ 7, 4 TO 60, 121 LABEL STR0004 OF oDlgMF PIXEL // "Objetivo do Programa"
41
42 // Este programa tem como objetivo
43 // efetuar a impressã,o da leitura de memçria fiscal da
44 // impressora fiscal
45 @ 19, 15 SAY STR0005 SIZE 100, 40 OF oDlgMF PIXEL FONT oFont //"Este programa tem como objetivo efetuar a impressão da leitura de memória fiscal da impressora fiscal."
46 @ 63,4 TO 90,121 LABEL STR0006 PIXEL OF oDlgMF // "Selecione Relatório"
47 @ 70,7 Radio oTipoRel Var nTipoRel Items STR0007,STR0008 3D Size 85,10 ON CHANGE ( STBMemVal(nTipoRel,@oReducIni,@oReducFim,@oCheck1) ) PIXEL OF oDlgMF // Memoria Fiscal, MFD
    (Memoria de Fita Detalhe)
48
49 @ 93,4 TO 118,121 LABEL STR0009 OF oDlgMF PIXEL // 'Leitura por data'
50 @ 103,10 SAY STR0010 SIZE 48, 7 OF oDlgMF PIXEL // Inicial:
51 @ 103,70 SAY STR0011 SIZE 48, 7 OF oDlgMF PIXEL // Final:

```

```

52
53
54 @ 103, 30 MSGET oDataIni Var dDataIni SIZE 32, 8 OF oDlGMF PIXEL
55 @ 103, 85 MSGET oDataFim Var dDataFim SIZE 32, 8 OF oDlGMF PIXEL
56
57
58 @ 122,4 TO 150, 121 LABEL STR0012 OF oDlGMF PIXEL // Leitura por: Reducao / C00 (MFD)
59
60 @ 132,10 SAY STR0010 SIZE 48, 7 OF oDlGMF PIXEL // Inicial:
61 @ 132,70 SAY STR0011 SIZE 48, 7 OF oDlGMF PIXEL // Final:
62
63 @ 132, 30 MSGET oReducIni Var cReducIni PICTURE '@E 9999' WHEN empty(dDataIni).AND.empty(dDataFim) VALID !Empty(cReducIni) .AND. Val(cReducIni)>0 SIZE 30, 8 OF oDlGMF PIXEL
64 @ 132, 85 MSGET oReducFim Var cReducFim PICTURE '@E 9999' WHEN empty(dDataIni).AND.empty(dDataFim) VALID !Empty(cReducFim) .AND. Val(cReducFim)>0 SIZE 30, 8 OF oDlGMF PIXEL
65
66 @ 153,4 TO 185,121 LABEL STR0013 OF oDlGMF PIXEL //'Saida da Leitura'
67 @ 160,10 RADIO oCheck1 VAR nCheck 3D SIZE 60,10 PROMPT STR0014, STR0015 OF oDlGMF PIXEL //'Impressora'/'Disco'
68
69 STFMessage("STWMemryFisc2", "YESN0", STR0016)//"Impressora OK?"
70
71
72 DEFINE SBUTTON FROM 190, 65 TYPE 1; // OK
73 ACTION (nOpca := 1,IF(STFShowMessage("STWMemryFisc2"),oDlGMF:End(),nOpca:=0)) ENABLE OF oDlGMF // "Impressora Ok?", "Atenção"
74 DEFINE SBUTTON FROM 190, 94 TYPE 2;
75 ACTION ( oDlGMF:End(), lCont := .F. ) ENABLE OF oDlGMF
76
77
78 ACTIVATE MSDIALOG oDlGMF CENTERED

```

Componentes/Interface/STIMultimediaRegister.PRW

192 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIMULTIMEDIAREGISTER.CH"
5
6 //-----
7 /*{Protheus.doc} STIMultiMedia
8 Interface Registro de Mídia
9
10 @param
11 @return Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STIMultiMedia()
17
18 Local aMidias          := STDMRGet()           // Armazena Mídias Cadastradas
19
20 If Len(aMidias) > 0
21     STIExchangePanel( { || STIPanMultiMedia() } )
22
23
24 Else
25
26     STFMessage("STIMultimedia","STOP",STR0001) //"Não existe nenhum registro de mídia cadastrado"
27     STFShowMessage("STIMultimedia","STOP",STR0001) //"Não existe nenhum registro de mídia cadastrado"
28
29 EndIf
30
31 Return Nil
32
33
34 //-----
35 /*{Protheus.doc} STIPanMultiMedia
36 Contexto Registro de Mídia
37
38 @param
39 @return oPanMultiMedia      Painel Registro de Mídia
40 @obs
41 @sample
42 */
43 //-----
44 Function STIPanMultiMedia()
45
46 Local oPanMultiMedia      := Nil              // Painel Registro de Mídia
47 Local oPanGetMedia        := Nil              // Painel Get Seleção de Mídia
48 Local oPanBtnSale         := Nil              // Painel de Botões da Venda
49 Local oMedia              := Nil              // ListBox para Seleção de Mídia
50 Local oButton             := Nil              // Botao "Adicionar Mídias"
51 Local aMidias             := {}               // Armazena Mídias para seleção
52 Local oPanelMVC           := STIGetDlg()      // Dialog da tela principal

```

```

53
54 /*/
55     Título - "Orçamento"
56 /*/
57 Local nPosLb1Alt      := oPanelMVC:nHeight/2 * 0.02 //Posicao: Altura
58 Local nPosLb1Hoz      := oPanelMVC:nWidth/2 * 0.039 //Posicao: Horizontal
59 /*/
60     Título - "Seleção de Mídia"
61 /*/
62 Local nPosLb2Alt      := (oPanelMVC:nHeight/2 * 0.02) * 3.7 //Posicao: Altura
63 Local nPosLb2Hoz      := oPanelMVC:nWidth/2 * 0.039 //Posicao: Horizontal
64 /*/
65     ListBox
66 /*/
67 Local nLisVerPerc      := 6                // Percentual de Posição Vertical do ListBox
68 Local nLisHorPerc      := 0.039           // Percentual de Posição Horizontal do ListBox
69 Local nLisHeiPerc      := 0.11            // Percentual de Altura do ListBox
70
71 /*/
72     Botão "Adicionar Mídia"
73 /*/
74 Local nBtnWidPerc      := oPanelMVC:nWidth/2 * 0.25           // Percentual de Largura do Botão "Adicionar Mídia"
75 Local nBtnHeiPerc      := oPanelMVC:nHeight/2 * 0.07          // Percentual de Altura do Botão "Adicionar Mídia"
76 Local nBtnVerPerc      := (oPanelMVC:nHeight/2 * 0.02) * 14   // Percentual da Posição Vertical do Botão "Adicionar Mídia"
77 Local nBtnHorPerc      := oPanelMVC:nWidth/2 * 0.40           // Percentual da Posição Horizontal do Botão "Adicionar Mídia"
78 /*/
79     Botão "Cancelar"
80 /*/
81 Local nBtnCwidPerc     := oPanelMVC:nWidth/2 * 0.25           // Percentual de Largura do Botão "Adicionar Mídia"
82 Local nBtnCheiPerc     := oPanelMVC:nHeight/2 * 0.07          // Percentual de Altura do Botão "Adicionar Mídia"
83 Local nBtnCverPerc     := (oPanelMVC:nHeight/2 * 0.02) * 14   // Percentual da Posição Vertical do Botão "Adicionar Mídia"
84 Local nBtnChorPerc     := oPanelMVC:nWidth/2 * 0.71           // Percentual da Posição Horizontal do Botão "Adicionar Mídia"
85 /*/
86     Panel
87 /*/
88 Local nPosPnlHoz := ((oPanelMVC:nHeight/2)/2 * 0.2) + 135 //Posicao: horizontal do painel
89
90
91 /*/
92     Panel Get de Mídia
93 /*/
94 oPanGetMedia := TPanel():New(nPosPnlHoz,00,"",Nil,,,,,(oPanelMVC:nWidth/2),(oPanelMVC:nHeight/2) * 0.60)
95 oPanGetMedia:SetCSS( POSCSS (GetClassName(oPanGetMedia), CSS_PANEL_CONTEXT ))
96
97 /*/
98     Busca Mídias Cadastradas para seleção
99 /*/
100 aMidias := STIGetMedias()
101
102 /*/
103     Objetos
104 /*/

```

```

105 oSay1 := TSay():New(nPosLb1Alt,nPosLb1Hoz,{||STR0002},oPanGetMedia,,,,,T,,,,) //"Vendas > Registro de Mídia"
106 oSay2 := TSay():New(nPosLb2Alt,nPosLb1Hoz,{||STR0003},oPanGetMedia,,,,,T,,,,) //"Seleção de Mídia"
107 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
108 oSay2:SetCSS( POSCSS (GetClassName(oSay2), CSS_LABEL_FOCAL ))
109
110 oMedia := TListBox():Create(oPanGetMedia, (oPanelMVC:nHeight/2 * 0.02) * nLisVerPerc, oPanelMVC:nWidth/2 * nLisHorPerc, Nil, aMidias, ( oPanelMVC:nWidth/2 -
    (2*nLisHorPerc*oPanelMVC:nWidth/2) ),oPanelMVC:nHeight/2 * nLisHeiPerc,,,,,T,,{ || oButton:SetFocus() })
111 oMedia:SetCSS( POSCSS (GetClassName(oMedia), CSS_LISTBOX ))
112
113 oButton := TButton():New( nBtnVerPerc,nBtnHorPerc,STR0004,oPanGetMedia,{ || STISetMedia( oMedia ) }, ; //"Adicionar Mídia"
114 nBtnWidPerc,nBtnHeiPerc,,,,T.)
115 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
116
117
118 oBtnCanc := TButton():New( nBtnCVerPerc,nBtnCHorPerc,STR0005,oPanGetMedia,{ || STIExchangePanel( { || STIPanItemRegister() } ) }, ; //"Cancelar"
119 nBtnCWidPerc,nBtnCHeiPerc,,,,T.)
120 oBtnCanc:SetCSS( POSCSS (GetClassName(oBtnCanc), CSS_BTN_FOCAL ))
121
122 oMedia:GoTop()
123 oMedia:SetFocus()
124
125 Return oPanGetMedia
126
127
128 //-----
129 /*{Protheus.doc} STIGetMedias
130 Busca Midias para seleção na interface
131
132 @param
133 @return aMidias          Retorna Midias para seleção
134 @obs
135 @sample
136 */
137 //-----
138 Function STIGetMedias()
139
140 Local aMidias          := {}          // Retorna Midias para seleção
141 Local aDataMidias      := STDMRGet()  // Armazena Midias Cadastradas
142 Local nI               := 0          // Contador
143
144 /*
145     Prepara as mídias cadastradas para seleção
146 */
147 For nI := 1 To Len(aDataMidias)
148
149     AADD( aMidias , cValToChar(nI) + " - " + aDataMidias[nI][2] )
150
151 Next nI
152
153 Return aMidias
154
155

```

```

156 //-----
157 /*{Protheus.doc} STISetMedia
158 Adicionar Midia
159
160 @param   oMedia           ListBox Registro de Midia
161 @return
162 @obs
163 @sample
164 */
165 //-----
166 Function STISetMedia( oMedia )
167
168 Local cCodMidia      := ""           // Código de Midia a ser adicionado
169 Local aDataMidias    := STDMRGet()   // Armazena Midias Cadastradas
170
171 Default oMedia      := Nil
172
173 If oMedia <> Nil
174
175     If oMedia:GetPos() > 0
176
177         cCodMidia := aDataMidias[oMedia:GetPos()][1]
178
179         STBMRSetMedia( cCodMidia )
180
181         STIExchangePanel( { || STIPanItemRegister() } )
182
183     Else
184
185         STFMessage("STIMultiMedia","STOP",STR0006) //"Mídia não adicionada"
186         STFShowMessage("STIMultiMedia")
187
188     EndIf
189
190 EndIf
191
192 Return

```

Componentes/Interface/STINCCSelection.prw

632 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "FWMBROWSE.CH"
2 #INCLUDE "FWMVCDEF.CH"
3 #INCLUDE "PROTHEUS.CH"
4 #INCLUDE "APWIZARD.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE 'FWBROWSE.CH'
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STINCCSELECTION.CH"
9 #INCLUDE "STPOS.CH"
10
11 #DEFINE NCCSELECTED      1 // Campo logico que indica se a NCC foi selecionada
12 #DEFINE SE1NUM          3 // Posicao do campo El_NUM do Array aNCCsCli
13 #DEFINE SE1RECNO        5 // Posicao do Recno do registro dentro do Array aNCCsCli
14 #DEFINE SE1SALDO        6 // Posicao do campo El_SALDO do Array aNCCsCli
15 #DEFINE SE1PREFIXO      9 // Posicao do campo El_PREFIXO do Array aNCCsCli
16 #DEFINE SE1PARCELA     10 // Posicao do campo El_PARCELA do Array aNCCsCli
17 #DEFINE SE1TIPO        11 // Posicao do campo El_TIPO do Array aNCCsCli
18
19 Static nNccSelOld        := 0 // Valor das NCC's Selecionadas no orçamento
20 Static aNccsSelCliOld    := {} // Backup do Array aNCCsCli
21 Static aNCCsCli          := {} // Array com todas as NCCs do cliente
22 Static oLblSelTotal      := Nil // Objeto do label que informa o valor total das NCCs selecionadas.
23 Static oLblTotalVend     := Nil // Label que informa o restante para pagamento
24 Static cTextSelection    := STR0010 // "Item Selecionado - "
25 Static lZeraPayImport    := .F. // Indica se considera as formas de pagamentos importados
26 //-----
27 /*{Protheus.doc} STINCCSelection
28 Responsavel por montar a interface de selecao de NCCs
29
30 @return oMainPanel - Objeto contendo o painel de selecao de NCCs
31
32 */
33 //-----
34
35 Function STINCCSelection()
36 Local oPanelMVC          := STIGetPanel() // Objeto do Panel onde a interface de selecao de clientes sera criada
37 Local oMainPanel         := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
38
39 /*
40 Declaracao dos Objetos
41 */
42 Local oLblCab             := Nil // Objeto do label "Seleção de Notas de Crédito ao Cliente"
43 Local oLblList            := Nil // Objeto do label "Digite aqui para filtrar"
44 Local oListPanel          := Nil // Objeto do panel onde será criado o Browse com as NCCs do cliente.
45 Local oListNCCs           := Nil // Objeto do tipo TListBox, que exibe as NCCs do cliente.
46 Local oLblCliTotal        := Nil // Objeto do label que informa o valor total das NCCs do cliente.
47 Local oTotal              := STFGetTot() // Recebe o Objeto totalizador
48
49 /*
50 Variaveis de posicionamento de objetos
51 */
52 Local nPosVerPanListBox := oMainPanel:nHeight/2 * 0.0666// Posicao vertical do objeto TPanel oListPanel

```

```

53 Local nPosVerTListBox := oMainPanel:nHeight/2 * 0.05 // Posicao vertical do Say oLblList
54 Local nListPosHor := oMainPanel:nWidth/2 * 0.05 // Posicao Horizontal do objeto TListBox oListNCCs
55
56
57 /*/
58 Botão "Selecionar Cliente"
59 /*/
60 Local oButton := Nil // Botao "Selecionar Cliente"
61 Local aListNCCs := {} // Itens do objeto oListNCCs
62 Local nX := 0 // Contador do FOR
63 Local oListFont := TFont():New("Courier New") // Fonte utilizada no listbox
64
65 Local nSaldoNCCs := 0 // Total das NCCs do cliente
66 Local nTotalSel := 0 // Total das NCCs selecionadas
67 Local oTotFont := TFont():New( "Arial" ,Nil,-13,Nil,.T. ) // Objeto da fonte dos totalizadores
68
69 Local cText := "" //Texto selecionado
70 Local nValRest := 0 //REstante
71 Local oSelVlr := Nil //Objeto checkbox de seleção automatica de valores da NCC
72 Local LSELVLR := .F. //Seleciona automaticamente valores de NCC
73 Local lPetPOpNCC := ExistBlock("STTP0PNCC") //Ponto de Entrada para permitir ou não alterar NCCs definidos pelo Orçamento na Importação pelo Totvs PDV
74 Local cTpOpNcc := ""
75
76 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Monta a interface de seleção de NCCs" ) //Gera LOG
77
78 aNCCsCli := STDGetNCCs("1")
79 aNccsSelCliOld := Aclone(aNCCsCli)
80
81 //Limpa a variavel de saldo selecionados
82 STDSetNCCs("2")
83
84 If SuperGetMv("MV_LJNCCOR", Nil, .F.)
85 //Carrega a NCC que foi definida no orçamento
86 STDSetNCCs("1",aNccsSelCliOld)
87 aNCCsCli := STDGetNCCs("1")
88 EndIf
89
90 For nX := 1 To Len(aNCCsCli)
91 cText := STR0001+aNCCsCli[nX,SE1PREFIX0]+STR0002+aNCCsCli[nX,SE1NUM]+STR0003+aNCCsCli[nX,SE1PARCELA]+STR0004+AllTrim(Str(aNCCsCli[nX,SE1SALD0]))+STR0016+aNCCsCli[nX,SE1TI
P0]
92 If aNCCsCli[nX,NCCSELECTED]
93 cText := cTextSelection + cText
94 STDSetNCCs("2",aNCCsCli[nX,SE1SALD0])
95 EndIf
96 aAdd(aListNCCs,cText)
97
98 nSaldoNCCs += aNCCsCli[nX,SE1SALD0]
99 Next nX
100
101 //Retorna o Total Selecionado
102 nTotalSel := STDGetNCCs("2")
103 nNccSelOld := nTotalSel

```

```

104
105 //Calcula o Restante a receber
106 nValRest := oTotal:GetValue("L1_VLRTOT") - nTotalSel
107
108 //Tratamento só para não aparecer valor negativo no valor restante para pagamento.
109 If nValRest < 0
110     nValRest := 0
111 EndIf
112
113 /*/
114     Cabecalho da tela - "Seleção de Notas de Crédito ao Cliente"
115 /*/
116 oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0005},oMainPanel,,,,,.T,,,,) //"Seleção de Notas de Crédito ao Cliente"
117 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
118
119 /* Instancia do Painel do ListBox de selecao de NCCs */
120 oListPanel := TPanel():New(nPosVerPanListBox,000,"",oMainPanel,,,,,oMainPanel:nWidth/2.15,oMainPanel:nHeight/5)
121
122 /* Instancia do Label do ListBox de selecao de NCCs */
123 oLblList:= TSay():New(000,nListPosHor,{||STR0006},oListPanel,,oTotFont,,,,.T,,,,)
124     //"Selecione os titulos desejados e clique em 'Avançar'"
125 oLblList:SetCSS( POSCSS (GetClassName(oLblList), CSS_LABEL_FOCAL ))
126
127 /* Instancia do ListBox das NCCs do cliente */
128 oListNCCs := TListBox():Create(oListPanel, nPosVerTListBox, nListPosHor, Nil, aListNCCs, oListPanel:nWidth/2.2,
129     oListPanel:nHeight/2.5,,,,.T,,,{||STISelNCC(oListNCCs)},oListFont)
130 oListNCCs:SetCSS( POSCSS (GetClassName(oListNCCs), CSS_LISTBOX ))
131
132 /* Instancia do Label que informa o valor total das NCCs que o cliente possui */
133 oLblCliTotal := TSay():New(oMainPanel:nHeight/4.5,nListPosHor,{||STR0007 +AllTrim(Str(nSaldoNCCs))},oMainPanel,,oTotFont,,,,.T,,,,) //"Saldo Total: "
134 oLblCliTotal:SetCSS( POSCSS (GetClassName(oLblCliTotal), CSS_LABEL_FOCAL ))
135
136 /* Instancia do Label que informa o valor total das NCCs selecionadas */
137 oLblSelTotal := TSay():New(oMainPanel:nHeight/4.5,nListPosHor*7.5,{||STR0008+AllTrim(Str(nTotalSel))},oMainPanel,,oTotFont,,,,.T,,,,) //"Total Selecionado: "
138 oLblSelTotal:SetCSS( POSCSS (GetClassName(oLblSelTotal), CSS_LABEL_FOCAL ))
139
140 oSelVlr := TCheckBox():New(oMainPanel:nHeight/4, nListPosHor,STR0015,{|| lSelVlr },oMainPanel,100,,{|| (lSelVlr := STI7SelNcc(oListNCCs, lSelVlr),oSelVlr:Refresh())})
141     //"Seleção automática de valores"
142
143 /* Instancia do Label que informa o restante a ser pago na venda */
144 oLblTotalVend := TSay():New(oMainPanel:nHeight/3.5, nListPosHor, {||STR0011 + AllTrim(Str(nValRest,12,2)) +STR0012}, oMainPanel,,oTotFont,,,,.T,,,,,10)
145 oLblTotalVend:SetCSS( POSCSS (GetClassName(oLblTotalVend), CSS_LABEL_FOCAL ))
146
147 /* Instancia do botao "Avançar" */
148 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0013,oMainPanel,{ || STISNCCPayment()}, ; //"Avançar"
149     LARGBTN,ALTURABTN,,,,.T.)
150
151 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
152
153 If SuperGetMV("MV_LJASNNC",.F.,"1") == "2" //Seleciona automaticamente os valores caso esteja 2
154     lSelVlr := STI7SelNcc(oListNCCs, lSelVlr)
155 EndIf

```

```

153
154 If lPeTP0pNCC .AND. STBIsImpOrc()
155     cTpOpNcc := ExecBlock("STTPOPONCC",.F.,.F.)
156     oListNCCs:lReadOnly:= (cTpOpNcc=="1")
157     oSelVlr:lReadOnly:= (cTpOpNcc=="1")
158 Endif
159
160 Return oMainPanel
161
162 //-------------------------------------
163 /*{Protheus.doc} STISelectNCC
164 Responsavel por realizar a marcacao do item selecionado
165
166 @aparam oListNCCs - Objeto do tipo TListBox responsavel por apresentar as NCCs do cliente.
167 */
168 //-------------------------------------
169
170 Static Function STISelectNCC(oListNCCs)
171 Local nLine      := oListNCCs:nAt
172 Local cTextNCC   := oListNCCs:GetSelText()
173 Local oTotal     := STFGetTot()                // Recebe o Objeto totalizador
174 Local nTotalVend := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
175 Local nTotalNCCs := STDGetNCCs("2")            // Valor total das NCCs
176 Local nValRest   := 0                        // Valor restante para pagamento
177 Local lImport    := !Empty(STDGPBasket("SL1" , "L1_NUMORIG")) // Variavel de controle que identifica se é importação de orçamento
178
179 DEFAULT oListNCCs := Nil
180
181 If !lImport .OR. (lImport .AND. STFProfile(41)[1])
182     If ValType(oListNCCs) == "0"
183
184         If Len(cTextNCC) > Len(cTextSelection) .AND. SubStr(cTextNCC,1,Len(cTextSelection)) == cTextSelection // Caso o item esteja selecionado, desmarca o item.
185             STDSetNCCs("2",-aNCCsCli[nLine,SE1SALDO]) //0 saldo da NCC posicionada é deduzido do total selecionado
186             oListNCCs:Modify(SubStr(cTextNCC,Len(cTextSelection)+1),nLine) // 0 texto de selecao é removido
187             aNCCsCli[nLine,NCCSELECTED] := .F. // A NCC é desmarcada como selecionada
188             LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC removida", cTextSelection+oListNCCs:GetSelText() ) //Grava Log
189             =====
190             Else // Caso contrario, marca o item
191                 /* Verifico se o total de NCCs selecionadas nao extrapolou o valor total da venda */
192                 If nTotalVend > nTotalNCCs
193                     STDSetNCCs("2",aNCCsCli[nLine,SE1SALDO]) // Atualiza o saldo total de NCCs selecionadas
194                     oListNCCs:Modify(cTextSelection+oListNCCs:GetSelText(),nLine) // Modifica o texto para informar que a NCC foi selecionada
195                     aNCCsCli[nLine,NCCSELECTED] := .T. // A NCC é marcada como selecionada
196                     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC selecionada", cTextSelection+oListNCCs:GetSelText() ) //Grava Log
197                     =====
198                 Else
199                     STFMessage(ProcName(),"STOP",STR0014) //"Valor total já atingido."
200                     STFShowMessage(ProcName())
201                     STFCleanMessage(ProcName())
202             EndIf
203         EndIf
204     EndIf
205 EndIf

```

```

203      nTotalNCCs := STDGetNCCs("2")
204      If (nTotalVend-nTotalNCCs) > 0
205          nValRest := nTotalVend-nTotalNCCs
206      Else
207          nValRest := 0
208      EndIf
209
210      oLblSelTotal:SetText(STR0008+AllTrim(Str(STDGetNCCs("2")))) //Total Selecionado: "
211      oLblTotalVend:SetText(STR0011 + AllTrim(Str(nValRest,12,2)) + STR0012)
212
213      LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC - Total selecionado", STR0008+AllTrim(Str(STDGetNCCs("2")))) ) //Grava Log
214      LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC - Total restante", STR0011 + AllTrim(Str(nValRest,12,2)) + STR0012 ) //Grava Log
215
216      EndIf
217 Else
218     STFMessage(ProcName(),"STOP",STR0017) //"Usuario sem permissão para alterar a negociação do orçamento."
219     STFShowMessage(ProcName())
220 EndIf
221 Return
222
223
224 //-----
225 /* {Protheus.doc} STISNCCPayment
226 Funcao que seta um array estatico das NCCs com as informacoes que serao passadas para o componente de pagamento.
227 Tambem chama a tela de pagamento.
228
229 @return
230 @obs
231 @sample
232 */
233 //-----
234 Function STISNCCPayment()
235 Local oModelPayment := ModelPayme() // Modelo de dados do Pagamento
236 Local oMdlParc := oModelPayment:GetModel("PARCELAS")
237 Local oMdlPaym := oModelPayment:GetModel("APAYMENTS")
238 Local oTotal := STFGetTot() // Recebe o Objeto totalizador
239 Local nTotalVend := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
240 Local nTotalNCCs := STDGetNCCs("2") // Valor total das NCCs
241 Local nMVLJCPNCC := SuperGetMv("MV_LJCPNCC ",,1) // Tratamento para compensacao de NCC 1 - INCLUSAO DE NOVO TITULO|2 - ALTERACAO DO SALDO|3 - BAIXA TOTAL DA
NCC|4 - SALDO DA NCC COM TROCO
242 Local lImport := !Empty(STDGPBasket("SL1" , "L1_NUMORIG")) // Variavel de controle que identifica se é importação de orçamento
243
244 oMdlPaym:DeActivate()
245 oMdlPaym:Activate()
246
247 oMdlParc:DeActivate()
248 oMdlParc:Activate()
249
250 STDSetNCCs("1",aNCCsCli)
251

```

```

252 LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 parametro MV_LJCPNCC esta configurado como ", nMVLJCPNCC )
253 If nMVLJCPNCC == 4 //Somente quando esta config. 4 que os parametros de troco são necessário saber
254     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 parametro MV_LJTROCO esta configurado como ", SuperGetMv("MV_LJTROCO",,.F.) )
255     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 parametro MV_LJTRDIN esta configurado como ", SuperGetMv("MV_LJTRDIN",,0) )
256 EndIf
257
258 If nTotalNCCs == 0
259     //nTotalNCCs zerado indica que nenhuma NCC foi selecionada ou caso tenha foi desmarcada.
260     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Nenhuma NCC foi selecionada",nTotalNCCs)
261
262     If lImport .AND. nNccSelOld <> nTotalNCCs //Caso seja importação e as NCC's foram alteradas
263
264         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Existe uma diferença entre as NCC's do orçamento e as NCC's da venda")
265
266         STFMessage("STISNCCPay","YESNO",STR0018)//"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja continuar"
267
268         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja
continuar")
269
270         If STFShowMessage("STISNCCPay")
271             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Sim, desejo seguir com a alteração.")
272
273             //Indico que devera limpar as fomar de pagamento com a variavel lZeraPayImport = True
274             lZeraPayImport := .T.
275
276             //Carrego os paineis para os demais pagamentos
277             STIExchangePanel({|| STIPayment() })
278         Else
279             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Não, desejo desfazer a alteração.")
280             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Restaurando NCC's do orçamento.")
281             If nNccSelOld > nTotalVend .And. nMVLJCPNCC <> 4 // Caso o valor total das NCCs selecionadas seja maior que o valor total da
venda.
282                 //Desfaço as alterações feitas caso ele negue
283                 LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"0 valor da NCC salva no orçamento é maior que o VALOR TOTAL da venda ")
284
285                 //Indico que não devera limpar as fomar de pagamento com a variavel lZeraPayImport = False
286                 lZeraPayImport := .F.
287
288                 //Carrego os paineis para os demais pagamentos
289                 STIExchangePanel({|| STIPayment() })
290
291                 //Adiciono o pagamento da NCC no Grid de pagamentos
292                 If nNccSelOld > 0
293                     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC restaurada no valor de: " + cValToChar(nTotalVend))
294                     STIAddPay("CR", Nil, 1, Nil, Nil, nTotalVend)
295                 EndIf
296
297                 //Restauo o valor das NCC's e o Array aNCCsCli com as NCC's selecionadas
298                 STDSetNCCs("2", nTotalVend)
299                 LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Valor de pagamento com NCC's restaurado: " + cValToChar(nTotalVend))
300
301                 aNCCsCli := aNccsSelCliOld

```

```

302         STDSetNCCs("1",aNCCsCli)
303         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Array aNCCsCli resturado",aNCCsCli)
304     Else
305         //Desfaço as alterações feitas caso ele negue
306         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC's do orçamento é menor ou igual ao TOTAL DA VENDA")
307
308         //Indico que não devera limpar as fomar de pagamento com a variavel lZeraPayImport = False
309         lZeraPayImport := .F.
310
311         //Carrego os paineis para os demais pagamentos
312         STIExchangePanel({|| STIPayment() })
313
314         //Adiciono o pagamento da NCC no Grid de pagamentos
315         If nNccSelOld > 0
316             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC restaurada no valor de: " + cValToChar(nNccSelOld))
317             STIAddPay("CR", Nil, 1, Nil, Nil, nNccSelOld)
318         EndIf
319
320         //Restauro o valor das NCC's e o Array aNCCsCli com as NCC's selecionadas
321         STDSetNCCs("2",nNccSelOld)
322         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Valor de pagamento com NCC's restaurado: " + cValToChar(nNccSelOld))
323
324         aNCCsCli := aNccsSelCliOld
325         STDSetNCCs("1",aNCCsCli)
326         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Array aNCCsCli resturado",aNCCsCli)
327     EndIf
328 EndIf
329 Else //Caso seja auto serviço e não selecionou NCC
330     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC utilizada em venda Auto Serviço")
331     STIExchangePanel({|| STIPayment() })
332 EndIf
333 ElseIf nTotalNCCs > nTotalVend .And. nMVLJCPNCC <> 4 // Caso o valor total das NCCs selecionadas seja maior que o valor total da venda, sera setado o valor total da venda.
334     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Uma ou mais NCC's foram selecionadas, e o valor do credito é maior que o TOTAL DA VENDA",nTotalNCCs)
335
336     If lImport .AND. nNccSelOld <> nTotalNCCs //Caso seja importação e as NCC"s foram alteradas
337
338         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Existe uma diferença entre as NCC's do orçamento VS a NCC's da venda")
339
340         STFMessage("STISNCCPay","YESNO",STR0018) //"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja continuar"
341
342         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja continuar")
343
344         If STFShowMessage("STISNCCPay")
345             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Sim, desejo seguir com a alteração.")
346
347             //Indico que devera limpar as fomar de pagamento com a variavel lZeraPayImport = True
348             lZeraPayImport := .T.
349
350             //Carrego os paineis para os demais pagamentos
351             STIExchangePanel({|| STIPayment() })
352

```

```

353      //Adiciono o pagamento da NCC no Grid de pagamentos
354      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Valor do pagamento com NCC's alteradas de: " + cValToChar(nNccSelOld) + " Para: " + cValToChar(nTotalVend))
355      STIAddPay("CR", Nil, 1, Nil, Nil, nTotalVend)
356
357  Else //Desfaço as alterações feitas caso ele negue
358      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Não, desejo desfazer a alteração.")
359      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Restaurando NCC's do orçamento.")
360
361      //Verifico se o valor da Ncc salvo no orçamento é maior que o total da venda
362      If nNccSelOld > nTotalVend
363          LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"O valor da NCC salva no orçamento é maior que o VALOR TOTAL da venda ")
364
365          //Indico que não deverei limpar as fomar de pagamento com a variavel lZeraPayImport = False
366          lZeraPayImport := .F.
367
368          //Carrego os paineis para os demais pagamentos
369          STIExchangePanel({|| STIPayment() })
370
371      //Adiciono o pagamento da NCC no Grid de pagamentos
372      If nNccSelOld > 0
373          LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC restaurada no valor de: " + nTotalVend)
374          STIAddPay("CR", Nil, 1, Nil, Nil, nTotalVend)
375      EndIf
376
377      //Restauo o valor das NCC's e o Array aNCCsCli com as NCC's selecionadas
378      STDSetNCCs("2",nTotalVend)
379      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"O valor de pagamento com NCC's restaurado: " + nTotalVend)
380
381      aNCCsCli := aNccsSelCliOld
382      STDSetNCCs("1",aNCCsCli)
383      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Array aNCCsCli resturado",aNCCsCli)
384  Else
385      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"O valor da NCC salva no orçamento é menor que o VALOR TOTAL da venda")
386
387      //Indico que não deverei limpar as fomar de pagamento com a variavel lZeraPayImport = False
388      lZeraPayImport := .F.
389
390      //Carrego os paineis para os demais pagamentos
391      STIExchangePanel({|| STIPayment() })
392
393      //Adiciono o pagamento da NCC no Grid de pagamentos
394      If nNccSelOld > 0
395          LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC restaurada no valor de: " + cValToChar(nNccSelOld))
396          STIAddPay("CR", Nil, 1, Nil, Nil, nNccSelOld)
397      EndIf
398
399      //Restauo o valor das NCC's e o Array aNCCsCli com as NCC's selecionadas
400      STDSetNCCs("2",nNccSelOld)
401      LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"O valor de pagamento com NCC's restaurado: " + cValToChar(nTotalVend))
402
403      aNCCsCli := aNccsSelCliOld
404      STDSetNCCs("1",aNCCsCli)

```

```

405         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Array aNCCsCli resturado",aNCCsCli)
406     EndIf
407 EndIf
408 ElseIf !lImport .OR. (lImport .AND. nNccSelOld == nTotalNCCs ) //NCC utilizada em venda Auto Serviço ou não teve alteração nas NCC's.
409     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC utilizada em venda Auto Serviço ou não teve alteração nas NCC's.")
410
411     //Carrego os paineis para os demais pagamentos
412     STIExchangePanel({|| STIPayment() })
413
414     //Adiciono o pagamento da NCC no Grid de pagamentos
415     STIAddPay("CR", Nil, 1, Nil, Nil, nTotalVend)
416 EndIf
417 Else
418     LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC's do orçamento é menor ou igual ao TOTAL DA VENDA")
419
420     If lImport .AND. nNccSelOld <> nTotalNCCs //Caso seja importação e as NCC's foram alteradas
421
422         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Existe uma diferença entre as NCC's do orçamento VS a NCC's da venda")
423
424         STFMessage("STISNCCPay","YESNO",STR0018) //"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja continuar"
425
426         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Caso altere a negociação incluindo ou removendo uma NCC sera necessario refazer as formas de pagamento, deseja
continuar")
427
428         If STFShowMessage("STISNCCPay")
429             LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Sim, desejo seguir com a alteração.")
430
431             //Indico que devera limpar as fomar de pagamento com a variavel lZeraPayImport = True
432             lZeraPayImport := .T.
433
434             //Carrego os paineis para os demais pagamentos
435             STIExchangePanel({|| STIPayment() })
436
437             //Adiciono o pagamento da NCC no Grid de pagamentos
438             STIAddPay("CR", Nil, 1, Nil, Nil, nTotalNCCs)
439
440         Else //Desfaço as alterações feitas caso ele negue
441
442             //Indico que não devera limpar as fomar de pagamento com a variavel lZeraPayImport = False
443             lZeraPayImport := .F.
444
445             //Carrego os paineis para os demais pagamentos
446             STIExchangePanel({|| STIPayment() })
447
448             //Adiciono o pagamento da NCC no Grid de pagamentos
449             If nNccSelOld > 0
450                 LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC restaurada no valor de: " + cValToChar(nNccSelOld))
451                 STIAddPay("CR", Nil, 1, Nil, Nil, nNccSelOld)
452             EndIf
453
454             //Restauro o valor das NCC's e o Array aNCCsCli com as NCC's selecionadas
455             STDSetNCCs("2",nNccSelOld)

```

```

456         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"0 valor de pagamento com NCC's restaurado: " + cValToChar(nNccSelOld))
457
458         aNCCsCli := aNccsSelCliOld
459         STDSetNCCs("1",aNCCsCli)
460         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"Array aNCCsCli resturado",aNCCsCli)
461
462     EndIf
463     ElseIf !lImport .OR. (lImport .AND. nNccSelOld == nTotalNCCs) //NCC utilizada em venda Auto Serviço ou não teve alteração nas NCC's.
464         LjGrvLog("L1_NUM: " + STDGPBasket("SL1","L1_NUM"),"NCC utilizada em venda Auto Serviço ou não teve alteração nas NCC's.")
465
466         //Carrego os paineis para os demais pagamentos
467         STIExchangePanel({|| STIPayment() })
468
469         //Adiciono o pagamento da NCC no Grid de pagamentos
470         STIAddPay("CR", Nil, 1, Nil, Nil, nTotalNCCs)
471     EndIf
472 EndIf
473
474 Return Nil
475
476 //-----
477 /* {Protheus.doc} STINCCclearStatic
478 Responsavel por zerar todas as variaveis static
479
480 @return
481 */
482 //-----
483 Function STINCCclearStatic()
484
485     aNCCsCli      := {}
486     oLblSelTotal  := Nil
487     oLblTotalVend := Nil
488
489     STDSetNCCs("1")
490     STDSetNCCs("2")
491
492 Return
493
494 //-----
495 /*{Protheus.doc} STI7SelNcc
496 Responsavel por marcar/desmarcar a seleção automatica dos valores de NCC
497 a serem utilizados na venda.
498
499 @aparam oListNCCs - Objeto do tipo TListBox responsavel por apresentar as NCCs do cliente.
500 */
501 //-----
502 Static Function STI7SelNcc(oListNCCs, lSelVlr)
503 Local lImport      := !Empty(STDGPBasket("SL1" , "L1_NUMORIG")) // Variavel de controle que identifica se é importação de orçamento
504
505 lSelVlr := !lSelVlr
506
507 If !lImport .OR. (lImport .AND. STFProfile(41)[1])

```

```

508 STISelAutNCC(oListNCCs, lSelVlr)
509 Else
510     lSelVlr := .F.
511     STFMessage(ProcName(),"STOP",STR0017)//"Usuario sem permissão para alterar a negociação do orçamento."
512     STFShowMessage(ProcName())
513 EndIf
514
515 Return lSelVlr
516
517 //-----
518 /*{Protheus.doc} STI7SelNcc
519 Responsavel por marcar/desmarcar a seleção automatica dos valores de NCC
520 a serem utilizados na venda.
521
522 @aparam oListNCCs - Objeto do tipo TListBox responsavel por apresentar as NCCs do cliente.
523 @obs Esta selecção segue a ordem crescente das posições do array da NCC.
524 */
525 //-----
526 Static Function STISelAutNCC(oListNCCs, lSelVlr)
527 Local lContinua := .T. // Controle do While
528 Local nLenNCC := 0 // Quantidade maxima de linhas de NCC
529 Local nLine := 1 // Linha atual do objeto de NCC
530 Local nDiferenca := 0 // Diferença de valores entre total venda e NCC selecionadas
531 Local nValRest := 0 // Valor restante para pagamento
532 Local oTotal := STFGetTot() // Recebe o Objeto totalizador
533 Local nTotalVend := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
534 Local nTotalNCCs := STDGetNCCs("2") // Valor total das NCCs
535
536 If ValType(oListNCCs) == "0"
537
538     nLenNCC := Len(oListNCCs:aItems)
539
540     While lContinua
541
542         //-----Seleção de NCCs-----
543         If lSelVlr
544
545             nDiferenca := nTotalVend - nTotalNCCs
546
547             If nDiferenca <= 0 .Or. nLine > nLenNCC
548                 lContinua := .F.
549             Else
550                 oListNCCs:nAt := nLine
551                 If !aNCCsCli[nLine,NCCSELECTED] //Não esta selecionado?
552                     STDSetNCCs("2",aNCCsCli[nLine,SE1SALD0]) // Atualiza o saldo total de NCCs selecionadas
553                     oListNCCs:Modify(cTextSelection+oListNCCs:GetSelText(),nLine) // Modifica o texto para informar que a NCC foi selecionada
554                     aNCCsCli[nLine,NCCSELECTED] := .T. // A NCC é marcada como selecionada
555
556                     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC selecionada automaticamente", cTextSelection+oListNCCs:GetSelText() ) //Grava Log
557                     =====
558                     nTotalNCCs := STDGetNCCs("2") //atualiza valor total de NCC selecionada

```

```

559         EndIf
560
561     EndIf
562
563     nLine++ //Atualiza linha do objeto
564
565     //-----Retirada de seleção de NCCs-----
566     Else
567
568         If nTotalNCCs == 0 .Or. nLine > nLenNCC
569             lContinua := .F.
570         Else
571             oListNCCs:nAt := nLine
572             If aNCCsCli[nLine,NCCSELECTED] //Esta selecionado?
573                 STDSetNCCs("2",-aNCCsCli[nLine,SEISALD0]) //0 saldo da NCC posicionada é deduzido do total selecionado
574                 oListNCCs:Modify(SubStr(oListNCCs:GetSelText(),Len(cTextSelection)+1),nLine) // 0 texto de selecao é removido
575                 aNCCsCli[nLine,NCCSELECTED] := .F. // A NCC é desmarcada como selecionada
576
577                 LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC removida", cTextSelection+oListNCCs:GetSelText() )
578
579                 nTotalNCCs := STDGetNCCs("2") //atualiza valor total de NCC selecionada
580             EndIf
581         EndIf
582
583         nLine++ //Atualiza linha do objeto
584
585     EndIf
586 End
587
588 //Atualizando valores da tela
589 nTotalNCCs := STDGetNCCs("2")
590 If (nTotalVend-nTotalNCCs) > 0
591     nValRest := nTotalVend-nTotalNCCs
592 Else
593     nValRest := 0
594 EndIf
595
596 oLblSelTotal:SetText(STR0008+AllTrim(Str(STDGetNCCs("2")))) //Total Selecionado: "
597 oLblTotalVend:SetText(STR0011 + AllTrim(Str(nValRest,12,2)) + STR0012) //Restam ##para pagar
598
599 LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC - Total selecionado automaticamente", STR0008+AllTrim(Str(STDGetNCCs("2")) )
600 LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "NCC - Total restante", STR0011 + AllTrim(Str(nValRest,12,2)) + STR0012 )
601
602 EndIf
603
604 Return Nil
605
606 //-----
607 /*{Protheus.doc} STIGetZrPg
608 Função de Get para a variavel statica lZeraPayImport, a variavel indica se deverá considerar os pagamentos de um orçamento
609
610 @aparam

```

```
611 @obs
612 */
613 //-----
614 Function STIGetZrPg()
615
616 Return lZeraPayImport
617
618 //-----
619 /*{Protheus.doc} STISetZrPg
620 Função de Set para a variavel statica lZeraPayImport, a variavel indica se deverá considerar os pagamentos de um orçamento
621
622 @aparam
623 @obs
624 */
625 //-----
626 Function STISetZrPg(lZeraPg)
627
628 DEFAULT lZeraPg := .F.
629
630 lZeraPayImport := lZeraPg
631
632 Return
```

Componentes/Interface/STIOpenCash.PRW

144 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIOPENCASH.CH"
5 #INCLUDE "STPOS.CH"
6
7 Static oChange
8
9 //-----
10 /*{Protheus.doc} STIOpenCash
11 Contexto Abertura de Caixa
12
13 @param
14 @return
15 @obs
16 @sample
17 */
18 //-----
19 Function STIOpenCash()
20
21 Local lOpenCash := STB0penCash()
22
23 If !lOpenCash
24     STIExchangePanel( { || STIPanOpenCash(lOpenCash) } )
25 Else
26     STIBtnActivate()
27     STFMessage(ProcName(),"STOP",STR0004) //"Caixa já esta aberto!"
28     STFShowMessage(ProcName())
29     STIRegItemInterface()
30 EndIf
31
32 Return
33
34 //-----
35 /*{Protheus.doc} STIPanOpenCash
36 Contexto Abertura de Caixa
37
38 @param
39 @return
40 @obs
41 @sample
42 */
43 //-----
44 Function STIPanOpenCash(lOpenCash)
45
46 Local oPanelMVC      := STIGetPanel()           //Painel principal do dialog
47 Local oPanOpenCash  := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Painel Abertura de Caixa
48 Local nChange       := 0                       //Alteracao do conteudo do get
49 Local nGetVal       := 0
50 Local aStation      := STBInfoEst( 1, .T. )
51 Local aCaixas       := STIGetSA6(aStation[1])
52 Local aAuxCaixas    := {}

```

```

53 Local nTamSA6      := 0
54 Local cComboCx     := ""
55 Local oSay2        := NIL
56 Local oOrigem      := NIL
57 Local lAlterCx     := !(SuperGetMv( "MV_LJALTCX",,.T.))
58 Local oButCan      := Nil //Botao cancelar
59 Local lMobile      := STFGetCfg( "lMobile" , .F. )//Versao Mobile
60
61 Local nCaixas      := 0
62 Default lOpenCash := .F.
63
64 lMobile := ValType(lMobile) == "L" .AND. lMobile
65
66 If Findfunction( 'STBCashList' )
67   aAuxCaixas := STBCashList()
68 Else
69   nTamSA6 := At("/", "MV_CXLOJA") -1
70   If nTamSA6 <= 0
71     nTamSA6 := SA6->(TamSx3("A6_COD"))[1]
72   EndIf
73
74   cComboCx := Left(GetMv("MV_CXLOJA"),nTamSA6 )
75
76   For nCaixas := 1 to len(aCaixas)
77     AADD(aAuxCaixas,aCaixas[nCaixas][1][1])
78   Next
79 EndIf
80
81 STIBtnDeActivate()
82 STIChangeCssBtn()
83
84 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001 + " - " + aStation[1]},oPanOpenCash,,,,,.T.,,,) //"Abertura de Caixa"
85 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
86
87
88 oSay2:= TSay():New(POSVERT_LABEL1,POSHOR_1,{||"Caixa Origem"},oPanOpenCash,,,,,.T.,,,) //"Caixa Origem"
89 oSay2:SetCSS( POSCSS (GetClassName(oSay2), CSS_LABEL_FOCAL))
90
91 oOrigem := TComboBox():Create(oPanOpenCash, {||u| if( Pcount( )>0, cComboCx := u, cComboCx) }, POSVERT_GET1, POSHOR_1, ;
92   aAuxCaixas, 160, ALTURAGET,,;
93   ,,,;
94   .T.,,,;
95   ,,,;
96   ,cComboCx)
97 oOrigem:SetCSS( POSCSS (GetClassName(oOrigem), IIF(lMobile ,CSS_COMBOBOX ,CSS_LISTBOX) ))
98 oOrigem:lReadOnly := lAlterCx
99
100 oSay3 := TSay():New(POSVERT_LABEL2,POSHOR_1,{||STR0002},oPanOpenCash,,,,,.T.,,,) //"Informe o Troco"
101 oSay3:SetCSS( POSCSS (GetClassName(oSay3), CSS_LABEL_FOCAL ))
102
103 oChange := TGet():New(POSVERT_GET2,POSHOR_1,{||u| If(PCount()>0,nGetVal:=u,nGetVal)},oPanOpenCash,90,ALTURAGET,"@E 99,999,999.99",{|| (nGetVal >= 0)
  },,,,,.T.,,,,,,,,"nGetVal",,,,,.T.)

```

```

104 oChange:SetCSS( POSCSS (GetClassName(oChange), CSS_GET_FOCAL ))
105
106 oButton := TButton():New(   POSVERT_BTNFOCAL,;
107                             POSHOR_BTNFOCAL ,;
108                             STR0003,;
109                             oPanOpenCash ,;
110                             { || IIF( STWOpenSupply(nGetVal, cComboCx, aStation[1],aCaixas,.T.) ,;
111                               Eval({||STIRegItemInterface(.T.)}),;
112                               Nil ) },;
113                             LARGBTN ,;
114                             ALTURABTN ,;
115                             ,,,.T.,;
116                             ,,,{|| !Empty(cComboCx)}) // Abrir Caixa
117
118 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
119
120 If lOpenCash .Or. lMobile //Somente se o caixa já estiver aberto (???) ou se for Mobile
121     oBtnCa := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0005,oPanOpenCash,{|| STIRegItemInterface()}, ; //"Cancelar"
122                             LARGBTN,ALTURABTN,,,,.T.)
123
124     oBtnCa:SetCSS( POSCSS (GetClassName(oBtnCa), CSS_BTN_NORMAL ))
125 EndIf
126
127 STIChangeCssBtn()
128
129 oChange:SetFocus()
130
131 Return(oPanOpenCash)
132
133 //-----
134 /*/{Protheus.doc} STIGetCash
135 Get no objeto text da abertura de caixa
136
137 @param
138 @return
139 @obs
140 @sample
141 */
142 //-----
143 Function STIGetCash()
144 Return oChange

```

Componentes/Interface/STIPanButtons.PRW

72 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STIPANBUTTONS.CH"
4
5 //-----
6 /*{Protoheus.doc} STIPanBtnSale
7 Pannel de Botoes de Contexto
8
9 @param
10 @return
11 @obs
12 @sample
13 */
14 //-----
15 Function STIPanBtnSale( oPanel )
16
17 Local oPanBtnSale      := Nil
18 Local nBtnHeight       := 0
19 Local nBtnWidth        := 0
20 Local nSpace           := 0
21 Local nPercSpace       := 0.015
22 Local nPanHeightPerc   := 0.15
23
24 If oPanel <> Nil
25
26     /*/
27     Panel Botoes da Venda
28     /*/
29     oPanBtnSale := TPanel():New(00,00,"",oPanel,,,,,oPanel:nWidth/2,oPanel:nHeight/2 * nPanHeightPerc)
30
31     /*/
32     Espaco unico definido a partir do percentual referente à largura
33     /*/
34     nSpace      := ((oPanBtnSale:nWidth/2) * nPercSpace)
35
36     nBtnHeight  := ( oPanBtnSale:nHeight/2 - (3 * nSpace) ) / 2
37
38     nBtnWidth   := ( (oPanBtnSale:nWidth/2) - ( 4 * nSpace ) ) / 3
39
40     /*/
41     Registro de Item
42     /*/
43     @ nSpace , nSpace BUTTON STR0001 SIZE nBtnWidth , nBtnHeight PIXEL ACTION STIExchangePanel( { || STIPanItemRegister() } ) OF oPanBtnSale    //"Registro de Item"
44
45     /*/
46     Cliente / CPF
47     /*/
48     @ nSpace , nSpace + nBtnWidth + nSpace BUTTON STR0002 SIZE nBtnWidth , nBtnHeight PIXEL ACTION MSGALERT(STR0002) OF oPanBtnSale //"Cliente / CPF"
49
50     /*/
51     Cancelar Item
52     /*/

```

```
53 @ nSpace , nSpace + nBtnWidth + nSpace + nBtnWidth + nSpace BUTTON STR0003 SIZE nBtnWidth , nBtnHeight PIXEL ACTION STICancelItem() OF oPanBtnSale //"Cancelar Item"
54
55 /*/
56 Orçamento
57 /*/
58 @ nSpace + nBtnHeight + nSpace , nSpace BUTTON STR0004 SIZE nBtnWidth , nBtnHeight PIXEL ACTION STIImportSale() OF oPanBtnSale //"Orçamento"
59
60 /*/
61 Desconto na Venda
62 /*/
63 @ nSpace + nBtnHeight + nSpace , nSpace + nBtnWidth + nSpace BUTTON STR0005 SIZE nBtnWidth , nBtnHeight PIXEL ACTION STIDiscountTotal() OF oPanBtnSale //"Desconto na
Venda"
64
65 /*/
66 Consultar Produto
67 /*/
68 @ nSpace + nBtnHeight + nSpace , nSpace + nBtnWidth + nSpace + nBtnWidth + nSpace BUTTON STR0006 SIZE nBtnWidth , nBtnHeight PIXEL ACTION STIConsult() OF oPanBtnSale
//"Consultar Produto"
69
70 EndIf
71
72 Return(oPanBtnSale)
```

Componentes/Interface/STIParcPay.prw

445 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

```

105      STR0004      ,; //[02] Desc do campo
106      "L1_FILIAL" ,; //[03] Id do Field
107      "C"          ,; //[04] Tipo do campo
108      8            ,; //[05] Tamanho do campo
109      0            ,; //[06] Decimal do campo
110      Nil          ,; //[07] Code-block de validacao do campo
111      Nil          ,; //[08] Code-block de validacao When do campo
112      Nil          ,; //[09] Lista de valores permitido do campo
113      Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
114      Nil          ,; //[11] Code-block de inicializacao do campo
115      Nil          ,; //[12] Indica se trata-se de um campo chave
116      Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
117      .T.          ) //[14] Indica se o campo e virtual
118
119      oStru:AddField( STR0005 ,; //[01] Titulo do campo //"Total Parcelas"
120                     STR0005 ,; //[02] Desc do campo //"Total Parcelas"
121                     "L1_TOTPAR" ,; //[03] Id do Field
122                     "N"          ,; //[04] Tipo do campo
123                     10           ,; //[05] Tamanho do campo
124                     0           ,; //[06] Decimal do campo
125                     Nil          ,; //[07] Code-block de validacao do campo
126                     Nil          ,; //[08] Code-block de validacao When do campo
127                     Nil          ,; //[09] Lista de valores permitido do campo
128                     Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
129                     Nil          ,; //[11] Code-block de inicializacao do campo
130                     Nil          ,; //[12] Indica se trata-se de um campo chave
131                     Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
132                     .T.          ) //[14] Indica se o campo e virtual
133
134      oStru:AddField( STR0006      ,; //[01] Titulo do campo
135                     STR0006      ,; //[02] Desc do campo
136                     "L1_TROCO"   ,; //[03] Id do Field
137                     "N"          ,; //[04] Tipo do campo
138                     16           ,; //[05] Tamanho do campo
139                     2            ,; //[06] Decimal do campo
140                     Nil          ,; //[07] Code-block de validacao do campo
141                     Nil          ,; //[08] Code-block de validacao When do campo
142                     Nil          ,; //[09] Lista de valores permitido do campo
143                     Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
144                     Nil          ,; //[11] Code-block de inicializacao do campo
145                     Nil          ,; //[12] Indica se trata-se de um campo chave
146                     Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
147                     .T.          ) //[14] Indica se o campo e virtual
148
149      oStru:AddField( STR0007 ,; //[01] Titulo do campo
150                     STR0007 ,; //[02] Desc do campo
151                     "L1_TOTVEN" ,; //[03] Id do Field
152                     "N"          ,; //[04] Tipo do campo
153                     16           ,; //[05] Tamanho do campo
154                     2            ,; //[06] Decimal do campo
155                     Nil          ,; //[07] Code-block de validacao do campo
156                     Nil          ,; //[08] Code-block de validacao When do campo

```

```

157         Nil          ;; //[09] Lista de valores permitido do campo
158         Nil          ;; //[10] Indica se o campo tem preenchimento obrigatorio
159         Nil          ;; //[11] Code-block de inicializacao do campo
160         Nil          ;; //[12] Indica se trata-se de um campo chave
161         Nil          ;; //[13] Indica se o campo pode receber valor em uma operacao de update.
162         .T.          ) //[14] Indica se o campo e virtual
163
164 Else
165
166     oStru:AddField( STR0008      ;; //[01] Titulo do campo
167                     STR0008      ;; //[02] Desc do campo
168                     "L1_DATE"    ;; //[03] Id do Field
169                     "D"          ;; //[04] Tipo do campo
170                     8            ;; //[05] Tamanho do campo
171                     0            ;; //[06] Decimal do campo
172                     Nil          ;; //[07] Code-block de validacao do campo
173                     Nil          ;; //[08] Code-block de validacao When do campo
174                     Nil          ;; //[09] Lista de valores permitido do campo
175                     Nil          ;; //[10] Indica se o campo tem preenchimento obrigatorio
176                     Nil          ;; //[11] Code-block de inicializacao do campo
177                     Nil          ;; //[12] Indica se trata-se de um campo chave
178                     Nil          ;; //[13] Indica se o campo pode receber valor em uma operacao de update
179                     .T.          ) //[14] Indica se o campo e virtual
180
181
182     oStru:AddField( STR0009      ;; //[01] Titulo do campo
183                     STR0009      ;; //[02] Desc do campo
184                     "L1_FORMA"   ;; //[03] Id do Field
185                     "C"          ;; //[04] Tipo do campo
186                     5            ;; //[05] Tamanho do campo
187                     0            ;; //[06] Decimal do campo
188                     Nil          ;; //[07] Code-block de validacao do campo
189                     Nil          ;; //[08] Code-block de validacao When do campo
190                     Nil          ;; //[09] Lista de valores permitido do campo
191                     Nil          ;; //[10] Indica se o campo tem preenchimento obrigatorio
192                     Nil          ;; //[11] Code-block de inicializacao do campo
193                     Nil          ;; //[12] Indica se trata-se de um campo chave
194                     Nil          ;; //[13] Indica se o campo pode receber valor em uma operacao de update
195                     .T.          ) //[14] Indica se o campo e virtual
196
197
198     oStru:AddField( STR0010      ;; //[01] Titulo do campo
199                     STR0010      ;; //[02] Desc do campo //"Valor Pago"
200                     "L1_VLPG"    ;; //[03] Id do Field
201                     "N"          ;; //[04] Tipo do campo
202                     16           ;; //[05] Tamanho do campo
203                     2            ;; //[06] Decimal do campo
204                     Nil          ;; //[07] Code-block de validacao do campo
205                     Nil          ;; //[08] Code-block de validacao When do campo
206                     Nil          ;; //[09] Lista de valores permitido do campo
207                     Nil          ;; //[10] Indica se o campo tem preenchimento obrigatorio
208                     Nil          ;; //[11] Code-block de inicializacao do campo

```

[illegible]

```

261      Nil      ,; //[15] CINIBROW - INICIALIZADOR DO BROWSE
262      .T.      ,; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
263      Nil      ) //[17] CPICTVAR - PICTURE VARIÁVEL
264
265      oStru:AddField( "L1_FORMA" ,; //[01] CIDFIELD - ID DO FIELD
266                    "02"      ,; //[02] CORDEM - ORDEM DO CAMPO
267                    STR0009 ,; //[03] CTITULO - TITULO DO CAMPO
268                    STR0009 ,; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
269                    {}      ,; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
270                    "C"      ,; //[06] CTYPE - TIPO
271                    "@!"    ,; //[07] CPICTURE - PICTURE DO CAMPO
272                    Nil      ,; //[08] BPICTVAR - BLOCO DE PICTURE VAR
273                    ""      ,; //[09] CLOOKUP - CHAVE PARA SER USADO NO LOOKUP
274                    .T.      ,; //[10] LCANCHANGE - LOGICO DIZENDO SE O CAMPO PODE SER ALTERADO
275                    Nil      ,; //[11] CFOLDER - ID DA FOLDER ONDE O FIELD ESTA
276                    Nil      ,; //[12] CGROUP - ID DO GROUP ONDE O FIELD ESTA
277                    Nil      ,; //[13] ACOMBOVALUES - ARRAY COM OS VALORES DO COMBO
278                    Nil      ,; //[14] NMAXLENCOMBO - TAMANHO MAXIMO DA MAIOR OPÇÃO DO COMBO
279                    Nil      ,; //[15] CINIBROW - INICIALIZADOR DO BROWSE
280                    .T.      ,; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
281                    Nil      ) //[17] CPICTVAR - PICTURE VARIÁVEL
282
283      oStru:AddField( "L1_VLPG" ,; //[01] CIDFIELD - ID DO FIELD
284                    "03"      ,; //[02] CORDEM - ORDEM DO CAMPO
285                    STR0010 ,; //[03] CTITULO - TITULO DO CAMPO
286                    STR0010 ,; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
287                    {}      ,; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
288                    "N"      ,; //[06] CTYPE - TIPO
289                    ""      ,; //[07] CPICTURE - PICTURE DO CAMPO
290                    Nil      ,; //[08] BPICTVAR - BLOCO DE PICTURE VAR
291                    ""      ,; //[09] CLOOKUP - CHAVE PARA SER USADO NO LOOKUP
292                    .T.      ,; //[10] LCANCHANGE - LOGICO DIZENDO SE O CAMPO PODE SER ALTERADO
293                    Nil      ,; //[11] CFOLDER - ID DA FOLDER ONDE O FIELD ESTA
294                    Nil      ,; //[12] CGROUP - ID DO GROUP ONDE O FIELD ESTA
295                    Nil      ,; //[13] ACOMBOVALUES - ARRAY COM OS VALORES DO COMBO
296                    Nil      ,; //[14] NMAXLENCOMBO - TAMANHO MAXIMO DA MAIOR OPÇÃO DO COMBO
297                    Nil      ,; //[15] CINIBROW - INICIALIZADOR DO BROWSE
298                    .T.      ,; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
299                    Nil      ) //[17] CPICTVAR - PICTURE VARIÁVEL
300
301      oStru:AddField( "L1_PARC" ,; //[01] CIDFIELD - ID DO FIELD
302                    "04"      ,; //[02] CORDEM - ORDEM DO CAMPO
303                    STR0001 ,; //[03] CTITULO - TITULO DO CAMPO
304                    STR0001 ,; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
305                    {}      ,; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
306                    "C"      ,; //[06] CTYPE - TIPO
307                    "@!"    ,; //[07] CPICTURE - PICTURE DO CAMPO
308                    Nil      ,; //[08] BPICTVAR - BLOCO DE PICTURE VAR
309                    ""      ,; //[09] CLOOKUP - CHAVE PARA SER USADO NO LOOKUP
310                    .T.      ,; //[10] LCANCHANGE - LOGICO DIZENDO SE O CAMPO PODE SER ALTERADO
311                    Nil      ,; //[11] CFOLDER - ID DA FOLDER ONDE O FIELD ESTA
312                    Nil      ,; //[12] CGROUP - ID DO GROUP ONDE O FIELD ESTA

```

```

313 Nil                ;; //[13] ACOMBOVALUES - ARRAY COM OS VALORES DO COMBO
314 Nil                ;; //[14] NMAXLENCOMBO - TAMANHO MAXIMO DA MAIOR OPÇÃO DO COMBO
315 Nil                ;; //[15] CINIBROW - INICIALIZADOR DO BROWSE
316 .T.                ;; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
317 Nil                ) //[17] CPICTVAR - PICTURE VARIÁVEL
318
319 Else
320
321   oStru:AddField(    "L1_TOTVEN"      ;; //[01] CIDFIELD - ID DO FIELD
322                     "05"              ;; //[02] CORDEM - ORDEM DO CAMPO
323                     STR0007           ;; //[03] CTITULO - TITULO DO CAMPO
324                     STR0007           ;; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
325                     {}                ;; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
326                     "N"              ;; //[06] CTYPE - TIPO
327                     "@E 999,999.99" ;; //[07] CPICTURE - PICTURE DO CAMPO
328                     Nil              ;; //[08] BPICTVAR - BLOCO DE PICTURE VAR
329                     ""               ;; //[09] CLOOKUP - CHAVE PARA SER USADO NO LOOKUP
330                     .F.              ;; //[10] LCANCHARGE - LOGICO DIZENDO SE O CAMPO PODE SER ALTERADO
331                     Nil              ;; //[11] CFOLDER - ID DA FOLDER ONDE O FIELD ESTA
332                     Nil              ;; //[12] CGROUP - ID DO GROUP ONDE O FIELD ESTA
333                     Nil              ;; //[13] ACOMBOVALUES - ARRAY COM OS VALORES DO COMBO
334                     Nil              ;; //[14] NMAXLENCOMBO - TAMANHO MAXIMO DA MAIOR OPÇÃO DO COMBO
335                     Nil              ;; //[15] CINIBROW - INICIALIZADOR DO BROWSE
336                     .T.              ;; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
337                     Nil              ) //[17] CPICTVAR - PICTURE VARIÁVEL
338
339   oStru:AddField(    "L1_TOTPAR"      ;; //[01] CIDFIELD - ID DO FIELD
340                     "06"              ;; //[02] CORDEM - ORDEM DO CAMPO
341                     STR0005           ;; //[03] CTITULO - TITULO DO CAMPO
342                     STR0005           ;; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
343                     {}                ;; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
344                     "N"              ;; //[06] CTYPE - TIPO
345                     "@E 999999"      ;; //[07] CPICTURE - PICTURE DO CAMPO
346                     Nil              ;; //[08] BPICTVAR - BLOCO DE PICTURE VAR
347                     ""               ;; //[09] CLOOKUP - CHAVE PARA SER USADO NO LOOKUP
348                     .F.              ;; //[10] LCANCHARGE - LOGICO DIZENDO SE O CAMPO PODE SER ALTERADO
349                     Nil              ;; //[11] CFOLDER - ID DA FOLDER ONDE O FIELD ESTA
350                     Nil              ;; //[12] CGROUP - ID DO GROUP ONDE O FIELD ESTA
351                     Nil              ;; //[13] ACOMBOVALUES - ARRAY COM OS VALORES DO COMBO
352                     Nil              ;; //[14] NMAXLENCOMBO - TAMANHO MAXIMO DA MAIOR OPÇÃO DO COMBO
353                     Nil              ;; //[15] CINIBROW - INICIALIZADOR DO BROWSE
354                     .T.              ;; //[16] LVIRTUAL - INDICA SE O CAMPO é VIRTUAL
355                     Nil              ) //[17] CPICTVAR - PICTURE VARIÁVEL
356
357   oStru:AddField(    "L1_TROCO"      ;; //[01] CIDFIELD - ID DO FIELD
358                     "07"              ;; //[02] CORDEM - ORDEM DO CAMPO
359                     STR0006           ;; //[03] CTITULO - TITULO DO CAMPO
360                     STR0006           ;; //[04] CDESCRIC - ESCRICAO COMPLETA DO CAMPO
361                     {}                ;; //[05] AHELP - ARRAY COM O HELP DOS CAMPOS
362                     "N"              ;; //[06] CTYPE - TIPO
363                     "@E 999,999.99" ;; //[07] CPICTURE - PICTURE DO CAMPO
364                     Nil              ;; //[08] BPICTVAR - BLOCO DE PICTURE VAR

```

[illegible]

[illegible]

Componentes/Interface/STIPayCard.prw

885 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "POSCSS.CH"
7 #INCLUDE "STIPAYCARD.CH"
8 #INCLUDE "STPOS.CH"
9
10 Static lCard      := .F.      //Informa se a venda teve transacao em cartao
11 Static oModel     := Nil      //Model de forma de pagamento cartao
12 Static cCboAdmFin := ''      //Administradora financeira
13 Static lPAFECF    := STBIsPAF() //Verifica se é PAF-ECF
14 Static lHomolPaf  := STBHomolPaf() //Homologação PAF-ECF
15 Static lContTef   := .F.      //Se ira utilizar a contingencia de passar no POS
16 Static nCards     := 1       // Contador para quando importar um orçamento com pagamento em mais de um cartão
17 Static aJurosAdm := {.F., "", 0, 0} // Indica se teve cálculo de Juros de Adm Financeira
18                                     //aJurosAdm[1] -> Tem Juros | aJurosAdm[2] -> Codigo da Adm | aJurosAdm[3] -> Aliquota | aJurosAdm[4] -> Valor dos Juros
19
20 Static oLblData   := Nil      //Objeto de Label Data
21 Static oGetData   := Nil      //Objeto de Get Data
22 Static cGetData   := ''      //Recebe valor de oGetData
23 Static oLblValor  := Nil      //Objeto de Label Valor
24 Static oGetValor  := Nil      //Objeto de Get Valor
25 Static cGetVal    := STBCalcSald("1") //Apresenta o saldo restante do pagamento e recebe valor de oGetValor
26 Static oLblParcels := Nil      //Objeto de Label Parcelas
27 Static oGetParcels := Nil      //Objeto de Get Parcelas
28 Static nGetParc   := 1       //Recebe valor de oGetParcels
29 Static oBtnOk     := Nil      //Objeto do botao Ok
30 Static oBtnCa     := Nil      //Objeto do botao Cancelar
31 Static oLblAdmFin := Nil      //Objeto de Label Adm Financeira
32 Static oCboAdmFin := Nil      //Objeto de Combo Adm Financiara
33 Static oLblNSU    := Nil      //Objeto de Label NSU
34 Static oGetNSU    := Nil      //Objeto de Get NSU
35 Static cGetNSU    := Space(TAMSx3("L4_NSUTEF")[1]) //Vaiavel Get NSU
36 Static oLblAutoiz := Nil      //Objeto de Label Autorização
37 Static oGetAutoriz := Nil      //Objeto de Get Autorização
38 Static cGetAutoriz := Space(TAMSx3("L4_AUTORIZ")[1]) //Vaiavel Get Autorização
39 Static oLblJurAdm := Nil      /// Objeto Label Juros Adm Fin
40 Static oGetJurAdm := Nil      /// Objeto Get Juros Adm Fin
41 Static nGetJurAdm := Val(Space(TamSX3("L1_JUROS")[1])) //Vaiavel Get Juros Adm Fin
42 Static aAdmFin    := {}      // Array onde carrega as administradoras financeiras
43 Static lMultNeg   := .F.      // Verifica se foi usando Multi Negociação
44
45
46 //-----
47 /*{Protheus.doc} STIPayCard
48
49 @param
50 @return
51 @obs
52 @sample

```

```

53 /*/
54 //-----
55 Function STIPayCard(oPnlAdconal,cTypeCard,nValue, nParcels )
56
57 Local nVlJuroAdm      := 0
58 Local lMVLJJURCC      := SuperGetMV( "MV_LJJURCC",,.F. ) // calcula juros da Adm financeira
59 Local bChange         := {} STBCalcTax(cCboAdmFin, nGetParc, cGetVal, @nVlJuroAdm), nGetJurAdm := nVlJuroAdm, oGetJurAdm:Refresh()
60 Local lReadOnly       := STIGetPayR0() //indica se os campos de pagamento estão como Somente Leitura (permissão Alterar Parcelas do caixa)
61 Local lRecebTit       := STIGetRecTit() //Indica se eh recebimento de titulos
62 Local lReadOnTEF      := IIF(ExistFunc("LjVincPgEl"),LjVincPgEl(),.F.) //Verifica se obriga a digitação do NSU e o código de autorização
63
64 Default nValue        := 0
65 Default nParcels      := 1 //Recebe valor de oGetParcels
66
67 cGetVal      := STBCalcSald("1")
68 cGetNSU      := Space(TAMsX3("L4_NSUTEF")[1])
69 cGetAutoriz  := Space(TAMsX3("L4_AUTORIZ")[1])
70
71
72 If nValue > 0
73     cGetVal := nValue
74 EndIf
75
76 If nParcels > 0
77     nGetParc := nParcels
78 Else
79     nParcels := 1
80 EndIf
81
82 If IsPDOrPix(cTypeCard) .AND. lContTef
83     STISetContTef(.F.)
84 EndIf
85
86 If lMVLJJURCC .AND. lContTef .AND. cTypeCard $ "CC|CD"
87     cGetVal:= cGetVal - nGetJurAdm
88     nVlJuroAdm := nGetJurAdm
89 EndIf
90
91 If lContTef .OR. ((!STWChkTef("CC") .AND. cTypeCard == "CC") .Or. (!STWChkTef("CD") .AND. cTypeCard == "CD")) .OR. (!STWChkTef("PD") .AND. cTypeCard == "PD") .OR.
    (!STWChkTef("PX") .AND. cTypeCard == "PX")
    If cTypeCard == 'CC'
        If lRecebTit //quando é recebimento de título, o vencimento é calculado no LojXRec
        cGetData := dDataBase
        else
        cGetData := dDataBase + IIF(ValType(SuperGetMV("MV_LJINTER",Nil,'')) <> 'N',30,SuperGetMV("MV_LJINTER"))
        Endif
        aAdmFin := STDAdmFinan('CC')
    ElseIf cTypeCard == 'CD'
        cGetData := dDataBase
        aAdmFin := STDAdmFinan('CD')
    EndIf
92
93
94
95
96
97
98
99
100
101
102
103

```

```

104 Else
105     If cTypeCard == 'CC'
106         If lRecebTitle //quando é recebimento de título, o vencimento é calculado no LojXRec
107             cGetData := dDataBase
108         Else
109             cGetData := dDataBase + IIF(ValType(SuperGetMV("MV_LJINTER",Nil,'')) <> 'N',30,SuperGetMV("MV_LJINTER"))
110         Endif
111     ElseIf cTypeCard == 'CD' .OR. cTypeCard == 'PD' .OR. cTypeCard == 'PX'
112         cGetData := dDataBase
113     EndIf
114
115     If lMVLJJURCC
116         aAdmFin := STDAdmFinan(cTypeCard)
117     EndIf
118 EndIf
119
120 If lContTef .Or. nCards == 1
121
122     If STIFMultNeg() //Caso Seja multi Negociação não pode altear o valor da parcelas e as parcelas.
123         lReadOnly := .T.
124         lMultNeg := .T.
125     ElseIf lMultNeg
126         lReadOnly := STIGetPayR0()
127         lMultNeg := .F.
128     EndIf
129
130     If oLblData <> Nil
131         STIClnVar(.F.)
132     EndIf
133
134     /* Label e Get: Data */
135     oLblData := TSay():New(POSVERT_PAYLABEL1, POSHOR_PAYCOL1, {||STR0001}, oPnlAdconal,, ,,,,T,,,,8)
136     oGetData := TGet():New(POSVERT_PAYGET1,POSHOR_PAYCOL1,{|u| If(PCount()>0,cGetData:=u,cGetData)},oPnlAdconal,LARG_GET_DATE,ALTURAGET-5,,,,,,,,T,,,,{||.F.},,,,,,,,,,T.)
137
138     oLblData:SetCSS( POSCSS (GetClassName(oLblData), CSS_LABEL_FOCAL ))
139     oGetData:SetCSS( POSCSS (GetClassName(oGetData), CSS_GET_NORMAL ))
140
141     /* Label e Get: Valor */
142     oLblValor := TSay():New(POSVERT_PAYLABEL1, 123, {||STR0002}, oPnlAdconal,, ,,,,T,,,,8)
143     oGetValor := TGet():New(POSVERT_PAYGET1-2, 123,{|u| If(PCount()>0,cGetVal:=u,cGetVal)},oPnlAdconal,LARG_GET_VALOR,ALTURAGET,"@E
144     99,999,999.99",bChange,,,,,T,,,,,,,,lReadOnly,,,,,T.)
145
146     oLblValor:SetCSS( POSCSS (GetClassName(oLblValor), CSS_LABEL_FOCAL ))
147     oGetValor:SetCSS( POSCSS (GetClassName(oGetValor), CSS_GET_NORMAL ))
148
149     /* Label e Get: Parcelas */
150     oLblParcels := TSay():New(POSVERT_PAYLABEL1, 204, {||STR0003}, oPnlAdconal,, ,,,,T,,,,8)
151     oGetParcels := TGet():New(POSVERT_PAYGET1-2, 204,{|u| If(PCount()>0,nGetParc:=u,nGetParc)},oPnlAdconal,40,ALTURAGET,"@E 9999",bChange,,,,,T,,,,,,,,lReadOnly,,,,,T.)
152
153     oLblParcels:SetCSS( POSCSS (GetClassName(oLblParcels), CSS_LABEL_FOCAL ))
154     oGetParcels:SetCSS( POSCSS (GetClassName(oGetParcels), CSS_GET_NORMAL ))

```

```

155 /* Juros Adm. Fin. */
156 oLblJurAdm := TSay():New(POSVERT_PAYLABEL3, POSHOR_PAYCOL1, {|| STR0025}, oPnlAdconal,, ,,,,T,,,,8) // "Juros Adm. Fin."
157 oGetJurAdm := TGet():New(POSVERT_PAYGET3-3, POSHOR_PAYCOL1, {|u| nGetJurAdm := nVlJuroAdm }, oPnlAdconal, 60, ALTURAGET-2, "@E 99,999,999.99", ,,,,T,,,,,T,,,,,T.)
158
159 oLblJurAdm:SetCSS( POSCSS (GetClassName(oLblJurAdm), CSS_LABEL_FOCAL ))
160 oGetJurAdm:SetCSS( POSCSS (GetClassName(oGetJurAdm), CSS_GET_NORMAL ))
161
162 oLblJurAdm:Hide()
163 oGetJurAdm:Hide()
164
165
166 //so exibe combo da administradora se estiver cadastrada
167 If Len(aAdmFin) > 0
168
169 /* Label e Get: Administradora Financeira */
170 oLblAdmFin := TSay():New(POSVERT_PAYLABEL2, POSHOR_PAYCOL1, {|| STR0007}, oPnlAdconal,, ,,,,T,,,,8) //'Adm.Financeira'
171 oCboAdmFin := TComboBox():New( POSVERT_PAYGET2-2 , POSHOR_PAYCOL1, {|u| If(PCount()>0,cCboAdmFin:=u,cCboAdmFin)} , aAdmFin , , ;
172 LARG_GET_DATE+40, 80 , oPnlAdconal , Nil , ;
173 bChange , /* bValid*/ , /* nClrBack*/ , /* nClrText*/ , ;
174 .T./ * lPixel*/ , Nil , Nil , ;
175 /* bWhen*/ , Nil , Nil , Nil , ;
176 Nil , cCboAdmFin , /* cLabelText*/ , /* nLabelPos*/ , ;
177 Nil , /* nLabelColor*/ )
178
179 oCboAdmFin:lEditable := .T.
180 oCboAdmFin:llistOnly := .T.
181 oLblAdmFin:SetCSS( POSCSS (GetClassName(oLblAdmFin), CSS_LABEL_FOCAL ))
182 oCboAdmFin:SetCSS( POSCSS (GetClassName(oCboAdmFin), CSS_GET_NORMAL ))
183 oCboAdmFin:SetItems(aAdmFin)
184
185 EndIf
186
187 If lMVLJJURCC .AND. cTypeCard $ "CC|CD" .AND. oLblJurAdm <> Nil
188 oLblJurAdm:Show()
189 oGetJurAdm:Show()
190 EndIf
191
192 If lContTef .OR. ((!STWChkTef("CC") .AND. cTypeCard == "CC") .OR. (!STWChkTef("CD") .AND. cTypeCard == "CD")) .OR. (!STWChkTef("PD") .AND. cTypeCard == "PD") .OR.
(!STWChkTef("PX") .AND. cTypeCard == "PX")
193
194 If oLblAdmFin <> Nil
195 oLblAdmFin:Show()
196 oCboAdmFin:Show()
197 EndIf
198 /* NSU */
199 oLblNSU := TSay():New(POSVERT_PAYLABEL2, 123 , {|| STR0011}, oPnlAdconal,, ,,,,T,,,,8) // "NSU"
200 oGetNSU := TGet():New(POSVERT_PAYGET2-2, 123 , {|u| If(PCount()>0,cGetNSU:=u,cGetNSU)}, oPnlAdconal, 80, ALTURAGET, "@! " +
Replicate("N", Len(cGetNSU)), ,,,,T,,,,,lReadOnTEF,,,,,T.)
201
202 oLblNSU:SetCSS( POSCSS (GetClassName(oLblNSU), CSS_LABEL_FOCAL ))
203 oGetNSU:SetCSS( POSCSS (GetClassName(oGetNSU), CSS_GET_NORMAL ))
204

```

[illegible]

[illegible]

```

306          Nil          ;; //[13] Indica se o campo pode receber valor em uma operacao de update.
307          .T.          ) //[14] Indica se o campo e virtual
308
309 oStru:AddField( STR0002 ;; //[01] Titulo do campo
310 STR0002 ;; //[02] Desc do campo
311 "L4_VALOR" ;; //[03] Id do Field
312 "N" ;; //[04] Tipo do campo
313 16 ;; //[05] Tamanho do campo
314 2 ;; //[06] Decimal do campo
315 Nil ;; //[07] Code-block de validacao do campo
316 Nil ;; //[08] Code-block de validacao When do campo
317 Nil ;; //[09] Lista de valores permitido do campo
318 Nil ;; //[10] Indica se o campo tem preenchimento obrigatorio
319 Nil ;; //[11] Code-block de inicializacao do campo
320 Nil ;; //[12] Indica se trata-se de um campo chave
321 Nil ;; //[13] Indica se o campo pode receber valor em uma operacao de update.
322 .T. ) //[14] Indica se o campo e virtual
323
324 oStru:AddField( STR0003 ;; //[01] Titulo do campo
325 STR0003 ;; //[02] Desc do campo
326 "L4_PARC" ;; //[03] Id do Field
327 "N" ;; //[04] Tipo do campo
328 10 ;; //[05] Tamanho do campo
329 0 ;; //[06] Decimal do campo
330 Nil ;; //[07] Code-block de validacao do campo
331 Nil ;; //[08] Code-block de validacao When do campo
332 Nil ;; //[09] Lista de valores permitido do campo
333 Nil ;; //[10] Indica se o campo tem preenchimento obrigatorio
334 Nil ;; //[11] Code-block de inicializacao do campo
335 Nil ;; //[12] Indica se trata-se de um campo chave
336 Nil ;; //[13] Indica se o campo pode receber valor em uma operacao de update.
337 .T. ) //[14] Indica se o campo e virtual
338
339 oStru:AddField( STR0008 ;; //[01] Titulo do campo //'Adm.Finan'
340 STR0008 ;; //[02] Desc do campo //'Adm.Finan'
341 "L4_ADMINIS" ;; //[03] Id do Field
342 "C" ;; //[04] Tipo do campo
343 TamSx3('L4_ADMINIS')[1] ;; //[05] Tamanho do campo
344 0 ;; //[06] Decimal do campo
345 Nil ;; //[07] Code-block de validacao do campo
346 Nil ;; //[08] Code-block de validacao When do campo
347 Nil ;; //[09] Lista de valores permitido do campo
348 Nil ;; //[10] Indica se o campo tem preenchimento obrigatorio
349 Nil ;; //[11] Code-block de inicializacao do campo
350 Nil ;; //[12] Indica se trata-se de um campo chave
351 Nil ;; //[13] Indica se o campo pode receber valor em uma operacao de update.
352 .T. ) //[14] Indica se o campo e virtual
353
354 oStru:AddField( STR0011 ;; //[01] Titulo do campo //"NSU"
355 STR0011 ;; //[02] Desc do campo //"NSU"
356 "L4_NSUTEF" ;; //[03] Id do Field
357 "C" ;; //[04] Tipo do campo

```

```

358      TamSx3('L4_NSUTEF')[1],; //[05] Tamanho do campo
359      0      ,; //[06] Decimal do campo
360      Nil      ,; //[07] Code-block de validacao do campo
361      Nil      ,; //[08] Code-block de validacao When do campo
362      Nil      ,; //[09] Lista de valores permitido do campo
363      Nil      ,; //[10] Indica se o campo tem preenchimento obrigatorio
364      Nil      ,; //[11] Code-block de inicializacao do campo
365      Nil      ,; //[12] Indica se trata-se de um campo chave
366      Nil      ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
367      .T.      ) //[14] Indica se o campo e virtual
368
369  oStru:AddField( STR0012      ,; //[01] Titulo do campo  //"Autorização"
370                  STR0012      ,; //[02] Desc do campo   //"Autorização"
371                  "L4_AUTORIZ",; //[03] Id do Field
372                  "C"           ,; //[04] Tipo do campo
373                  TamSx3('L4_AUTORIZ')[1],; //[05] Tamanho do campo
374                  0             ,; //[06] Decimal do campo
375                  Nil           ,; //[07] Code-block de validacao do campo
376                  Nil           ,; //[08] Code-block de validacao When do campo
377                  Nil           ,; //[09] Lista de valores permitido do campo
378                  Nil           ,; //[10] Indica se o campo tem preenchimento obrigatorio
379                  Nil           ,; //[11] Code-block de inicializacao do campo
380                  Nil           ,; //[12] Indica se trata-se de um campo chave
381                  Nil           ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
382                  .T.           ) //[14] Indica se o campo e virtual
383
384  oStru:AddField( STR0020      ,; //[01] Titulo do campo  //'ID Cartão'
385                  STR0021      ,; //[02] Desc do campo   //'Identific. Cartão'
386                  "L4_FORMAID" ,; //[03] Id do Field
387                  "C"           ,; //[04] Tipo do campo
388                  TamSX3('L4_FORMAID')[1] ,; //[05] Tamanho do campo
389                  0             ,; //[06] Decimal do campo
390                  Nil           ,; //[07] Code-block de validacao do campo
391                  Nil           ,; //[08] Code-block de validacao When do campo
392                  Nil           ,; //[09] Lista de valores permitido do campo
393                  Nil           ,; //[10] Indica se o campo tem preenchimento obrigatorio
394                  Nil           ,; //[11] Code-block de inicializacao do campo
395                  Nil           ,; //[12] Indica se trata-se de um campo chave
396                  Nil           ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
397                  .T.           ) //[14] Indica se o campo e virtual
398
399  oStru:AddField( STR0026      ,; //[01] Titulo do campo  //"Doc. TEF"
400                  STR0026      ,; //[02] Desc do campo   //"Doc. TEF"
401                  "L4_DOCTEF",; //[03] Id do Field
402                  "C"           ,; //[04] Tipo do campo
403                  TamSx3('L4_DOCTEF')[1],; //[05] Tamanho do campo
404                  0             ,; //[06] Decimal do campo
405                  Nil           ,; //[07] Code-block de validacao do campo
406                  Nil           ,; //[08] Code-block de validacao When do campo
407                  Nil           ,; //[09] Lista de valores permitido do campo
408                  Nil           ,; //[10] Indica se o campo tem preenchimento obrigatorio
409                  Nil           ,; //[11] Code-block de inicializacao do campo

```

[illegible]

```

462
463 Default lPContTef      := .F.
464 Default cGetNSU        := ""
465 Default cGetAutoriz    := ""
466 Default nParcel        := IIF(ValType(oGetParcels) == '0', Val(oGetParcels:cText) , 1)
467
468 //Parcela tem que ser 1 - mesmo se for a vista
469 If nParcel < 1
470     nParcel := 1
471 EndIf
472
473 ModelCard()
474 oMdl := oModel:GetModel("CARDMASTER")
475
476 oMdl:DeActivate()
477 oMdl:Activate()
478
479 oMdl:LoadValue("L4_FILIAL", xFilial("SL4"))
480
481 If !LJRMBA .AND. !Empty(cCboAdmFin) .AND. cTypeCard == "CD" .AND. ExistFunc("STDGetDias")
482     oMdl:LoadValue("L4_DATA", oGetData:cText + STDGetDias(SubStr(cCboAdmFin,1,TamSx3('L4_ADMINIS')[1])))
483 Else
484     oMdl:LoadValue("L4_DATA", oGetData:cText)
485 EndIf
486
487 oMdl:LoadValue("L4_VALOR", oGetValor:cText + oGetJurAdm:cText)
488 If SL4->(ColumnPos("L4_ACRCART")) > 0
489     oMdl:LoadValue("L4_ACRCART", oGetJurAdm:cText)
490 EndIf
491 oMdl:LoadValue("L4_PARC", nParcel)
492 oMdl:LoadValue("L4_ADMINIS", SubStr(cCboAdmFin,1,TamSx3('L4_ADMINIS')[1]))
493
494 If !Empty(cGetNSU)
495     oMdl:LoadValue("L4_NSUTEF", cGetNSU)
496     oMdl:LoadValue("L4_DOCTEF", cGetNSU)
497 EndIf
498 If !Empty(cGetAutoriz)
499     oMdl:LoadValue("L4_AUTORIZ", cGetAutoriz)
500 EndIf
501
502 nTotPago := STIGetTotal() + oGetValor:cText + oGetJurAdm:cText //Retorna o Total Pago
503 nTotVenda := STDGPBasket( "SL1" , "L1_VLRTOT" ) + oGetJurAdm:cText
504 /*
505     Verificar se o cliente deseja doar para o Instituto Arredondar
506 */
507 If cTypeCard $ "CC|CD" //Ideal perguntar antes do TEF
508     STBInsArredondar(AllTrim(cTypeCard))
509 EndIf
510
511 //Nao permite troco em cartao, em caso de homologacao de PAF, deverá ser realizado tratamento por meio de lHomolog
512 If !lRecebTitle .AND. nTotPago > nTotVenda
513     If ((STWChkTef("CC") .Or. cTypeCard == "CC") .Or. (STWChkTef("CD") .Or. cTypeCard == "CD") .Or. (STWChkTef("PD") .Or. cTypeCard == "PD") .Or. (STWChkTef("PX") .Or.

```

```

        cTypeCard == "PX"))
514     lRet := .F.
515     STFMessage(ProcName(),"STOP",STR0009 ) // "Valor informado superior ao saldo a pagar. Não é permitido troco em cartão."
516     STFShowMessage(ProcName())
517     STFCleanMessage(ProcName())
518     EndIf
519 EndIf
520
521 //Valida a digitação do NSU e código de autorização
522 If lVldDigNSU
523     If lValidaNSU .AND. (lPContTef .OR. (!STWChkTef("CC") .AND. cTypeCard $ "CC|CD")) .AND. (Empty(cGetNSU) .OR. Empty(cGetAutoriz))
524         lRet := .F.
525         STFMessage(ProcName(),"STOP",STR0022) //"Por favor, preencha o código NSU e o código de autorização!"
526         STFShowMessage(ProcName())
527     EndIf
528 EndIf
529
530 If lRet .AND. lPromPgto .AND. lFuncPromo .AND. nTotDA1 > 0
531     STBVldDesc('2',,oMdl:GetValue("L4_VALOR"))
532     STBSetReg(.T.)
533 EndIf
534
535 If lRet
536     STIBtnDeActivate()
537
538     If lHomolPaf
539         STFCleanInterfaceMessage()
540     EndIf
541
542
543
544     oTEF20 := STBGetTEF()
545     //Verifica se a transacao foi realizada com sucesso
546     lConfPay := STWTypeTran(oMdl, oTEF20, cTypeCard, nParcel, lPContTef)
547
548     If lIsPgtPOS .And. ExistFunc("LJBldVLRST") .And. cTypeCard $ "CC|CD"
549         LJBldVLRST( !lConfPay )
550     EndIf
551
552     aCallADM := STIGetaCallADM()
553     aFormMultNeg := Iif(ExistFunc("STIGetFormMN"), STIGetFormMN(), {})
554
555     If STBIsImpOrc() .And. Len(aCallADM) > 1 .And. (nCards < Len(aCallADM) .Or. !lConfPay) .And. ( aCallADM[nCards + IIF(lConfPay, 1, 0) ,5] $ "CC|CD" )
556         If lConfPay
557             nCards += 1
558         EndIf
559         // Atualiza os valores do Painel para o proximo Cartão
560         STIPayCard(oPnlAdconal, aCallAdm[nCards][5], aCallAdm[nCards][2], aCallAdm[nCards][3])
561     ElseIf STIGetMult() .And. Len(aFormMultNeg) > 1 .And. aFormMultNeg[nCards][4] .And. aFormMultNeg[nCards + IIF(lConfPay, 1, 0)][1] $ "CC|CD"
562         // Atualiza os valores do Painel para o proximo Cartão da Multinegociacao
563         STIPayCard(oPnlAdconal, aFormMultNeg[nCards][1], aFormMultNeg[nCards][2], aFormMultNeg[nCards][3])
564

```

[illegible]

[illegible]

[illegible]

[illegible]

```

772 aListAdms[oListAdms:nAt,1] := !aListAdms[oListAdms:nAt,1]
773
774 For nI := 1 To Len(aListAdms)
775     If nI <> oListAdms:nAt
776         aListAdms[nI,1] := .F.
777     EndIf
778 Next nI
779
780 nPosAdm := oListAdms:nAt
781 oListAdms:Refresh()
782
783 Return(Nil)
784
785
786 //-----
787 /*{Protheus.doc} STIClnVar
788 Função para limpar os objetos Static da criação da tela de cartão
789 @param
790 @return Nil
791 @obs
792 @sample
793 */
794 //-----
795 Static Function STIClnVar(lAll)
796
797
798 IIf (ValType(oLblData)      == '0', (oLblData      := Nil, FreeObj( oLblData ) ), Nil)
799 IIf (ValType(oGetData)      == '0', (oGetData      := Nil, FreeObj( oGetData ) ), Nil)
800 IIf (ValType(oLblValor)     == '0', (oLblValor     := Nil, FreeObj( oLblValor ) ), Nil)
801 IIf (ValType(oGetValor)     == '0', (oGetValor     := Nil, FreeObj( oGetValor ) ), Nil)
802 IIf (ValType(oLblParcels)   == '0', (oLblParcels   := Nil, FreeObj( oLblParcels )), Nil)
803 IIf (ValType(oGetParcels)   == '0', (oGetParcels   := Nil, FreeObj( oGetParcels )), Nil)
804 IIf (ValType(oBtnOk)        == '0', (oBtnOk        := Nil, FreeObj( oBtnOk ) ), Nil)
805 IIf (ValType(oBtnCa)        == '0', (oBtnCa        := Nil, FreeObj( oBtnCa ) ), Nil)
806 IIf (ValType(oLblAdmFin)    == '0', (oLblAdmFin    := Nil, FreeObj( oLblAdmFin ) ), Nil)
807 IIf (ValType(oCboAdmFin)    == '0', (oCboAdmFin    := Nil, FreeObj( oCboAdmFin ) ), Nil)
808 IIf (ValType(oLblNSU)       == '0', (oLblNSU       := Nil, FreeObj( oLblNSU ) ), Nil)
809 IIf (ValType(oGetNSU)       == '0', (oGetNSU       := Nil, FreeObj( oGetNSU ) ), Nil)
810 IIf (ValType(oLblAutoiz)    == '0', (oLblAutoiz    := Nil, FreeObj( oLblAutoiz ) ), Nil)
811 IIf (ValType(oGetAutoriz)   == '0', (oGetAutoriz   := Nil, FreeObj( oGetAutoriz )), Nil)
812 IIf (ValType(oGetJurAdm)    == '0', (oGetJurAdm    := Nil, FreeObj( oGetJurAdm ) ), Nil)
813 IIf (ValType(oLblAdmFin)    == '0', (oLblAdmFin    := Nil, FreeObj( oLblAdmFin ) ), Nil)
814
815 If lAll
816     cGetData := ""
817     cGetVal := ""
818     cGetNSU := ""
819     cGetAutoriz := ""
820     nGetParc := 1
821     aAdmFin := {}
822     nCards := 1
823     nGetJurAdm := 0

```

```

824 EndIf
825
826 Return .T.
827 //-----
828 /*{Protheus.doc} STISetnCards
829 Função para Setar a variável contadora de cartão
830 @param
831 @return      nCards
832 @obs
833 @sample
834 */
835 //-----
836 Function STISetnCards(nCard)
837 Default nCard := 1
838
839 nCards := nCard
840
841 Return nCards
842
843 /*{Protheus.doc} STIGetAJur
844 Retorna o array aJurosAdm
845
846 @param
847 @return      aJurosAdm
848 @obs
849 @sample
850 */
851 Function STIGetAJur()
852 Return aJurosAdm
853
854 /*{Protheus.doc} STISetAJur
855 Seta valor ao array aJurosAdm
856
857 @param
858 @param aJuros, array, dados dos juros da Adm
859 @return
860 @obs
861 @sample
862 */
863 Function STISetAJur(aJuros)
864
865 Default aJuros := {.F., "", 0, 0}
866
867 aJurosAdm := aJuros
868
869 Return
870
871
872 /*{Protheus.doc} STIClrObj
873     Limpa os objetos da interface de Pagamento com Cartão
874     @type Function
875     */

```

```
876 Function STIClrObj()  
877  
878 If ValType(oModel) == '0'  
879     oModel:DeActivate()  
880     oModel := Nil  
881 Endif  
882  
883 STIClnVar(.T.)  
884  
885 Return
```

Componentes/Interface/STIPayCash.prw

245 linhas após remoção de metadados de autoria/data

```

1 #Include "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "POSCSS.CH"
7 #INCLUDE "STIPAYCASH.CH"
8 #INCLUDE "STPOS.CH"
9
10 Static oModel      := Nil
11 Static nVlOrc      := 0
12
13 //-----
14 /*/{Protheus.doc} STIPayCash
15
16 @param
17 @return
18 @obs
19 @sample
20 */
21 //-----
22 Function STIPayCash(oPnlAdconal)
23 Local oLblValor      := Nil           //Objeto do label 'Valor do Pagamento'
24 Local oGetValor      := Nil           //Objeto do get 'Valor do Pagamento'
25 Local cGetVal        := IIF(nVlOrc == 0,STBCalcSald("1"),nVlOrc) //Apresenta o saldo restante do pagamento e recebe valor de oGetValor
26
27 Local oBtnOk         := Nil           //Objeto do botao Ok
28 Local oBtnCa         := Nil           //Objeto do botao Cancelar
29
30 Local lReadOnly      := FindFunction("STIGetPayR0") .AND. STIGetPayR0() //indica se os campos de pagamento estão como Somente Leitura (permissão Alterar Parcelas do caixa)
31
32 Default oPnlAdconal := Nil
33
34 /* Label: 'Valor do Pagamento' */
35 oLblValor := TSay():New(POSVERT_PAYLABEL1+1, POSHOR_PAYCOL1+5,,
36                        {||STR0001}, oPnlAdconal,,;
37                        ,,,T.,;
38                        ,,,8) //'Valor do Pagamento em Dinheiro'
39
40 /* Get: Valor */
41 oGetValor := TGet():New(POSVERT_PAYGET1+1 ,POSHOR_PAYCOL1+5,{|u| If(PCount())>0,cGetVal:=u,cGetVal)},oPnlAdconal,,
42                        LARG_GET_VALOR,ALTURAGET,"@E 99,999,999,999.99",,,;
43                        ,,,,;
44                        .T.,,,,,,lReadOnly,,,,,,.T.)
45 oGetValor:SetFocus()
46
47 oLblValor:SetCSS( POSCSS (GetClassName(oLblValor), CSS_LABEL_FOCAL ))
48 oGetValor:SetCSS( POSCSS (GetClassName(oGetValor), CSS_GET_FOCAL ))
49
50 /* Button: OK */
51 oBtnOk := TButton():New(POSVERT_BTNPAY,POSHOR_BTNCONFPAY,STR0002,oPnlAdconal,,
52                        {|| STICScnfPay(oGetValor, oPnlAdconal)},LARGBTN,ALTURABTN,,;

```

```

53             ,,,T.) //'Efetuar Pagamento'
54
55 /* Button: Cancelar */
56 oBtnCa := TButton():New(POSVERT_BTNPAY,00,STR0003,oPnlAdconal,;
57             {|| IIF(ExistFunc("STBCancPay"),IIF(STBCancPay(),STIPayCancel(oPnlAdconal),Nil),STIPayCancel(oPnlAdconal))},LARGBTN,ALTURABTN,,;
58             ,,,T.) //'Cancelar'
59
60 oBtnOk:SetCSS( POSCSS(GetClassName(oBtnOk), CSS_BTN_FOCAL) )
61 oBtnCa:SetCSS( POSCSS(GetClassName(oBtnCa), CSS_BTN_ATIVO) )
62
63 nVlOrc := 0
64
65 Return .T.
66
67
68 //-----
69 /*{Protheus.doc} ModelCash
70
71 @param
72 @return
73 @obs
74 @sample
75 */
76 //-----
77 Function ModelCash()
78
79 Local oStruMst := FWFormModelStruct():New()
80
81 If ValType(oModel) == '0'
82     oModel:DeActivate()
83     oModel := Nil
84 EndIf
85
86 oModel := MPFormModel():New('STIPayCash')
87 oModel:SetDescription(STR0009) //"Pagamento Dinheiro"
88
89 oStruMst:AddTable("SL4",{ "L4_FILIAL"},STR0004) //"Pagamento"
90
91 oStruMst := STIStuMod(oStruMst)
92 oModel:AddFields( 'CASHMASTER', Nil, oStruMst)
93 oModel:GetModel ( 'CASHMASTER' ):SetDescription(STR0005) //"Selecionar Pagamento"
94
95 Return oModel
96
97
98 //-----
99 /*{Protheus.doc} STIStuMod
100
101 @param
102 @return
103 @obs
104 @sample

```

```

105 /*/
106 //-----
107 Static Function STIStruMod(oStruMst)
108
109 Default oStruMst      := Nil
110
111 oStruMst:AddField( STR0006      ,; //[01] Titulo do campo
112                   STR0006      ,; //[02] Desc do campo
113                   "L4_FILIAL" ,; //[03] Id do Field
114                   "C"         ,; //[04] Tipo do campo
115                   8           ,; //[05] Tamanho do campo
116                   0           ,; //[06] Decimal do campo
117                   Nil         ,; //[07] Code-block de validacao do campo
118                   Nil         ,; //[08] Code-block de validacao When do campo
119                   Nil         ,; //[09] Lista de valores permitido do campo
120                   Nil         ,; //[10] Indica se o campo tem preenchimento obrigatorio
121                   Nil         ,; //[11] Code-block de inicializacao do campo
122                   Nil         ,; //[12] Indica se trata-se de um campo chave
123                   Nil         ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
124                   .T.         ) //[14] Indica se o campo e virtual
125
126 oStruMst:AddField( STR0007      ,; //[01] Titulo do campo
127                   STR0007      ,; //[02] Desc do campo
128                   "L4_DATA"    ,; //[03] Id do Field
129                   "D"         ,; //[04] Tipo do campo
130                   8           ,; //[05] Tamanho do campo
131                   0           ,; //[06] Decimal do campo
132                   Nil         ,; //[07] Code-block de validacao do campo
133                   Nil         ,; //[08] Code-block de validacao When do campo
134                   Nil         ,; //[09] Lista de valores permitido do campo
135                   .T.         ,; //[10] Indica se o campo tem preenchimento obrigatorio
136                   FwBuildFeature( STRUCT_FEATURE_INIPAD,"dDataBase" ) ,; //[11] Code-block de inicializacao do campo
137                   Nil         ,; //[12] Indica se trata-se de um campo chave
138                   Nil         ,; //[13] Indica se o campo pode receber valor em uma operacao de update
139                   .F.         ) //[14] Indica se o campo e virtual
140
141
142 oStruMst:AddField( STR0008 ,; //[01] Titulo do campo
143                   STR0008 ,; //[02] Desc do campo
144                   "L4_VALOR",; //[03] Id do Field
145                   "N"     ,; //[04] Tipo do campo
146                   16      ,; //[05] Tamanho do campo
147                   2       ,; //[06] Decimal do campo
148                   Nil     ,; //[07] Code-block de validacao do campo
149                   Nil     ,; //[08] Code-block de validacao When do campo
150                   Nil     ,; //[09] Lista de valores permitido do campo
151                   Nil     ,; //[10] Indica se o campo tem preenchimento obrigatorio
152                   Nil     ,; //[11] Code-block de inicializacao do campo
153                   Nil     ,; //[12] Indica se trata-se de um campo chave
154                   Nil     ,; //[13] Indica se o campo pode receber valor em uma operacao de update
155                   .T.     ) //[14] Indica se o campo e virtual
156

```

```

157 Return oStruMst
158
159 //-----
160 /*{Protheus.doc} STICSCnfPay
161 Funcao responsavel por confirmar o pagamento em dinheiro
162
163 @param
164 @return
165 @obs
166 @sample
167 */
168 //-----
169 Function STICSCnfPay(oGetValor, oPnlAdconal)
170
171 Local oMdl          := Nil          //Model do cash
172 Local oListBox      := Nil //Objeto list box da tela principal do pagamento
173 Local aPgto        := {}
174 Local lMobile       := STFGetCfg("lMobile", .F.) //Smart Client Mobile
175
176 lMobile := ValType(lMobile) == "L" .AND. lMobile
177
178 /* Cria estrutura do model */
179 ModelCash()
180 oMdl := oModel:GetModel("CASHMASTER")
181
182 /* Desativa e ativa o model */
183 oMdl:DeActivate()
184 oMdl:Activate()
185
186 /* Inclui o valor no model */
187 oMdl:LoadValue("L4_DATA", dDataBase)
188 oMdl:LoadValue("L4_VALOR", oGetValor:cText)
189
190 /* Adiciona o */
191 STIAddPay("R$", oMdl, 1,,,oGetValor:cText, @aPgto)
192
193 oPnlAdconal:Hide()
194
195 If !lMobile
196     oListBox := STIGetLstB()
197     oListBox:SetFocus()
198 EndIf
199
200 STIEnblPaymentOptions()
201
202 Return .T.
203
204 //-----
205 /*{Protheus.doc} STICS0rc
206 Chamada somente quando for importacao de orcamento
207
208 @param

```

```

209 @return
210 @obs
211 @sample
212 /*/
213 //-----
214 Function STICS0rc(dDate, nValor)
215
216 Local oObjPanPay := NIL
217 Local oRaas      := Nil
218 Local oFidelityC := Nil
219
220 nVl0rc      := nValor
221
222 If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
223     oRaas := STBGetRaas()
224
225     If Valtype(oRaas) == "0" .AND. oRaas:ServiceIsActive("TFC")
226
227         oFidelityC := oRaas:GetFidelityCore()
228
229         If oFidelityC:ChoseToUse()
230             nVl0rc -= oFidelityC:GetBonus()
231         EndIf
232     EndIf
233 EndIf
234 EndIf
235
236 oObjPanPay := STIGetPan()
237
238 If oObjPanPay == NIL
239     STIPayment(.F.)
240     oObjPanPay := STIGetPan()
241 EndIf
242
243 STIAddNewPan( oObjPanPay , "R$")
244
245 Return .T.

```